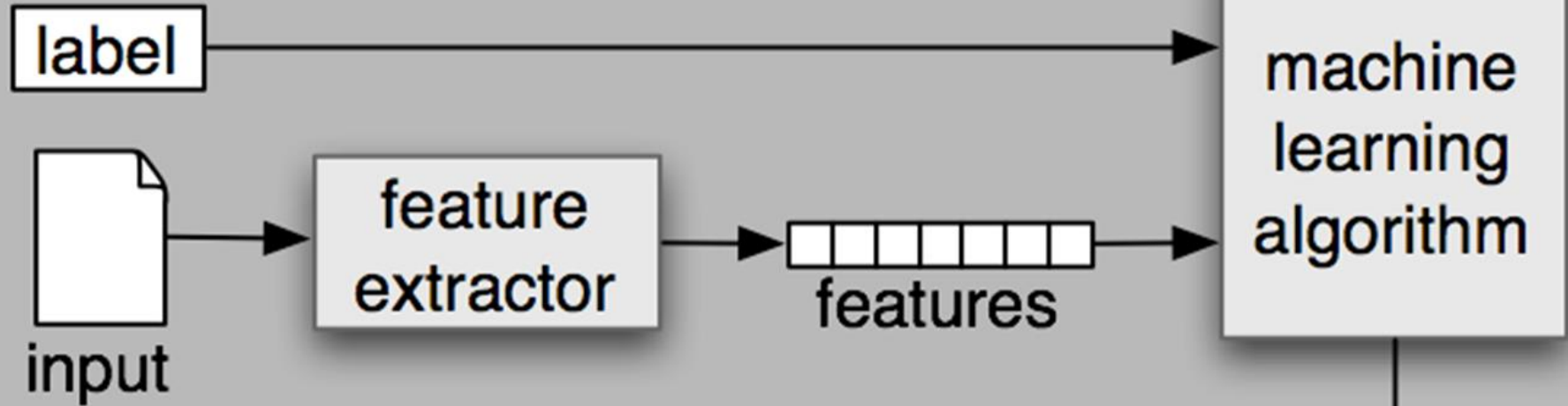


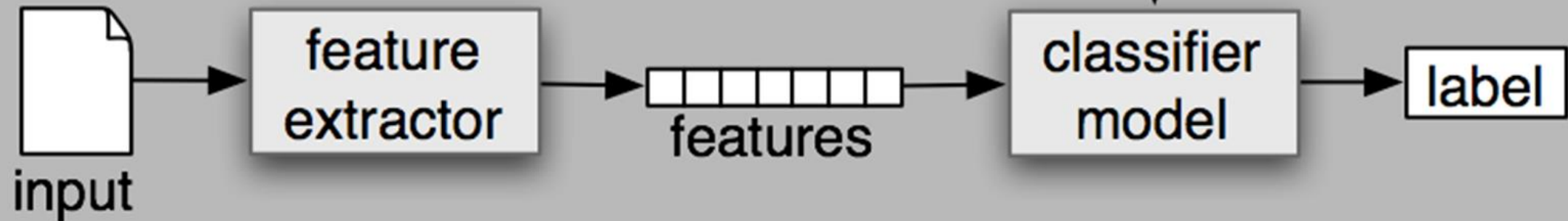
Module 3

Recap

(a) Training

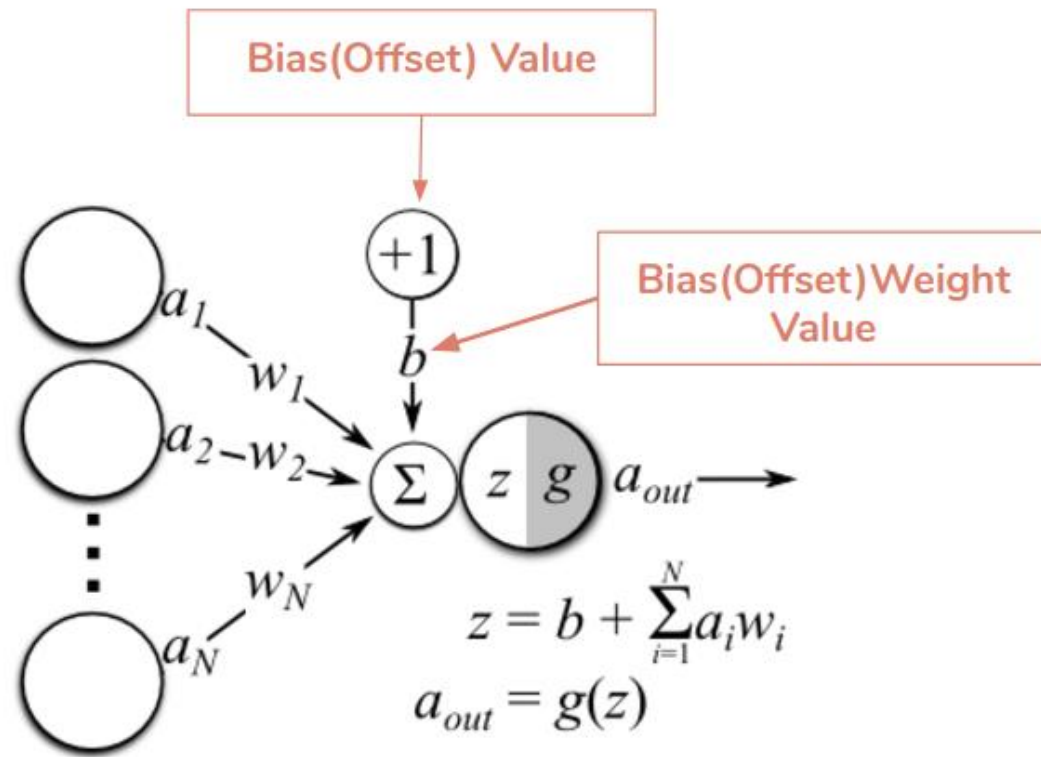


(b) Prediction



Recap (module 2)

- ML pipeline
- Representation learning → Deep learning
- Training algorithm → Optimization method
- Loss function, logit, activation function (softmax)
- Gradient Descent



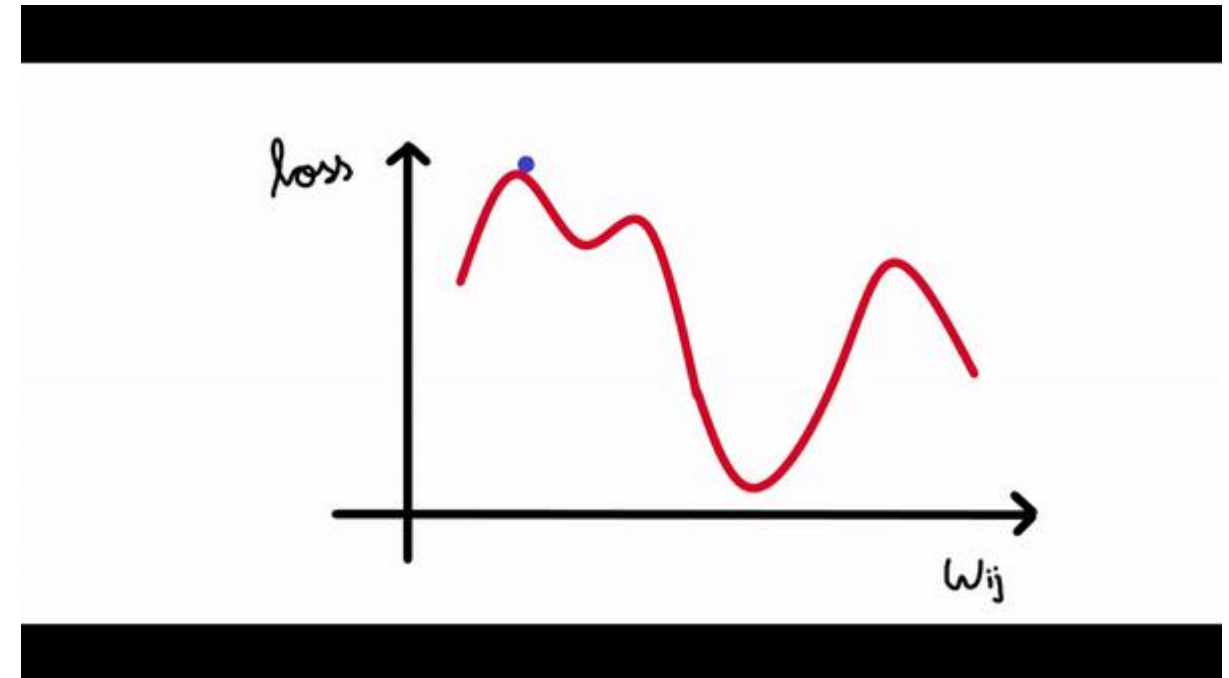
Gradient Descent is used to update the parameters of a computational node (neuron)

```
new_weight = old_weight - learning_rate * gradient
```

```
new_bias = old_bias - learning_rate * gradient
```

Gradient Descent Intuition

The partial derivative in GD tells us how a small change in any parameter affects the loss (error), and gradient descent uses this information to update parameters in the direction that makes the loss smaller.

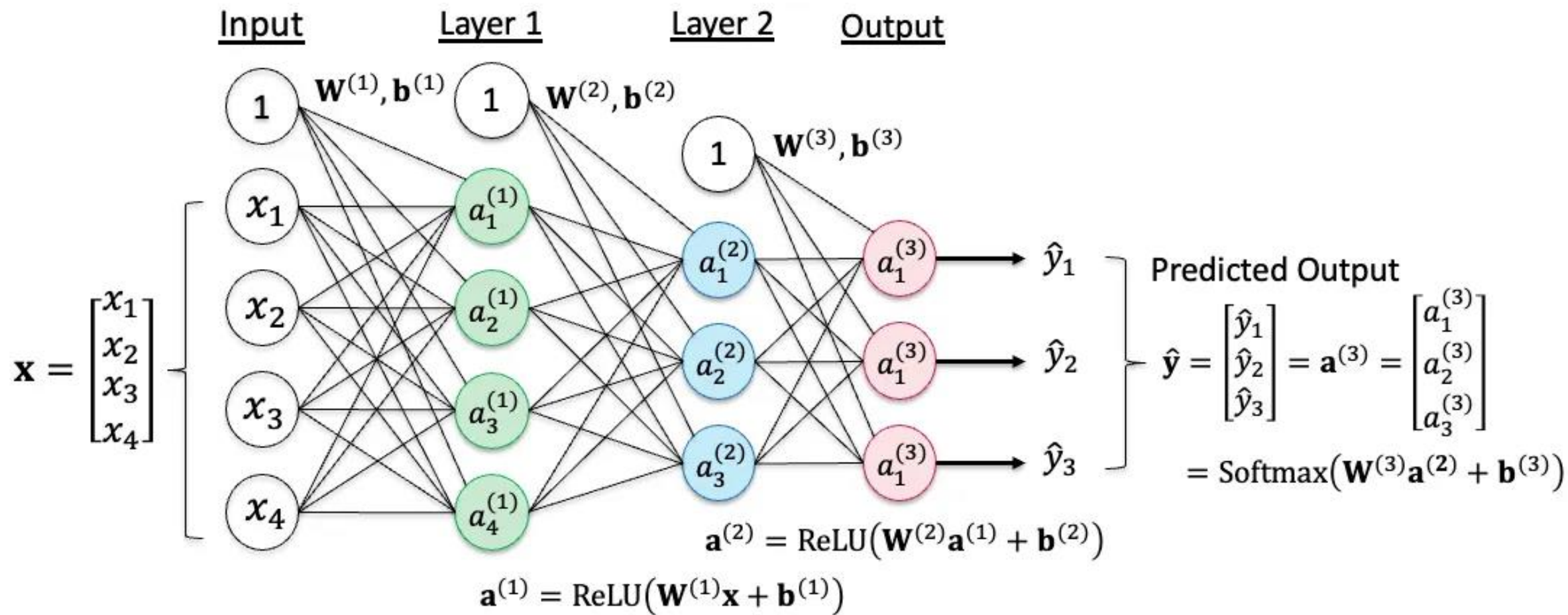


The **gradient** tells us **how this error changes** when we slightly adjust a parameter (e.g., weight or bias).

Thus,

$$\text{Error Gradient} = \frac{\partial L}{\partial \text{parameter}}$$

It is the **rate of change** of the error with respect to that parameter.



Gradient Descent Algorithm

Step 0: Randomly initialize weights and biases parameters: $\theta = \{\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \mathbf{W}^{(2)}, \mathbf{b}^{(2)}, \dots, \mathbf{W}^{(L)}, \mathbf{b}^{(L)}\}$

Step 1: Compute the cost function $\mathcal{L}(\theta)$, which measures how well the model is performing on the dataset.

Step 2: Find the gradients of the cost function with respect to each parameter $\nabla_{\theta} \mathcal{L}(\theta_t)$

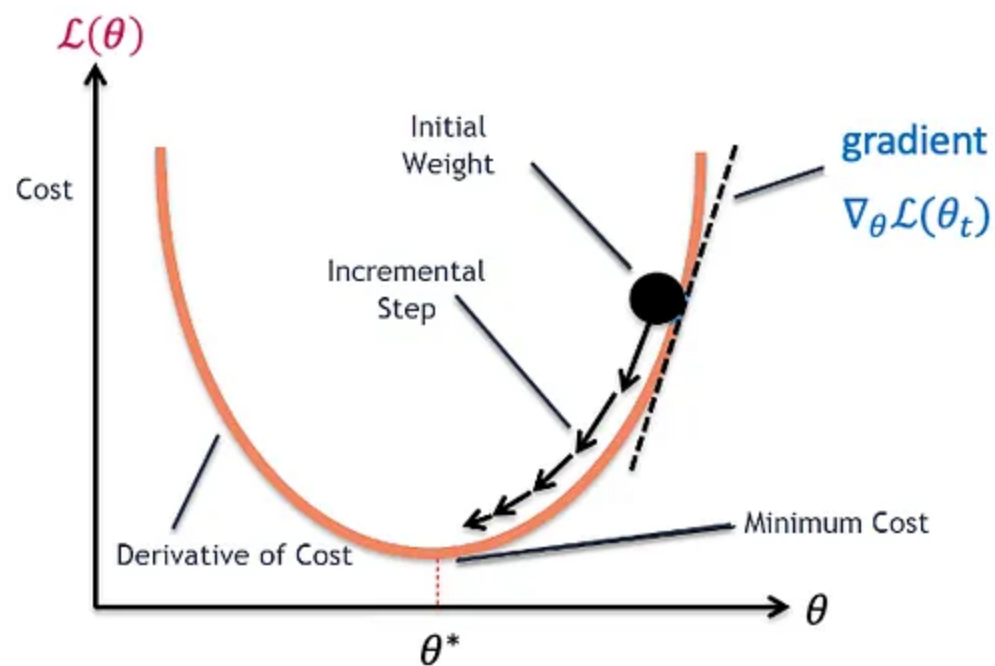
Step 3: Update the parameters by

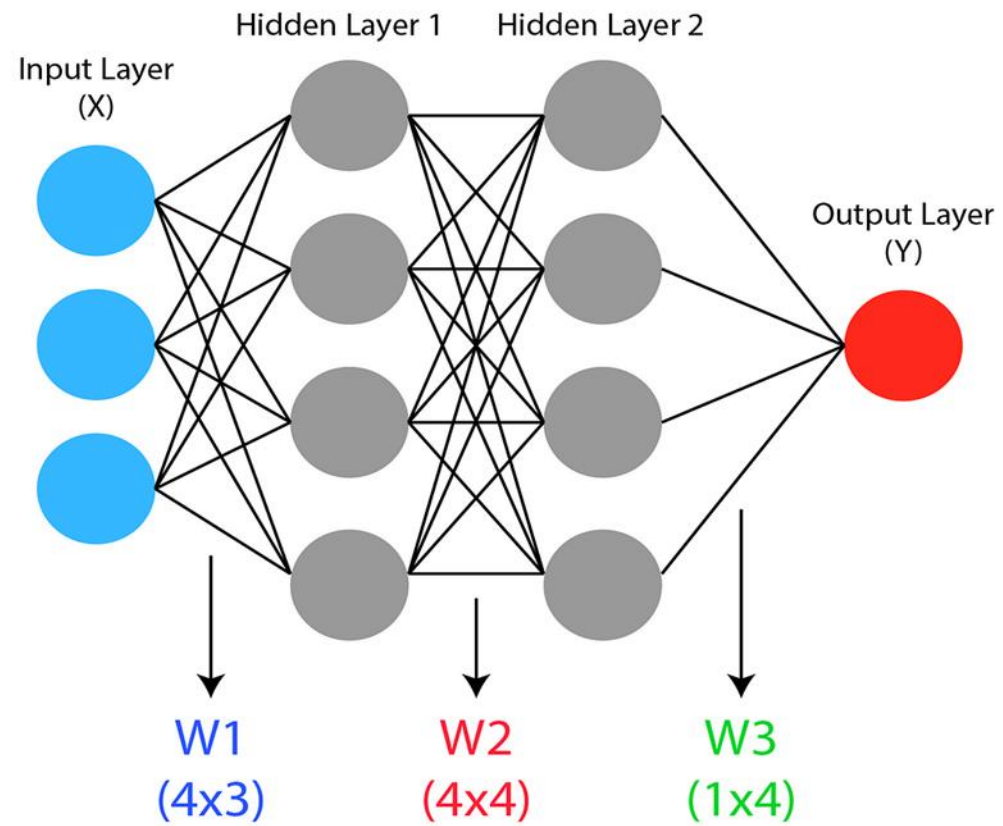
$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} \mathcal{L}(\theta_t)$$

where η is the **learning rate** that determines how big the updates should be in each iteration t .

- Repeat the above steps 1 to 3, until the cost is low enough or convergence.

$$\theta^* = \min_{\theta} \mathcal{L}(\theta) = \min_{\theta} \frac{1}{N} \sum_{i=1}^N \ell(y^{(i)}, f_{\theta}(\mathbf{x}^{(i)}))$$

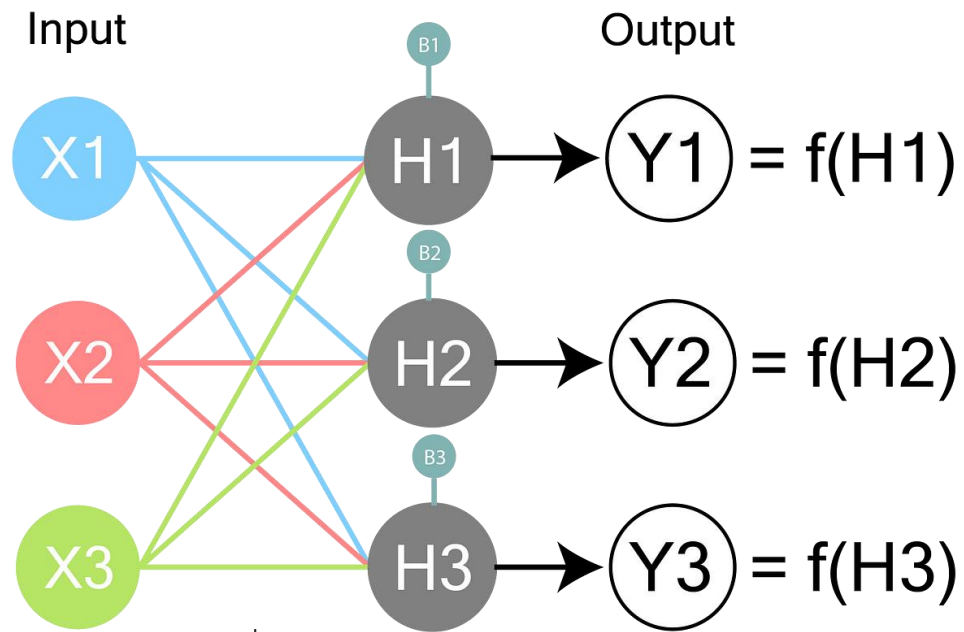




$$Y = f(W3 * f(W2 * f(W1 * X + B1) + B2) + B3)$$

Diagram illustrating the forward pass calculation for the output layer (Y) based on the network structure:

- Input Layer:** X
- Hidden Layer 1:** $f(W1 * X + B1)$
- Hidden Layer 2:** $f(W2 * f(W1 * X + B1) + B2)$
- Output layer (Y):** $f(W3 * f(W2 * f(W1 * X + B1) + B2) + B3)$



Intermediate Node Values

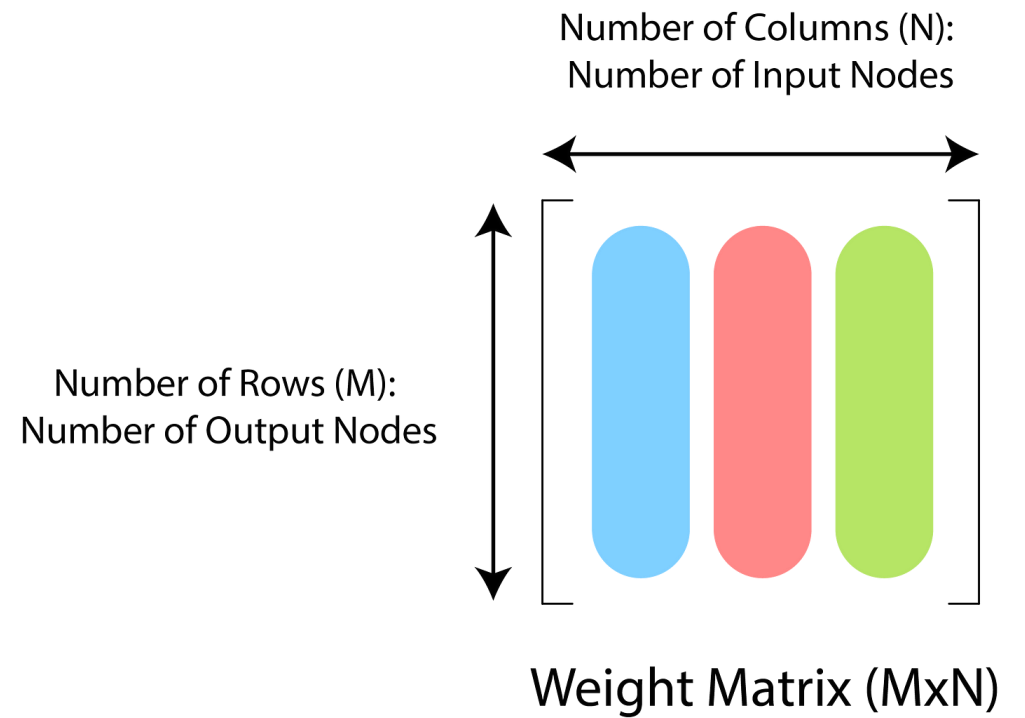
Blue Weights Red Weights Green Weights

Input Vector Bias Vector

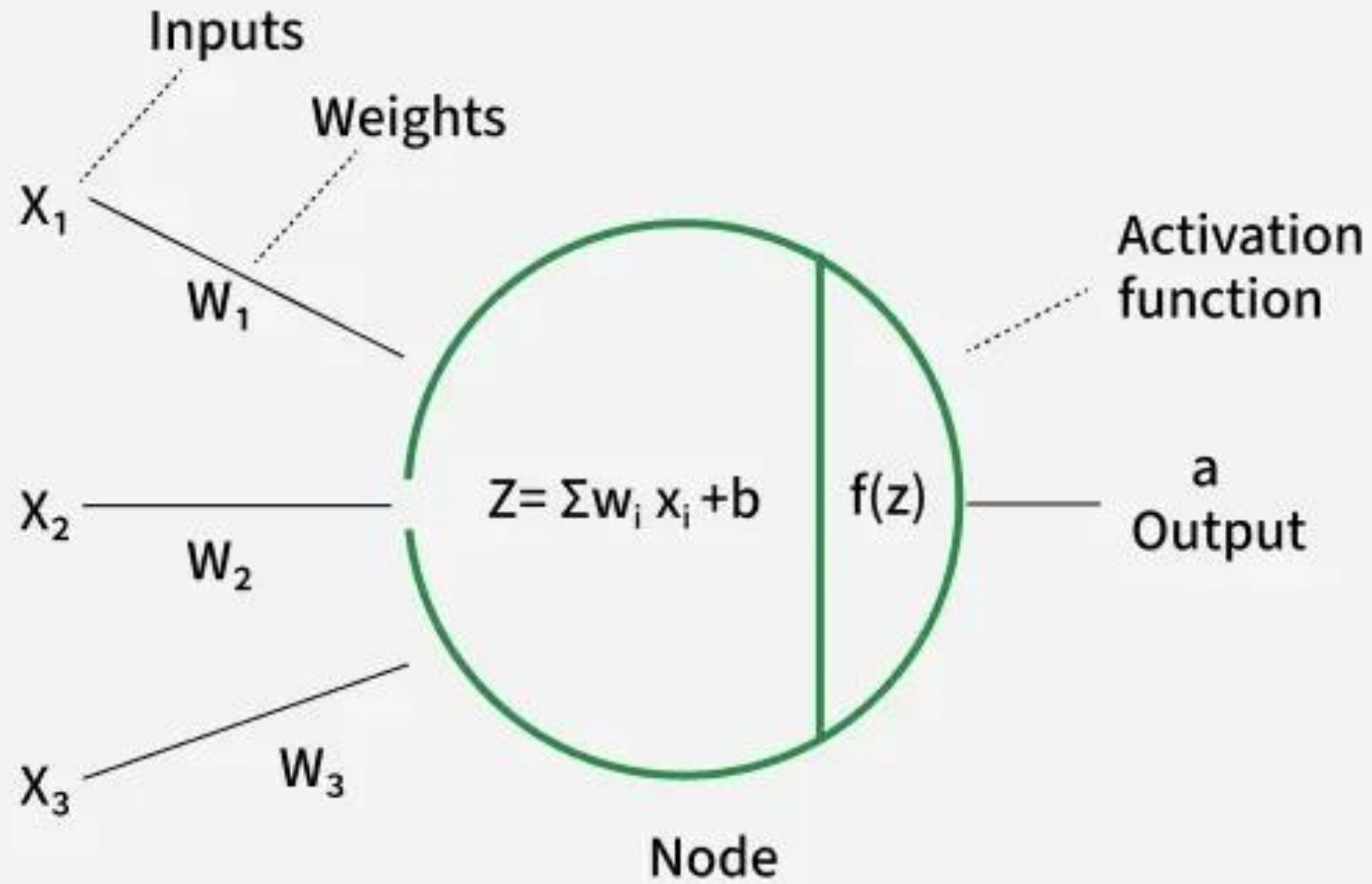
Weight Matrix

Matrix Multiplication

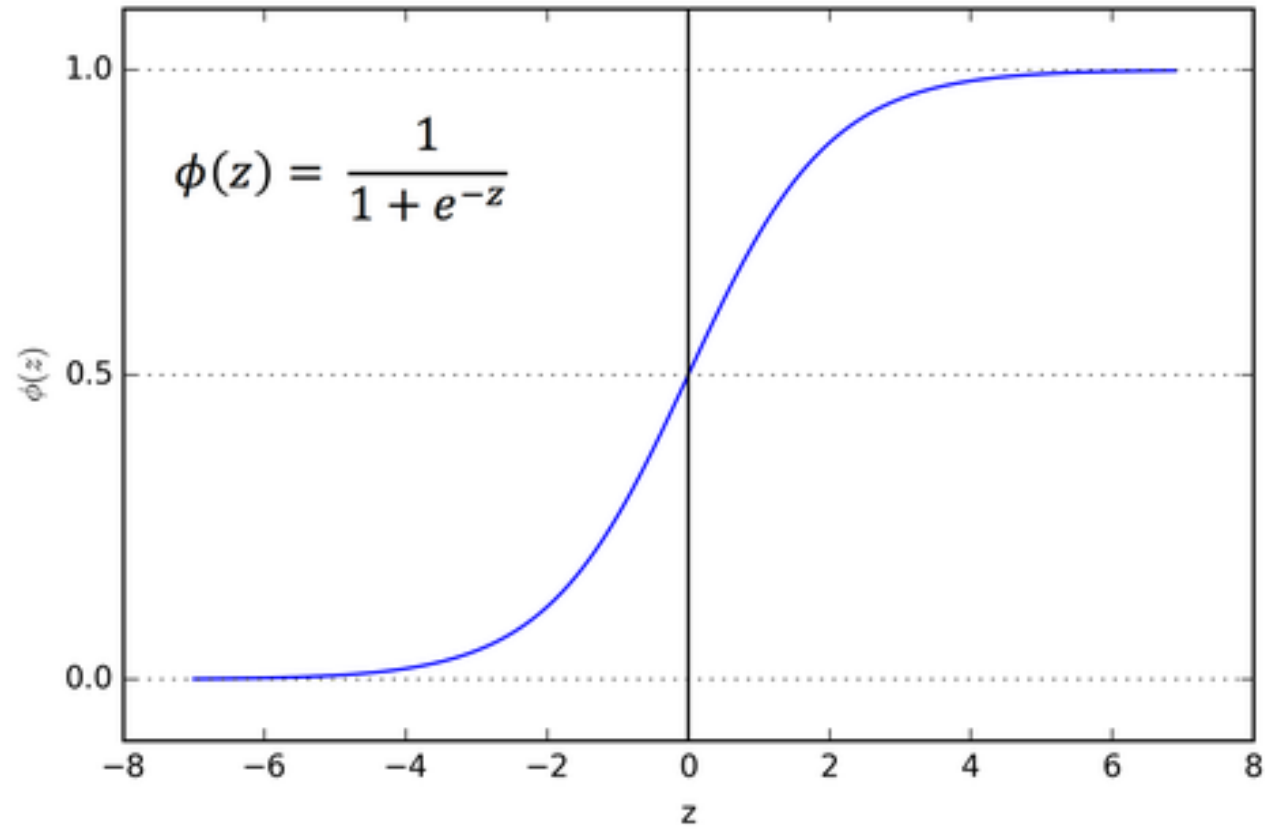
$$\begin{bmatrix} H1 \\ H2 \\ H3 \end{bmatrix} = \begin{bmatrix} \text{Blue Weights} & \text{Red Weights} & \text{Green Weights} \end{bmatrix} \times \begin{bmatrix} X1 \\ X2 \\ X3 \end{bmatrix} + \begin{bmatrix} B1 \\ B2 \\ B3 \end{bmatrix}$$



Activation functions in Neural Networks



Sigmoid/Logistic function



Let's do some simple math...

Definition of Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

We want:

$$\frac{d\sigma(z)}{dz} = ?$$

$$\frac{d\sigma}{dz} = -(1 + e^{-z})^{-2} \cdot (-e^{-z})$$

Simplify:

$$\frac{d\sigma}{dz} = \frac{e^{-z}}{(1 + e^{-z})^2}$$

Express in Terms of $\sigma(z)$

Express in Terms of $\sigma(z)$

We know:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad \Rightarrow \quad 1 - \sigma(z) = \frac{e^{-z}}{1 + e^{-z}}$$

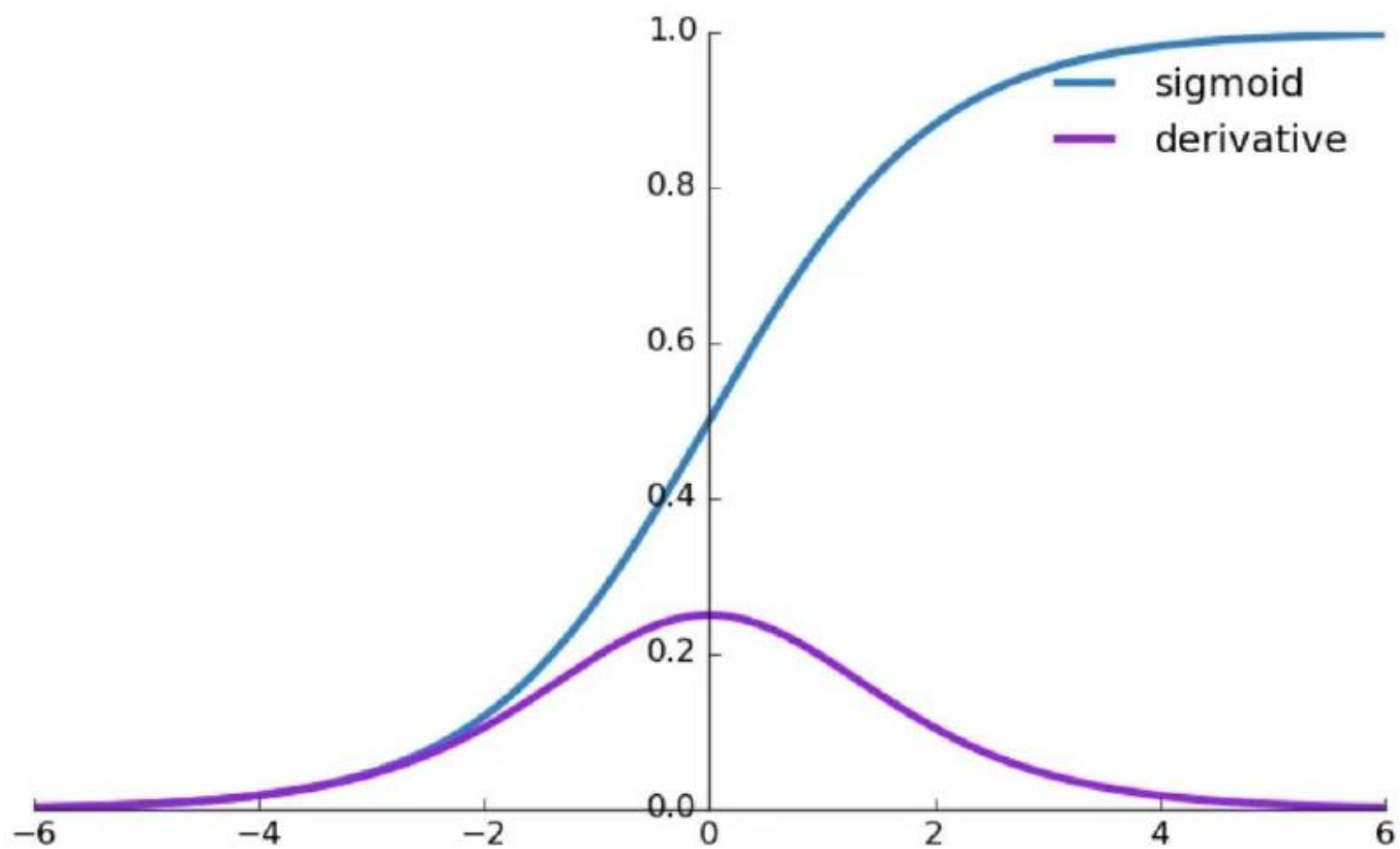
Now substitute these into the derivative expression:

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$$

Draw the graph of the derivative of the sigmoid w.r.t z

- When $z = 0$:
 $\sigma(0) = 0.5 \Rightarrow \sigma'(0) = 0.25$ (maximum slope)
- As $z \rightarrow +\infty$ or $z \rightarrow -\infty$:
 $\sigma'(z) \rightarrow 0$ (flat saturation zones)

Graph of sigmoid derivative



Let's consider a simple ML problem ...

- A regression problem with two features and one output:

Example: Predicting Crop Yield

Features (Inputs)

1. Rainfall (mm) $\rightarrow x_1$
2. Fertilizer used (kg per hectare) $\rightarrow x_2$

Output (Target)

- Crop yield (tons per hectare) $\rightarrow y$

Dataset (Example)

Rainfall (mm)	Fertilizer (kg/ha)	Yield (tons/ha)
500	100	2.1
600	120	2.8
700	130	3.2
800	150	3.7
900	160	4.0
1000	170	4.2

Problem Setup

Inputs:

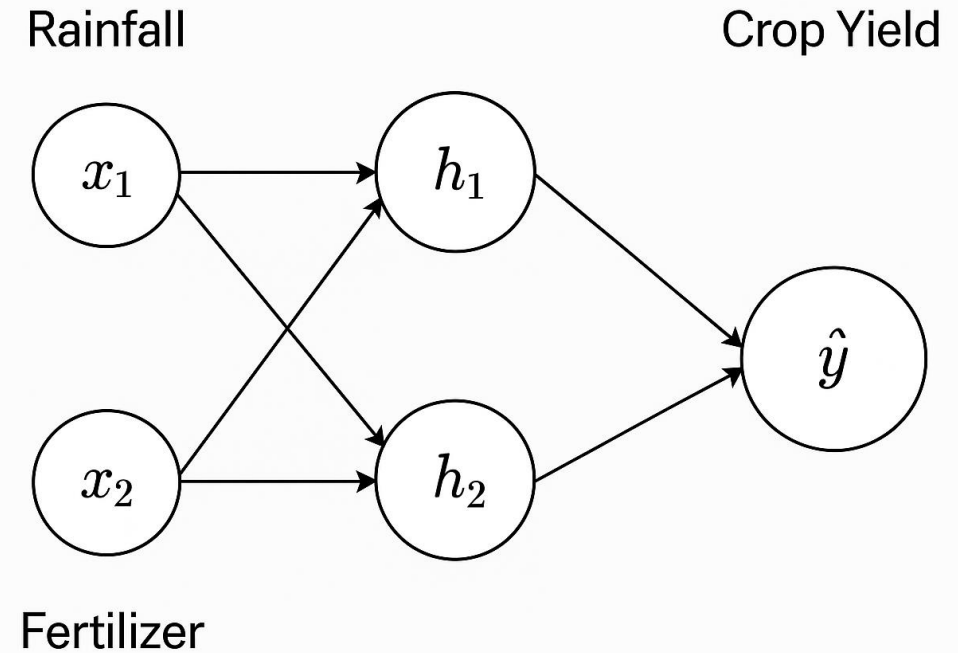
- x_1 : Rainfall (mm)
- x_2 : Fertilizer (kg/ha)

Output:

- y : Crop yield (tons/ha)

We'll design a 2-layer neural network:

1. Hidden layer: 2 neurons
2. Output layer: 1 neuron



Neural network parameters

Layer 1 (Hidden layer, 2 neurons)

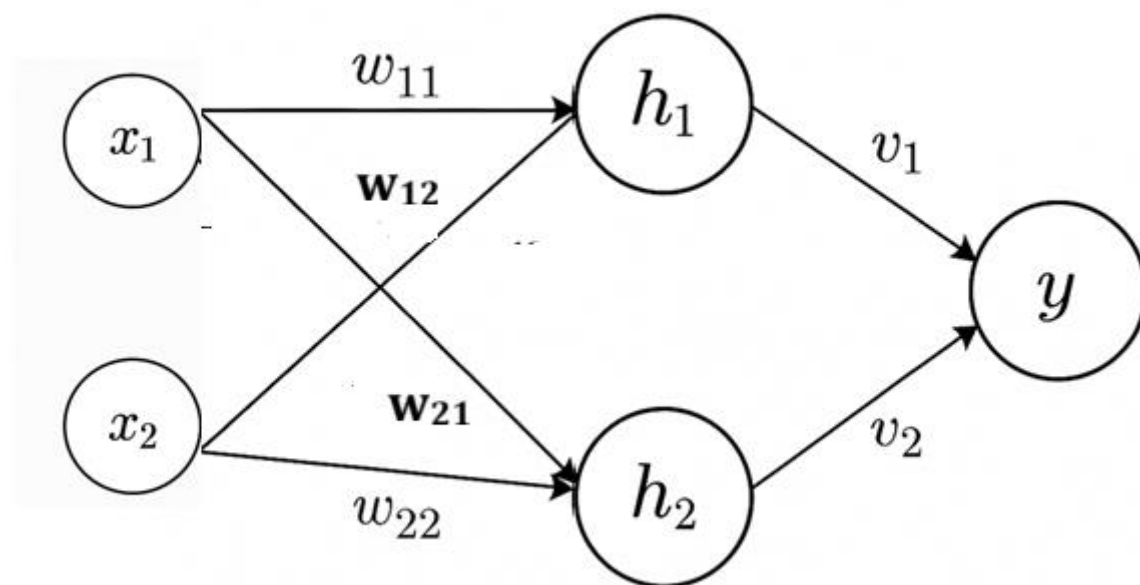
- Weights: $W^{[1]} \in \mathbb{R}^{2 \times 2}$
- Bias: $b^{[1]} \in \mathbb{R}^2$

Layer 2 (Output layer, 1 neuron)

- Weights: $W^{[2]} \in \mathbb{R}^{1 \times 2}$
- Bias: $b^{[2]} \in \mathbb{R}^1$

Given:

- $z \rightarrow$ **pre-activation** (the weighted sum before activation)
- $g(\cdot) \rightarrow$ **activation function**
- $h \rightarrow$ **activation (output)** of that neuron



Relationship between z , g , and h

$$h = g(z)$$

and since the pre-activation is a weighted sum:

$$z = \sum_i w_i x_i + b$$

where

- x_i = inputs from previous layer,
- w_i = corresponding weights,
- b = bias term.



Input Layer

Inputs:

$$x_1, x_2$$

(These are given — no parameters here.)

Hidden Layer (Layer 1)

Each neuron computes:

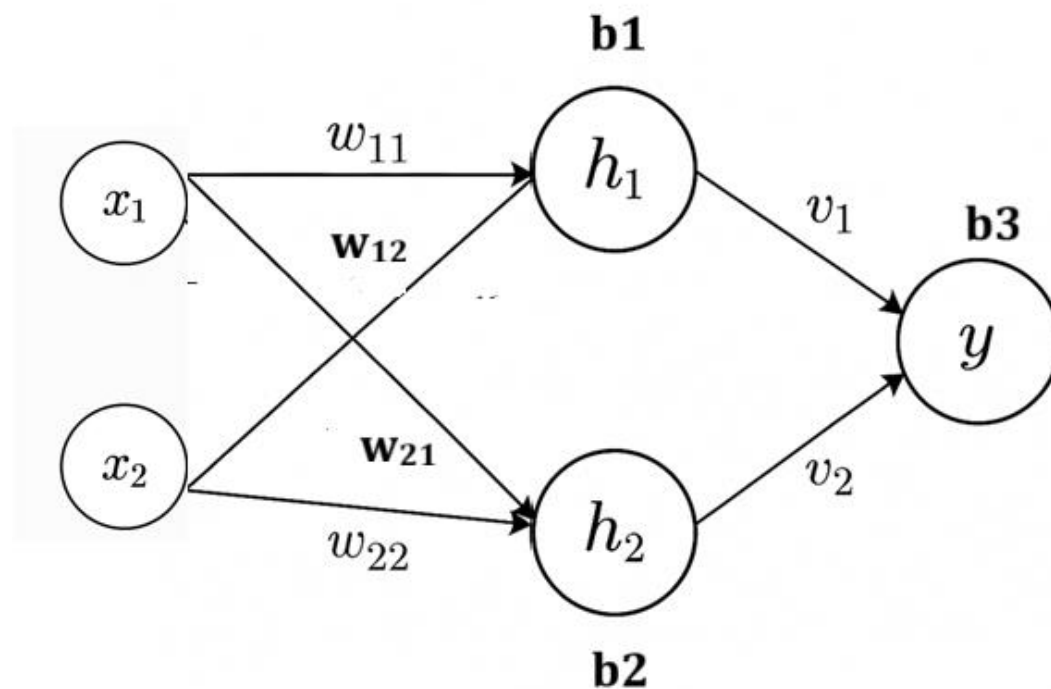
$$z_1 = w_{11}x_1 + w_{12}x_2 + b_1$$

$$z_2 = w_{21}x_1 + w_{22}x_2 + b_2$$

Then apply activation $g(\cdot)$:

$$h_1 = g(z_1)$$

$$h_2 = g(z_2)$$



So in compact form:

$$\mathbf{z}^{(1)} = W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}, \quad \mathbf{h}^{(1)} = g(\mathbf{z}^{(1)})$$

where

$$W^{(1)} = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

Output Layer (Layer 2)

Pre-activation:

$$z^{(2)} = v_1 h_1 + v_2 h_2 + b_3$$

Output activation (for regression, linear activation):

$$\hat{y} = g^{(2)}(z^{(2)}) = z^{(2)}$$

Neural Network Training Steps

1. Initialize all weights and biases with small random values.
2. For each epoch (1 to 100):
 - a. Perform forward propagation to compute predicted outputs.
 - b. Compute loss (Mean Squared Error).
 - c. Calculate gradients w.r.t all parameters
 - d. Update parameters (weights and biases) using gradient descent.
3. Repeat until all 100 epochs are completed.
4. Output final weights and use the trained model for prediction.

Forward pass (per sample)

1. Compute z_1, h_1
2. Compute z_2, h_2
3. Compute $\hat{y}$
4. Compute L

Forward pass (per sample)

1. Compute z_1, h_1
2. Compute z_2, h_2
3. Compute $\hat{y}$
4. Compute L

- $z_1 = w_{11}x_1 + w_{12}x_2 + b_1, h_1 = g(z_1)$
- $z_2 = w_{21}x_1 + w_{22}x_2 + b_2, h_2 = g(z_2)$
- $\hat{y} = v_1h_1 + v_2h_2 + b_3$
- $L = \frac{1}{2}(\hat{y} - y)^2$

So gradient descent needs....

How many partial derivatives?

List of all parameters

Layer	Parameters	Count
Input → Hidden	$w_{11}, w_{12}, w_{21}, w_{22}, b_1, b_2$	6
Hidden → Output	v_1, v_2, b_3	3
Total parameters	—	9

Hence, 9 derivatives to compute:

$$\begin{array}{cccc}\frac{\partial L}{\partial w_{11}}, & \frac{\partial L}{\partial w_{12}}, & \frac{\partial L}{\partial w_{21}}, & \frac{\partial L}{\partial w_{22}}, \\ \frac{\partial L}{\partial b_1}, & \frac{\partial L}{\partial b_2}, & & \\ \frac{\partial L}{\partial v_1}, & \frac{\partial L}{\partial v_2}, & \frac{\partial L}{\partial b_3}.\end{array}$$

Each of these gradients is used in the parameter update rule:

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}$$

where η is the learning rate.

Hidden neuron 1

- $z_1 = w_{11}x_1 + w_{12}x_2 + b_1$
- $h_1 = \text{ReLU}(z_1) = \max(0, z_1)$

Hidden neuron 2

- $z_2 = w_{21}x_1 + w_{22}x_2 + b_2$
- $h_2 = \text{ReLU}(z_2)$

Output neuron

- $\hat{y} = v_1h_1 + v_2h_2 + b_3$

Loss (per sample)

- $L = \frac{1}{2}(\hat{y} - y)^2$

9 derivatives to compute:

$$\begin{array}{cccc}\frac{\partial L}{\partial w_{11}}, & \frac{\partial L}{\partial w_{12}}, & \frac{\partial L}{\partial w_{21}}, & \frac{\partial L}{\partial w_{22}}, \\ \frac{\partial L}{\partial b_1}, & \frac{\partial L}{\partial b_2}, & & \\ \frac{\partial L}{\partial v_1}, & \frac{\partial L}{\partial v_2}, & \frac{\partial L}{\partial b_3}.\end{array}$$

Neural Network = a Big Composite Function

In a neural network:

$$x_1, x_2 \xrightarrow{w_{ij}} z_j \xrightarrow{f(\cdot)} h_j \xrightarrow{v_j} \hat{y} \xrightarrow{L} L$$

The **loss** L depends on the output $\hat{y}$,

which depends on activations h_j ,

which depend on pre-activations z_j ,

which depend on weights w_{ij} and inputs x_i .

So L is a nested function:

$$L = L(\hat{y}(h_j(z_j(w_{ij}x_i))))$$

1. Build the dependency graph

For your 2-2-1 network:

$$x_1, x_2$$

↓

$$z_1 = w_{11}x_1 + w_{12}x_2 + b_1$$

$$z_2 = w_{21}x_1 + w_{22}x_2 + b_2$$

↓

$$h_1 = \sigma(z_1), \quad h_2 = \sigma(z_2)$$

↓

$$\hat{y} = v_1h_1 + v_2h_2 + b_3$$

↓

$$L = \frac{1}{2}(\hat{y} - y)^2$$

What is the Chain Rule?

The **chain rule** in calculus describes how to find the derivative of a *composite function* — that is, a function made up of other functions.

If

$$y = f(u) \quad \text{and} \quad u = g(x),$$

then

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}.$$

Chain Rule in the Network Context

To find how a weight affects the loss:

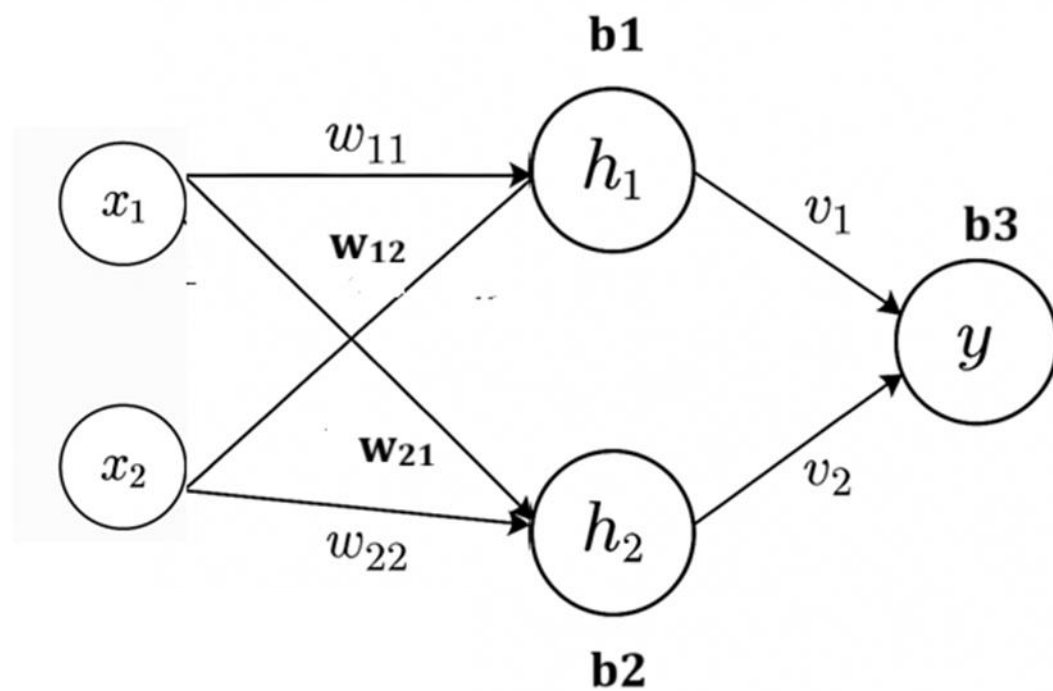
$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_j} \cdot \frac{\partial h_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ij}}.$$

Each term is a **local derivative** — how one node affects the next.

The product gives the **total effect** of w_{ij} on the loss.

Dependency-based gradient flow

Parameter	Dependency path from L	Chain rule expansion
v_1	$L \rightarrow \hat{y} \rightarrow v_1$	$\frac{\partial L}{\partial v_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial v_1}$
w_{11}	$L \rightarrow \hat{y} \rightarrow h_1 \rightarrow z_1 \rightarrow w_{11}$	$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial w_{11}}$
w_{12}	$L \rightarrow \hat{y} \rightarrow h_1 \rightarrow z_1 \rightarrow w_{12}$	same chain, replace w_{11} with w_{12}
w_{21}	$L \rightarrow \hat{y} \rightarrow h_2 \rightarrow z_2 \rightarrow w_{21}$	$\frac{\partial L}{\partial w_{21}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_2} \frac{\partial h_2}{\partial z_2} \frac{\partial z_2}{\partial w_{21}}$
b_1	$L \rightarrow \hat{y} \rightarrow h_1 \rightarrow z_1 \rightarrow b_1$	$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial b_1}$



9 derivatives to compute:

$$\begin{aligned} & \frac{\partial L}{\partial w_{11}}, \quad \frac{\partial L}{\partial w_{12}}, \quad \frac{\partial L}{\partial w_{21}}, \quad \frac{\partial L}{\partial w_{22}}, \\ & \frac{\partial L}{\partial b_1}, \quad \frac{\partial L}{\partial b_2}, \\ & \frac{\partial L}{\partial v_1}, \quad \frac{\partial L}{\partial v_2}, \quad \frac{\partial L}{\partial b_3}. \end{aligned}$$

Derivative of loss w.r.t. output

$$\frac{\partial L}{\partial \hat{y}} = (\hat{y} - y)$$

Derivative of output w.r.t. v_1

$$\frac{\partial \hat{y}}{\partial v_1} = h_1$$

$$\Rightarrow \frac{\partial L}{\partial v_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial v_1} = (\hat{y} - y) \cdot h_1$$

Derivative of output w.r.t. v_2

$$\frac{\partial \hat{y}}{\partial v_2} = h_2$$

$$\Rightarrow \frac{\partial L}{\partial v_2} = (\hat{y} - y) \cdot h_2$$

Derivative of output w.r.t. bias b_3

$$\frac{\partial \hat{y}}{\partial b_3} = 1$$

$$\Rightarrow \frac{\partial L}{\partial b_3} = (\hat{y} - y) \cdot 1 = (\hat{y} - y)$$

$$\frac{\partial L}{\partial v_1} = (\hat{y} - y)h_1$$

$$\frac{\partial L}{\partial v_2} = (\hat{y} - y)h_2$$

$$\frac{\partial L}{\partial b_3} = (\hat{y} - y)$$

Output-layer parameters

$$\frac{\partial L}{\partial v_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial v_1} = (\hat{y} - y) h_1$$

$$\frac{\partial L}{\partial v_2} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial v_2} = (\hat{y} - y) h_2$$

$$\frac{\partial L}{\partial b_3} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial b_3} = (\hat{y} - y)$$

Hidden neuron 1

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial w_{11}}$$

$$\frac{\partial L}{\partial w_{12}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial w_{12}}$$

$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial b_1}$$

Hidden neuron 2

$$\frac{\partial L}{\partial w_{21}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_2} \frac{\partial h_2}{\partial z_2} \frac{\partial z_2}{\partial w_{21}}$$

$$\frac{\partial L}{\partial w_{22}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_2} \frac{\partial h_2}{\partial z_2} \frac{\partial z_2}{\partial w_{22}}$$

$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_2} \frac{\partial h_2}{\partial z_2} \frac{\partial z_2}{\partial b_2}$$

Final Summary (All Gradients Together)

Output layer:

$$\frac{\partial L}{\partial v_1} = (\hat{y} - y) h_1, \quad \frac{\partial L}{\partial v_2} = (\hat{y} - y) h_2, \quad \frac{\partial L}{\partial b_3} = (\hat{y} - y)$$

Hidden layer:

$$\begin{aligned} \frac{\partial L}{\partial w_{11}} &= (\hat{y} - y) v_1 g'(z_1) x_1, & \frac{\partial L}{\partial w_{12}} &= (\hat{y} - y) v_1 g'(z_1) x_2, & \frac{\partial L}{\partial b_1} &= (\hat{y} - y) v_1 g'(z_1) \\ \frac{\partial L}{\partial w_{21}} &= (\hat{y} - y) v_2 g'(z_2) x_1, & \frac{\partial L}{\partial w_{22}} &= (\hat{y} - y) v_2 g'(z_2) x_2, & \frac{\partial L}{\partial b_2} &= (\hat{y} - y) v_2 g'(z_2) \end{aligned}$$

Case 1: Hidden layer uses sigmoid

Forward (per sample)

- $z_1 = w_{11}x_1 + w_{12}x_2 + b_1, \quad h_1 = \sigma(z_1)$
- $z_2 = w_{21}x_1 + w_{22}x_2 + b_2, \quad h_2 = \sigma(z_2)$
- $\hat{y} = v_1h_1 + v_2h_2 + b_3$ (linear output)
- $L = \frac{1}{2}(\hat{y} - y)^2$

Let $\sigma(z) = \frac{1}{1+e^{-z}}$ and $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

Neuron 1 (parameters w_{11}, w_{12}, b_1)

$$\frac{\partial L}{\partial w_{11}} = (\hat{y} - y) v_1 h_1 (1 - h_1) x_1$$

$$\frac{\partial L}{\partial w_{12}} = (\hat{y} - y) v_1 h_1 (1 - h_1) x_2$$

$$\frac{\partial L}{\partial b_1} = (\hat{y} - y) v_1 h_1 (1 - h_1)$$

Neuron 2 (parameters w_{21}, w_{22}, b_2)

$$\frac{\partial L}{\partial w_{21}} = (\hat{y} - y) v_2 h_2 (1 - h_2) x_1$$

$$\frac{\partial L}{\partial w_{22}} = (\hat{y} - y) v_2 h_2 (1 - h_2) x_2$$

$$\frac{\partial L}{\partial b_2} = (\hat{y} - y) v_2 h_2 (1 - h_2)$$

Backpropagation = systematically applying the chain rule backward along every dependency path from the loss to each parameter.



4. Why is it Called *Backpropagation*?

Because we apply the chain rule **in reverse order** — we start from the **loss $\mathcal{L}$** at the output and move **backward** through the network.

Step flow:

- 1 Compute the loss at the output (forward pass done).
- 2 Compute its derivative w.r.t. the output layer weights v_j .
- 3 Use those results to compute derivatives w.r.t. hidden layer activations h_j .
- 4 Continue backward layer by layer — using the chain rule each time.

This process **propagates the error backward** through the network, hence the name:

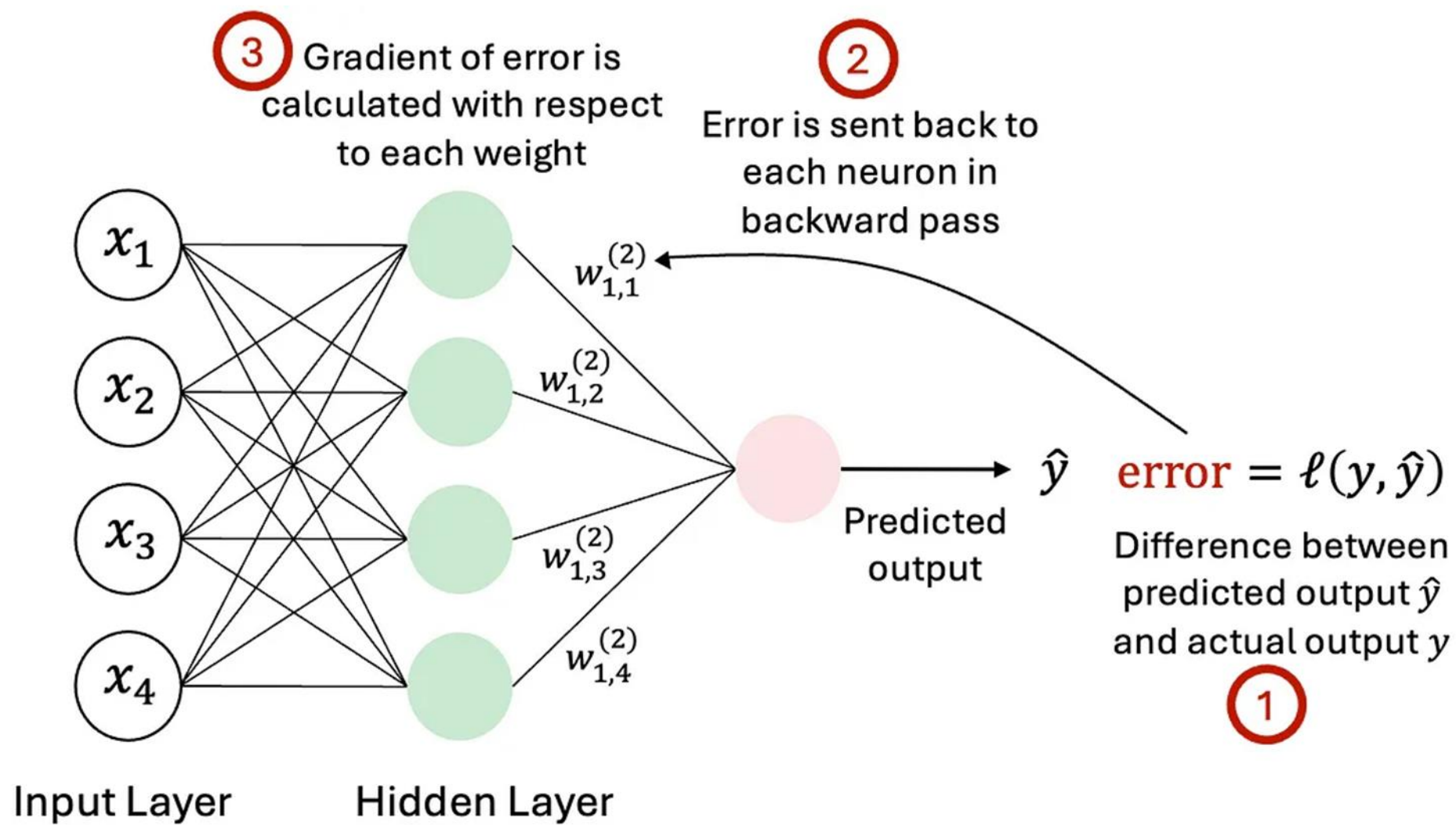


Back-Propagation = backward propagation of error gradients.

Why Backward Instead of Forward?

- In the **forward pass**, you compute values layer by layer to get the output.
- In the **backward pass**, you reuse those forward values (like h_j, z_j) to efficiently compute gradients.
- Each layer uses the **gradient from the next layer** to update its own parameters — this is computationally efficient because:
 - You don't recompute full derivatives from scratch.
 - You reuse intermediate partials via the chain rule.

Stage	Direction	Description
Forward pass	Input → Output	Compute activations and final loss
Backward pass	Output → Input	Propagate error derivatives backward using chain rule
Parameter update	—	Adjust each weight based on its gradient



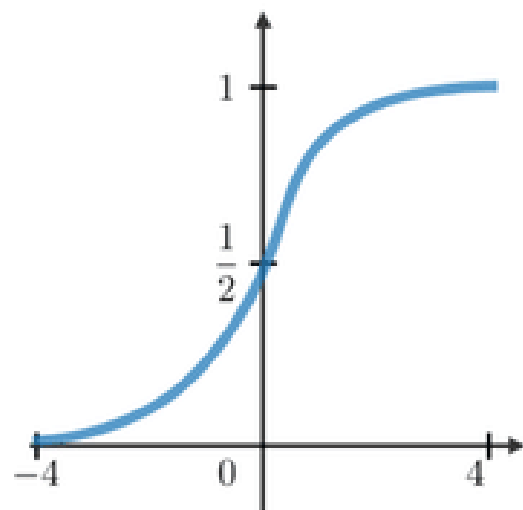
...but why is sigmoid not used in hidden layers for DNN?

It is a home task...FOYS

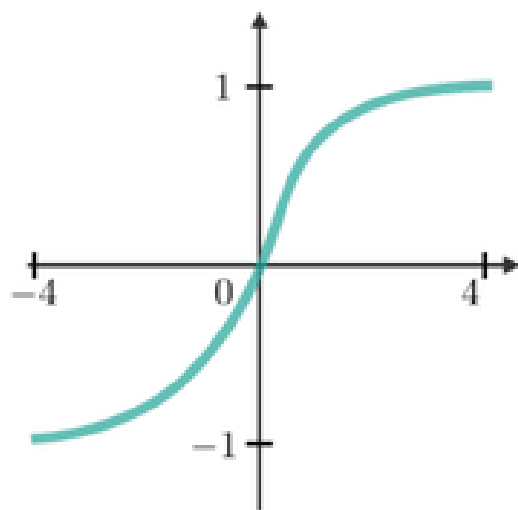
Hint: Gradient of Sigmoid

Sigmoid

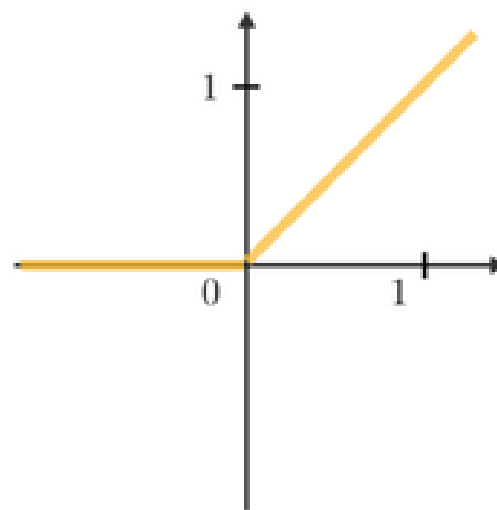
$$g(z) = \frac{1}{1 + e^{-z}}$$

**Tanh**

$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

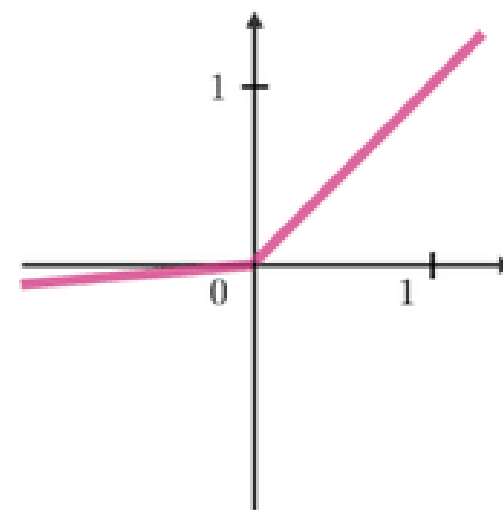
**ReLU**

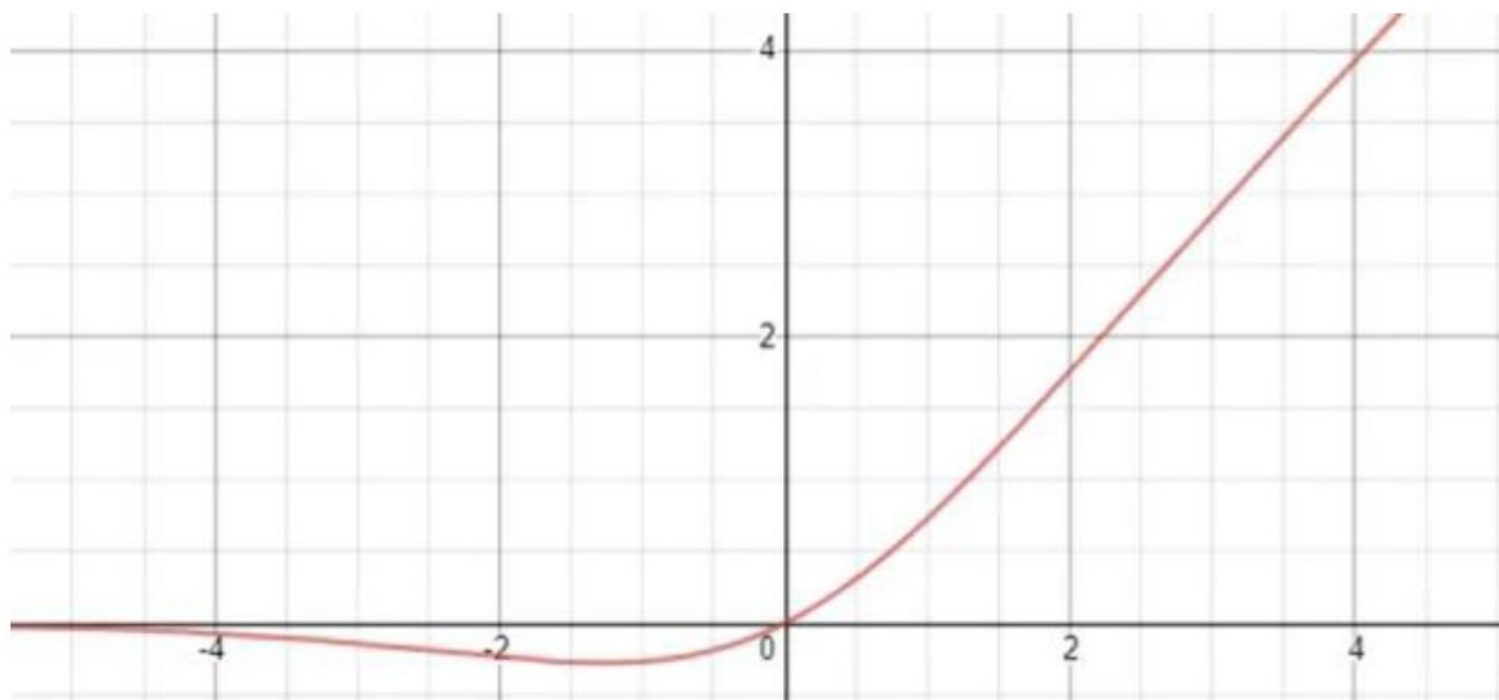
$$g(z) = \max(0, z)$$

**Leaky ReLU**

$$g(z) = \max(\epsilon z, z)$$

with $\epsilon \ll 1$





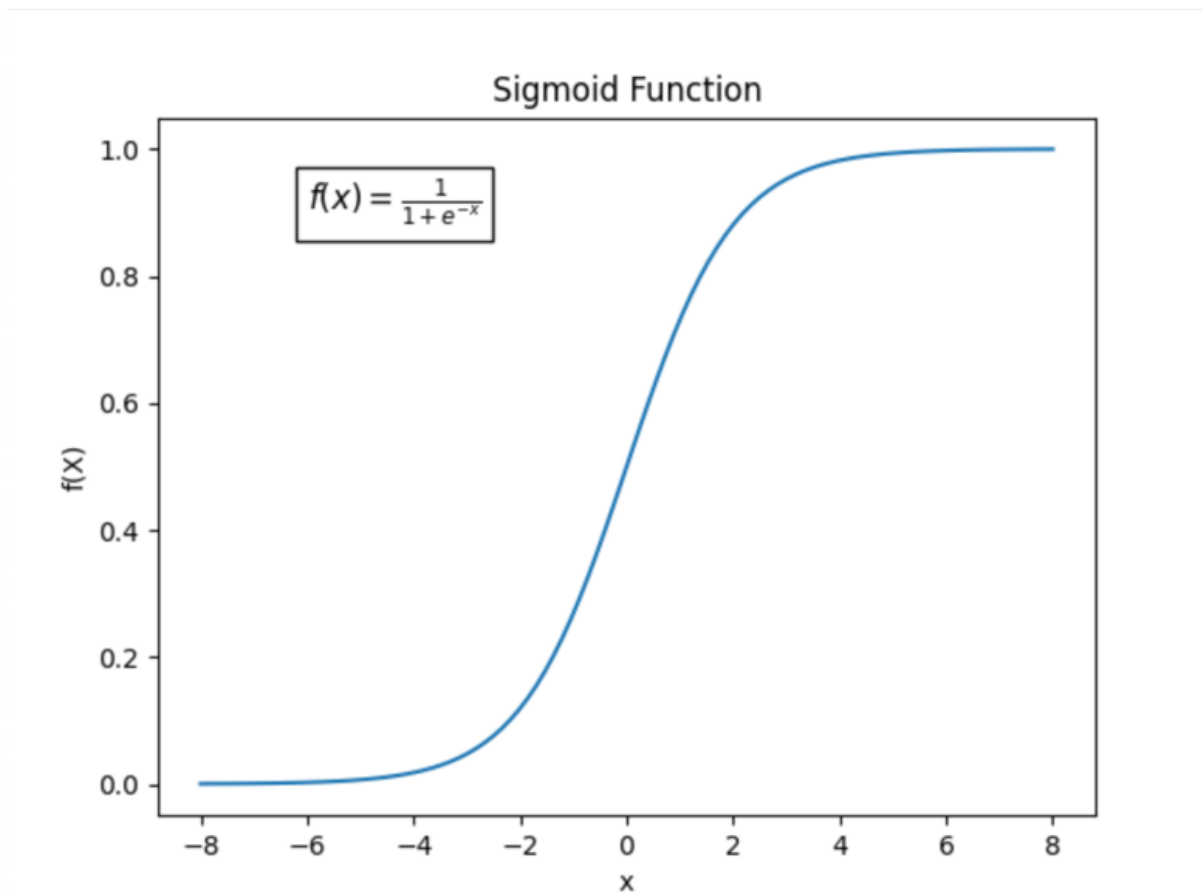
the Swish activation function is...

$$f(x) = x * (1 + \exp(-x))^{-1}$$

Why Sigmoid causes vanishing gradients, whereas ReLU preserves gradient flow in deeper networks.

Function	Formula
ReLU (Rectified Linear Unit)	$f(x) = \max(0, x)$
Leaky ReLU	$f(x) = \max(0.01x, x)$
Sigmoid	$f(x) = \frac{1}{1+e^{-x}}$
Tanh	$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
GELU (Gaussian Error Linear Unit)	$f(x) = x \cdot \Phi(x)$
Swish	$f(x) = x \cdot \sigma(x)$
Softmax	$f_i(x) = \frac{e^{x_i}}{\sum_j e^{x_j}}$

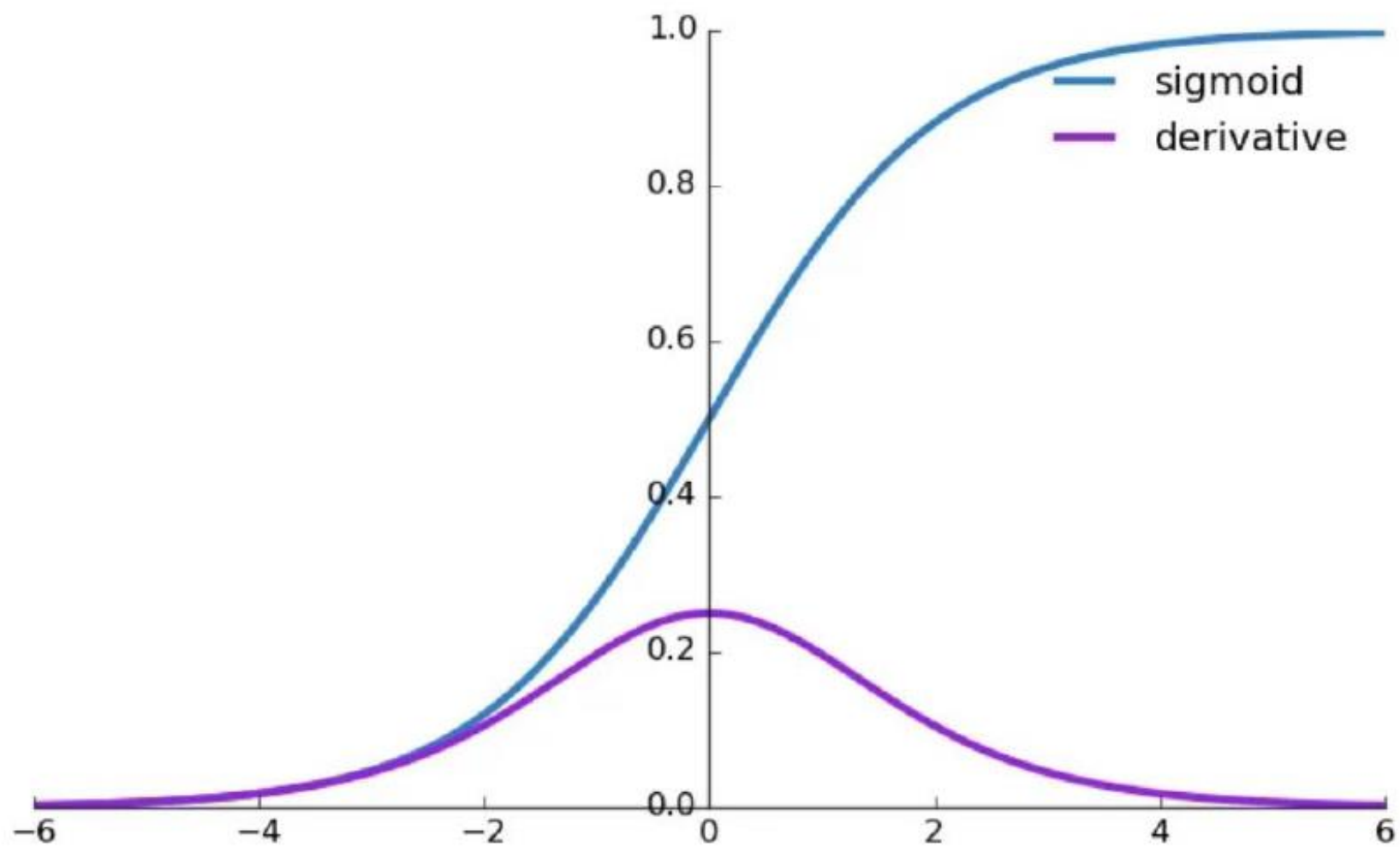
$$f(\mathbf{x}) = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$



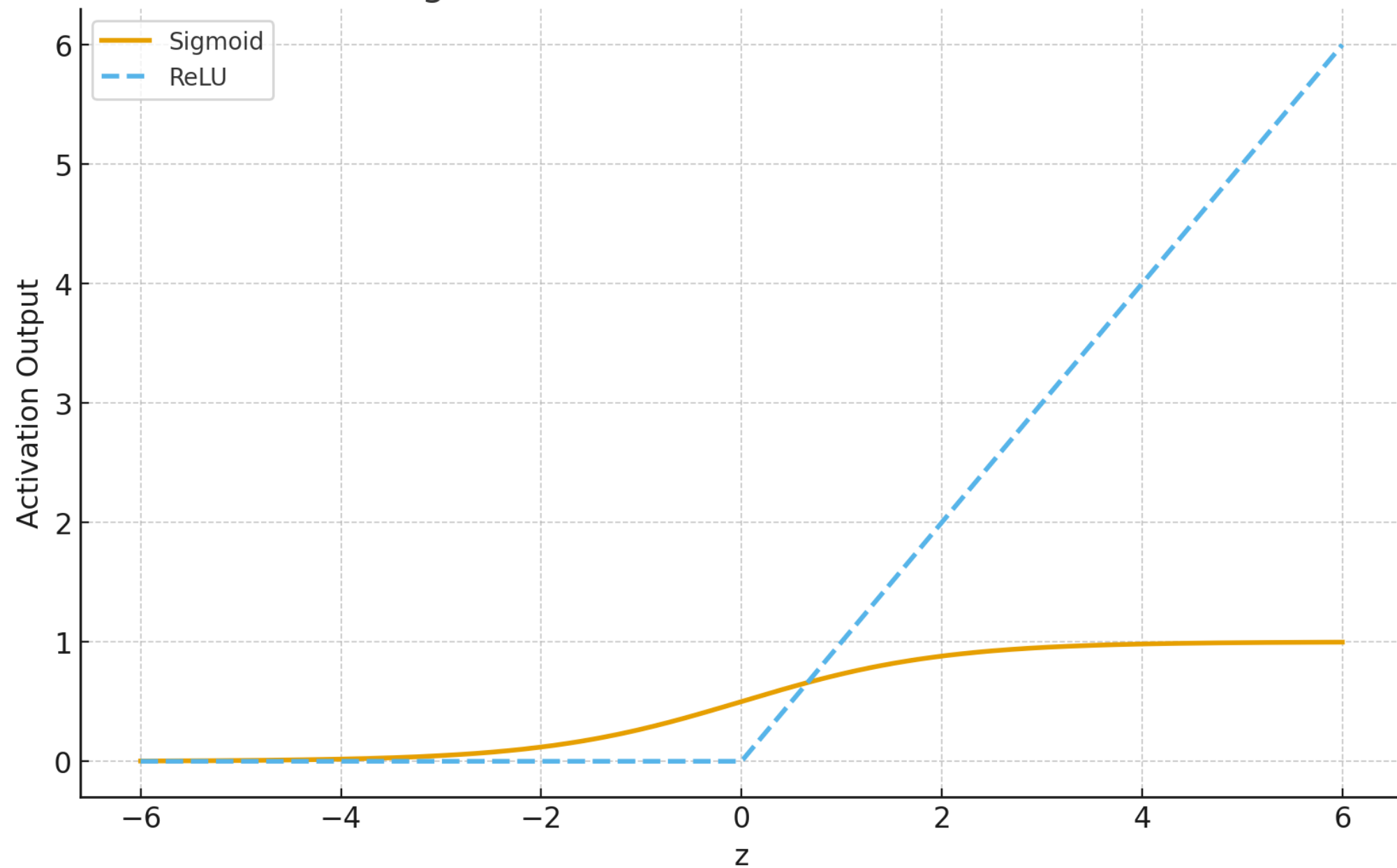
Draw the graph of the derivative of the sigmoid w.r.t z

- When $z = 0$:
 $\sigma(0) = 0.5 \Rightarrow \sigma'(0) = 0.25$ (maximum slope)
- As $z \rightarrow +\infty$ or $z \rightarrow -\infty$:
 $\sigma'(z) \rightarrow 0$ (flat saturation zones)

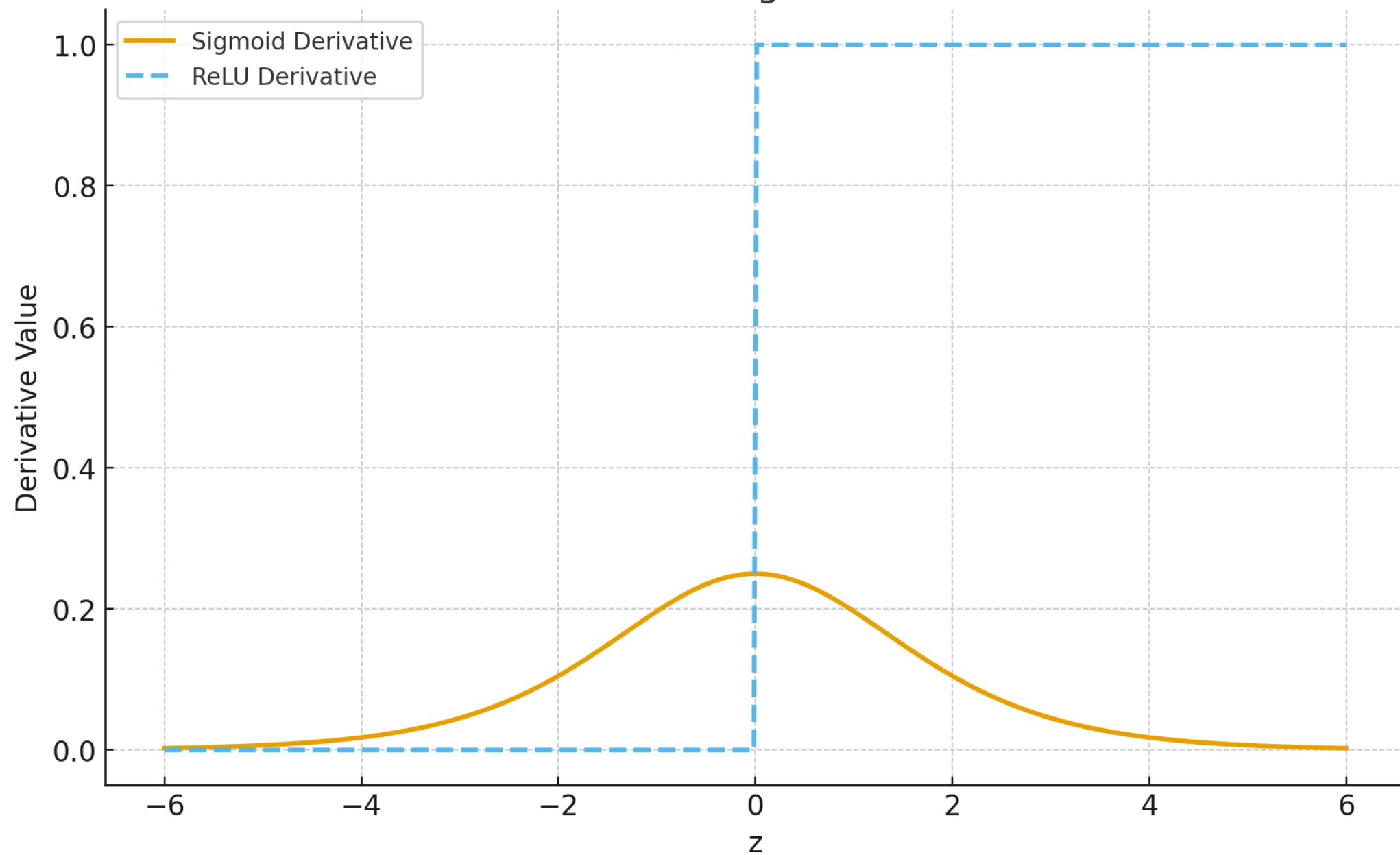
Graph of sigmoid derivative



Sigmoid vs ReLU Activation Functions



Derivatives of Sigmoid and ReLU



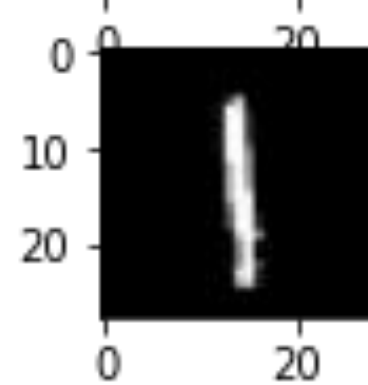
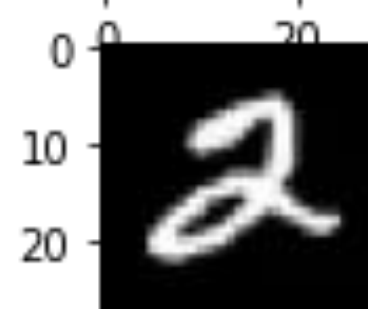
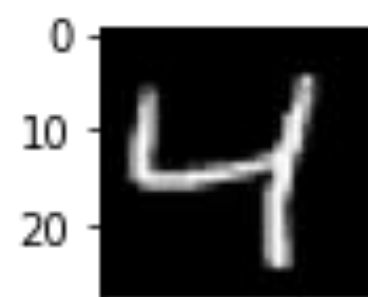
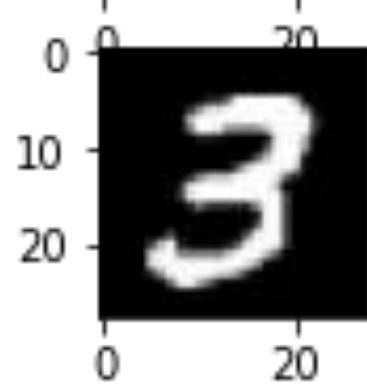
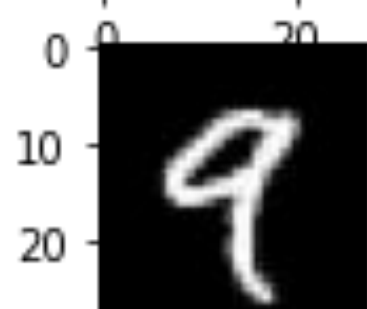
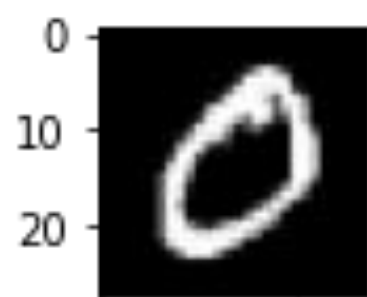
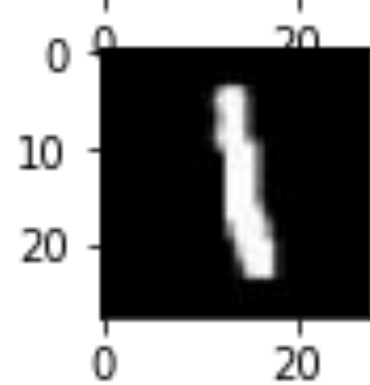
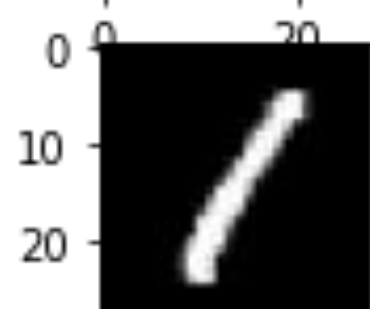
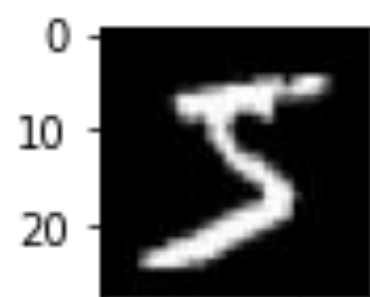
Apply NN on the image classification problem

MNIST Dataset (Modified National Institute of Standards and Technology)

1. **Type:** Handwritten digit images (classification problem)
2. **Classes:** 10 digits $\rightarrow \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
3. **Image size:** 28×28 pixels (grayscale $\rightarrow$ 1 channel)
4. **Total samples:** 70 000
 - 60 000 training images
 - 10 000 test images
5. **Pixel values:** 0 – 255 $\rightarrow$ usually normalized to [0, 1]

[illegible]

1. Loading and understanding the MNIST dataset
2. Visualizing the data
3. Building a neural network model with Keras
4. Training and evaluating the model
5. Making predictions and interpreting results



1	1	0
4	2	1
0	2	1

Pooled Feature Map

Flattening



1
1
0
4
2
1
0
2
1

4. Building the Neural Network Model

```
model = Sequential(  
    [    Input((28,28)),  
      Flatten(),  
      Dense(16, activation="relu", name="layer1"),  
      Dense(16, activation="relu", name="layer2"),  
      Dense(10, activation="softmax"),  
    ]  
)
```

5. Compiling the Model

```
model.compile(optimizer='adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
layer1 (Dense)	(None, 16)	12560
layer2 (Dense)	(None, 16)	272
dense (Dense)	(None, 10)	170

Total params: 13,002

Trainable params: 13,002

Non-trainable params: 0

7. Training the Model

```
model.fit(x=train_X, y=train_y, epochs=25, validation_split=0.2)
```

Epoch 1/25

1500/1500 [=====] - 4s 2ms/step - loss: 0.2970 -

Epoch 2/25

1500/1500 [=====] - 3s 2ms/step - loss: 0.2983 -

Epoch 3/25

1500/1500 [=====] - 3s 2ms/step - loss: 0.2979 -

Epoch 4/25

1500/1500 [=====] - 3s 2ms/step - loss: 0.2921 -

Epoch 5/25

1500/1500 [=====] - 3s 2ms/step - loss: 0.2886 -

Epoch 6/25

1500/1500 [=====] - 4s 2ms/step - loss: 0.2850 -

Epoch 7/25

1500/1500 [=====] - 4s 2ms/step - loss: 0.2797 -

Epoch 8/25

8. Evaluating the Model on Test Data

```
model.evaluate(x=test_X,y=test_y)
```

```
313/313 [=====] - 0s 1ms/step - loss: 0.3209  
[0.32091614603996277, 0.9162999987602234]
```

Motivation: From Dense Networks to CNNs

- Ignores Spatial Structure
 - In a dense layer, each pixel is treated as an independent input.
 - The 2D image (28×28) is **flattened** into a 1D vector of 784 elements.
 - Hence, information about *which pixels are neighbors* is completely lost.

Example:

If the digit "3" shifts slightly to the right, the pixel arrangement changes — the dense layer sees a "different input," even though it's the same digit.

2. Too Many Parameters

Let's compute for MNIST:

- Input = $28 \times 28 = 784$ neurons
- Hidden layer = 128 neurons

$$\text{Parameters} = 784 \times 128 + 128 = 100,480$$

For multiple layers, this grows **exponentially**.

Need for a better embedding layer before classification

- A layer which can
 - ✓ extract features using fewer parameters
 - ✓ Consider the spatial distribution of pixels (features)

NN with Convolution Layer

- Convolution can be done grid-like feature orientation or structure



5	2	3	1	2	4
2	4	1	0	3	1
5	1	0	2	8	3
0	2	1	5	2	4
2	7	0	0	2	1
1	3	2	8	7	0

Filter (kernel)

- *Kernel* = grid of weights
- Kernel is “applied” to the image
- Traditionally used in image processing

1	2	-1
0	1	2
-2	1	0

Features (grid-like orientation) * Kernel (filter) → Output

Tensor transformer

Tensor transformer

- Filter will scan the image and extract a feature map with a particular shape

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

5 x 5 – Image Matrix



1	0	1
0	1	0
1	0	1

3 x 3 – Filter Matrix

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

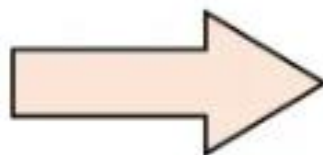
Image

4		

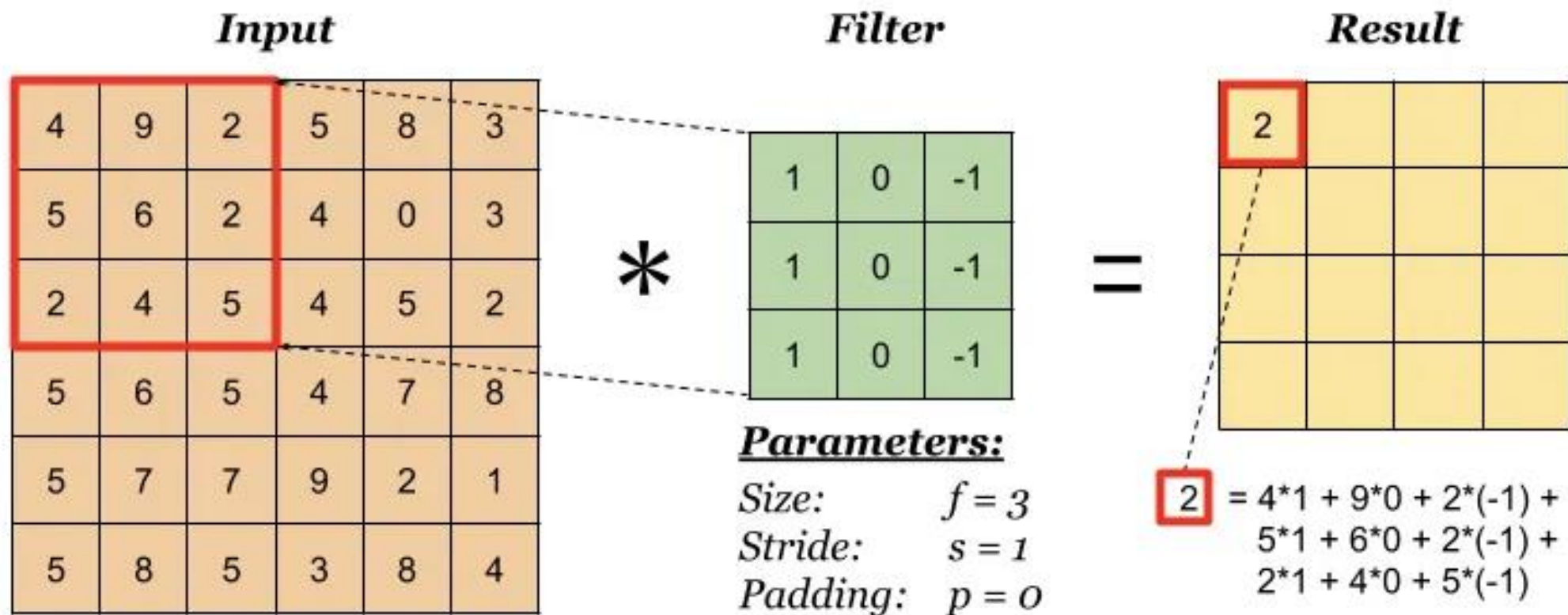
Convolved
Feature

1	2	3	4	5	6	7
11	12	13	14	15	16	17
21	22	23	24	25	26	27
31	32	33	34	35	36	37
41	42	43	44	45	46	47
51	52	53	54	55	56	57
61	62	63	64	65	66	67
71	72	73	74	75	76	77

Convolve with 3x3
filters filled with ones



108	126	
288	306	



Mathematical View of Convolution

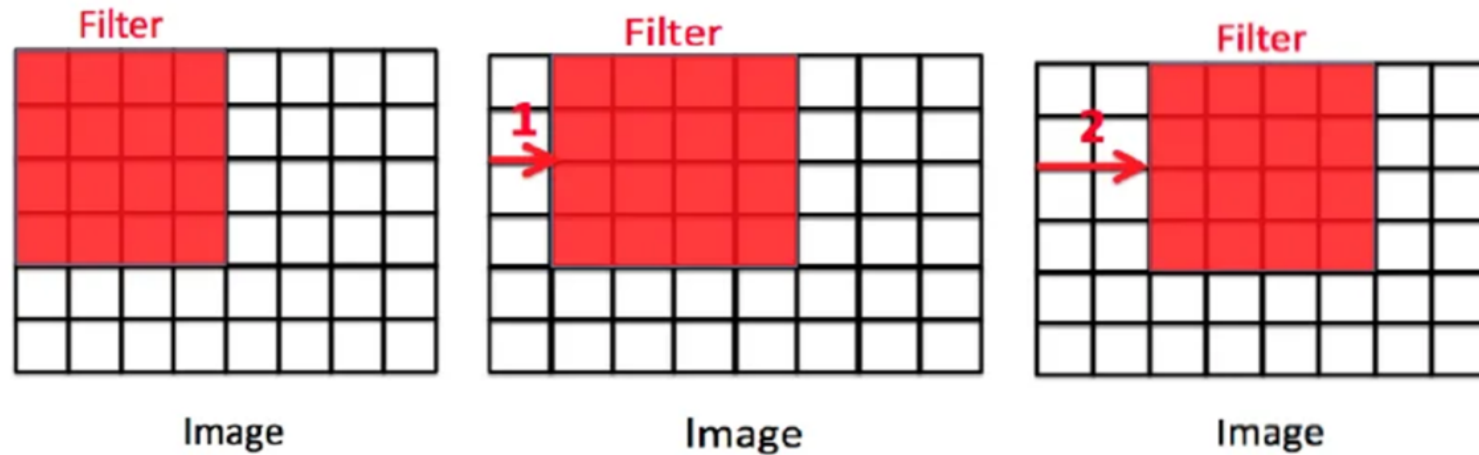
Equation:

$$Y(x, y) = \sum_{i,j} W_{ij} \cdot X(x + i, y + j)$$

Stride

- Step size used for sliding kernel on image
- Indicated in pixels

Stride



left image: stride =0, middle image: stride = 1, right image: stride =2

Output Size

For

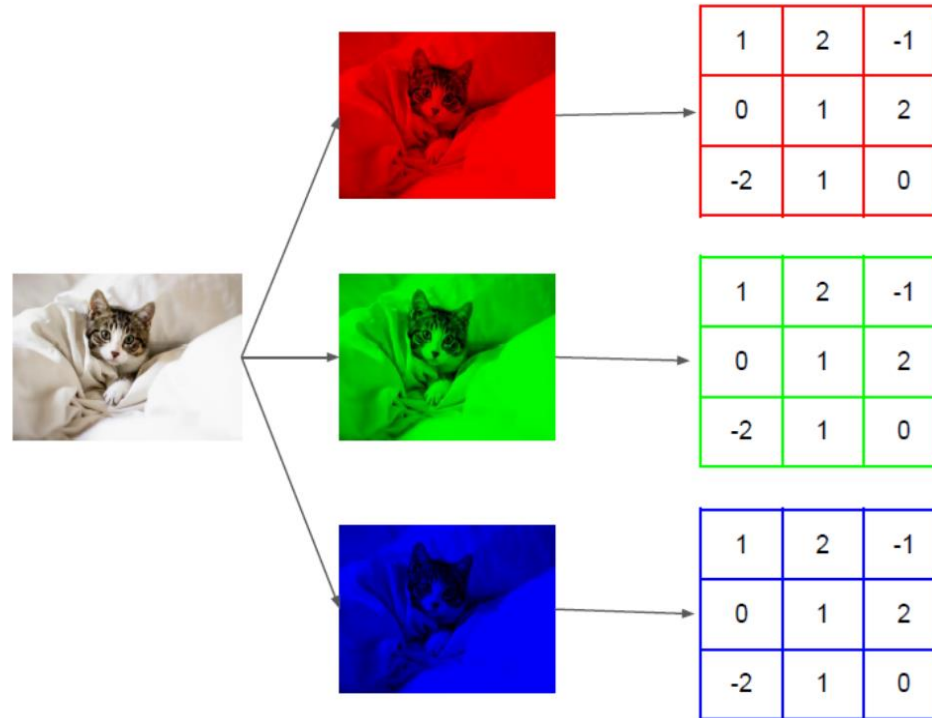
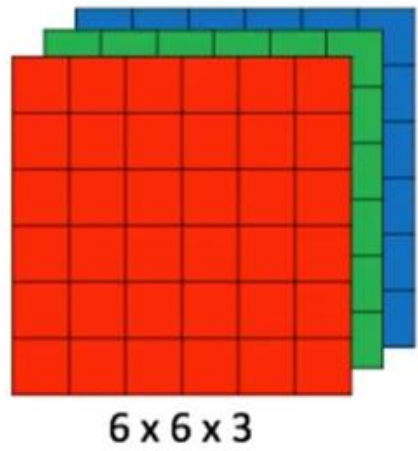
- Input size = $n \times n$
- Filter (kernel) size = $f \times f$
- Stride = s

the **output feature map size** is:

$$O = \frac{(n - f)}{s} + 1$$

RGB

Depth

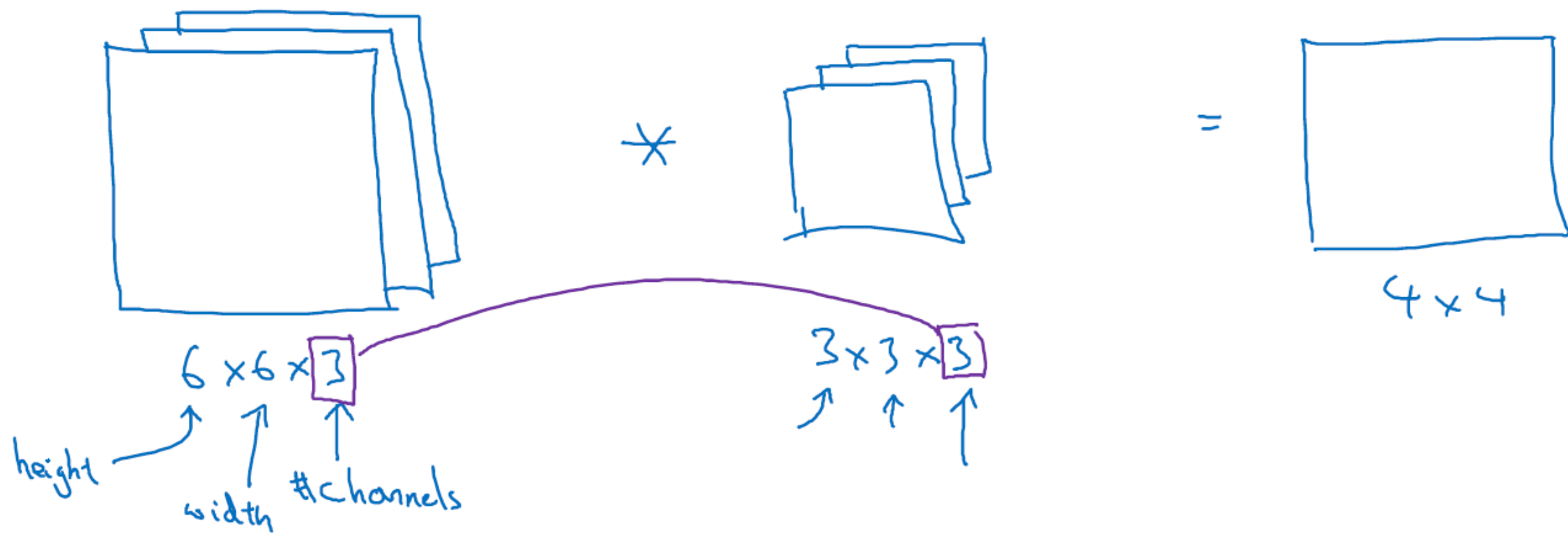


Convolution handles color channels

Given:

- Input image: RGB $\rightarrow$ 3 channels
 $\rightarrow$ Input shape = $n \times n \times 3$
- One filter (kernel): $f \times f$
 $\rightarrow$ Filter shape = $f \times f \times 3$
- Stride = s

Convolutions on RGB images



1. Output size per dimension

The spatial output (height and width) are computed exactly as before:

$$O = \frac{(n - f)}{s} + 1$$

So the **output feature map** will have shape:

$$O \times O \times 1$$

(because we are using only one filter)

How convolution works with RGB channels



? Explain this

Each filter has **depth** = 3 (one 2D kernel per channel):

- W_R : filter for Red channel
- W_G : filter for Green channel
- W_B : filter for Blue channel

When convolving:

$$\text{output}(x, y) = \sum_{c=1}^3 \sum_{i,j} W_{i,j,c} \cdot I(x+i, y+j, c)$$

That is, each color channel is multiplied with its corresponding slice of the filter, summed elementwise, then all 3 channel results are **summed together** to form **one scalar output** at each location.

Single-Channel (Gray Image)

$$\begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix} * \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} + \begin{bmatrix} b & b \\ b & b \end{bmatrix} = \begin{bmatrix} y_{11} & y_{12} \\ y_{21} & y_{22} \end{bmatrix}$$

RGB Image (3 Channels)

$$\left(\begin{bmatrix} R_{11} & R_{12} & R_{13} \\ R_{21} & R_{22} & R_{23} \\ R_{31} & R_{32} & R_{33} \end{bmatrix}, \begin{bmatrix} G_{11} & G_{12} & G_{13} \\ G_{21} & G_{22} & G_{23} \\ G_{31} & G_{32} & G_{33} \end{bmatrix}, \begin{bmatrix} B_{11} & B_{12} & B_{13} \\ B_{21} & B_{22} & B_{23} \\ B_{31} & B_{32} & B_{33} \end{bmatrix} \right) \\ * \left(\begin{bmatrix} w_{11}^R & w_{12}^R \\ w_{21}^R & w_{22}^R \end{bmatrix}, \begin{bmatrix} w_{11}^G & w_{12}^G \\ w_{21}^G & w_{22}^G \end{bmatrix}, \begin{bmatrix} w_{11}^B & w_{12}^B \\ w_{21}^B & w_{22}^B \end{bmatrix} \right) + \begin{bmatrix} b & b \\ b & b \end{bmatrix} = \begin{bmatrix} y_{11} & y_{12} \\ y_{21} & y_{22} \end{bmatrix}$$

- Input image (single channel for simplicity):

$$X \in \mathbb{R}^{n \times n}$$

- Filter (kernel):

$$W \in \mathbb{R}^{f \times f}$$

- Bias:

$$b \in \mathbb{R}$$

Input X is 3×3 ,

Filter W is 2×2 ,

Bias $b = 1$.

$$X = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix}, \quad W = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix}$$

The output Y will be 2×2 :

$$Y = \begin{bmatrix} x_{11}w_{11} + x_{12}w_{12} + x_{21}w_{21} + x_{22}w_{22} + b & x_{12}w_{11} + x_{13}w_{12} + x_{22}w_{21} + x_{23}w_{22} + b \\ x_{21}w_{11} + x_{22}w_{12} + x_{31}w_{21} + x_{32}w_{22} + b & x_{22}w_{11} + x_{23}w_{12} + x_{32}w_{21} + x_{33}w_{22} + b \end{bmatrix}$$

| 3. Example

Input: $32 \times 32 \times 3$

Filter: $3 \times 3 \times 3$

Stride: 1

$$\rightarrow O = (32 - 3) / 1 + 1 = 30$$

✓ Output = $30 \times 30 \times 1$

If you use **10 such filters**, then:

✓ Output = $30 \times 30 \times 10$

(each filter learns a different feature — e.g., edges, colors, textures)

1. Input (RGB image)

Input tensor:

$$\text{Input shape} = n \times n \times 3$$

(Height $\times$ Width $\times$ Channels)

Example: $32 \times 32 \times 3$ (like CIFAR-10)

2. Filters (Kernels)

Each filter has **the same depth as the input (3 channels)** but its own spatial size $f \times f$.

If you use C_{out} filters (say 16 filters), then:

$$\text{Each filter shape} = f \times f \times 3$$

$$\text{All filters together} = f \times f \times 3 \times C_{out}$$

Example

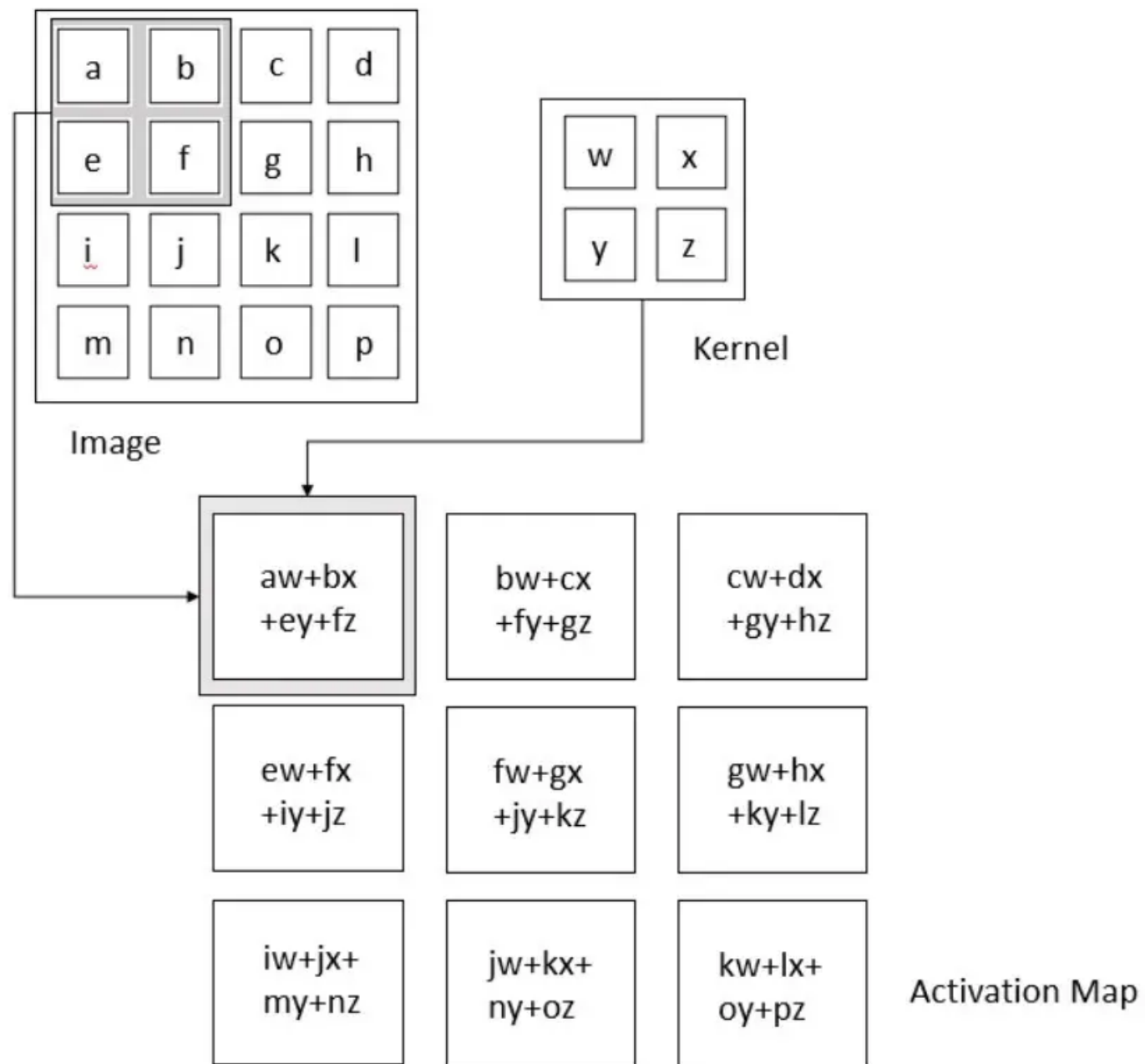
Input:

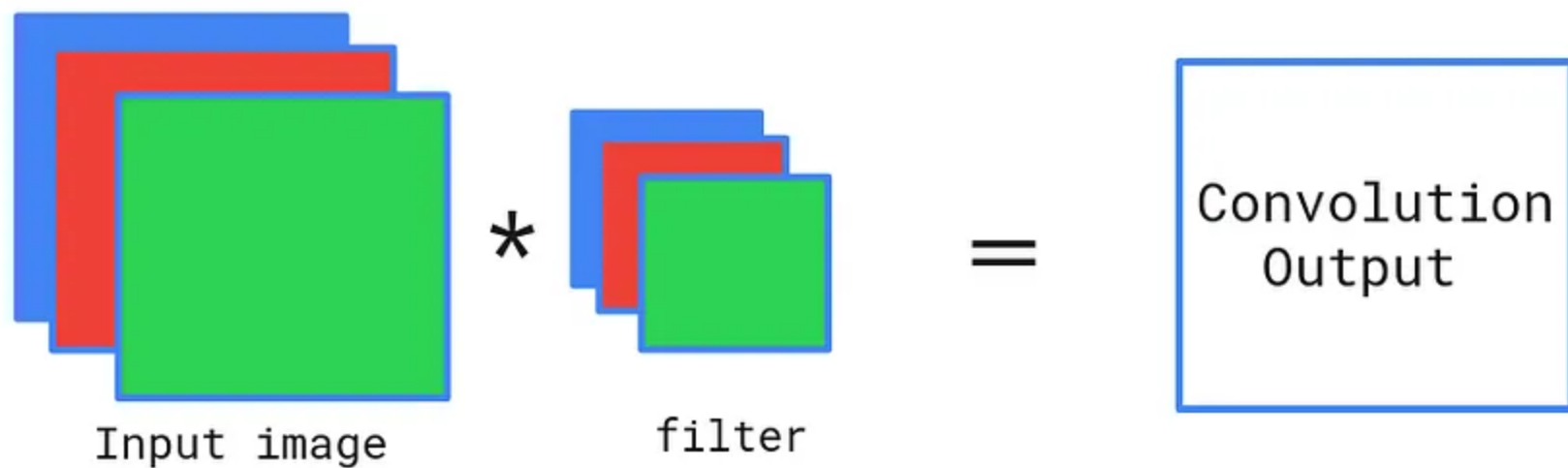
- RGB image $\rightarrow 32 \times 32 \times 3$
- Filter size $f = 3$
- Stride $s = 1$
- Number of filters $C_{out} = 8$

Then:

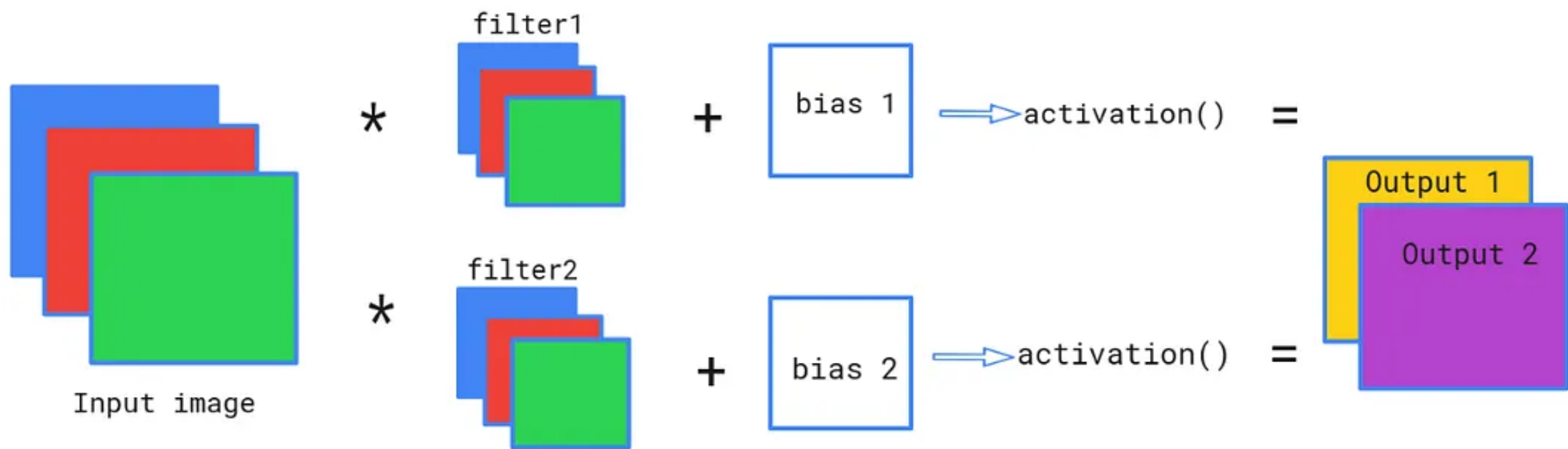
$$O = \frac{(32 - 3)}{1} + 1 = 30$$

✓ Output shape: $30 \times 30 \times 8$

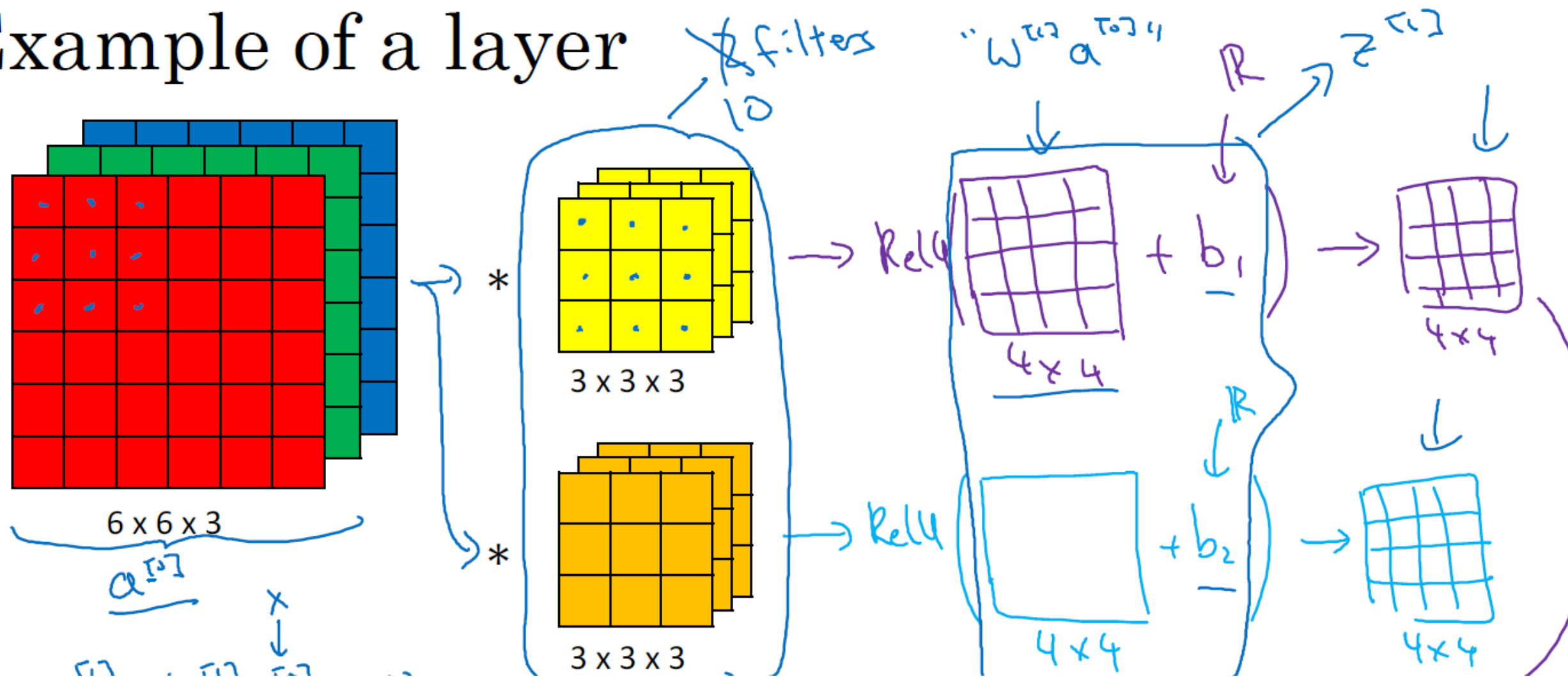




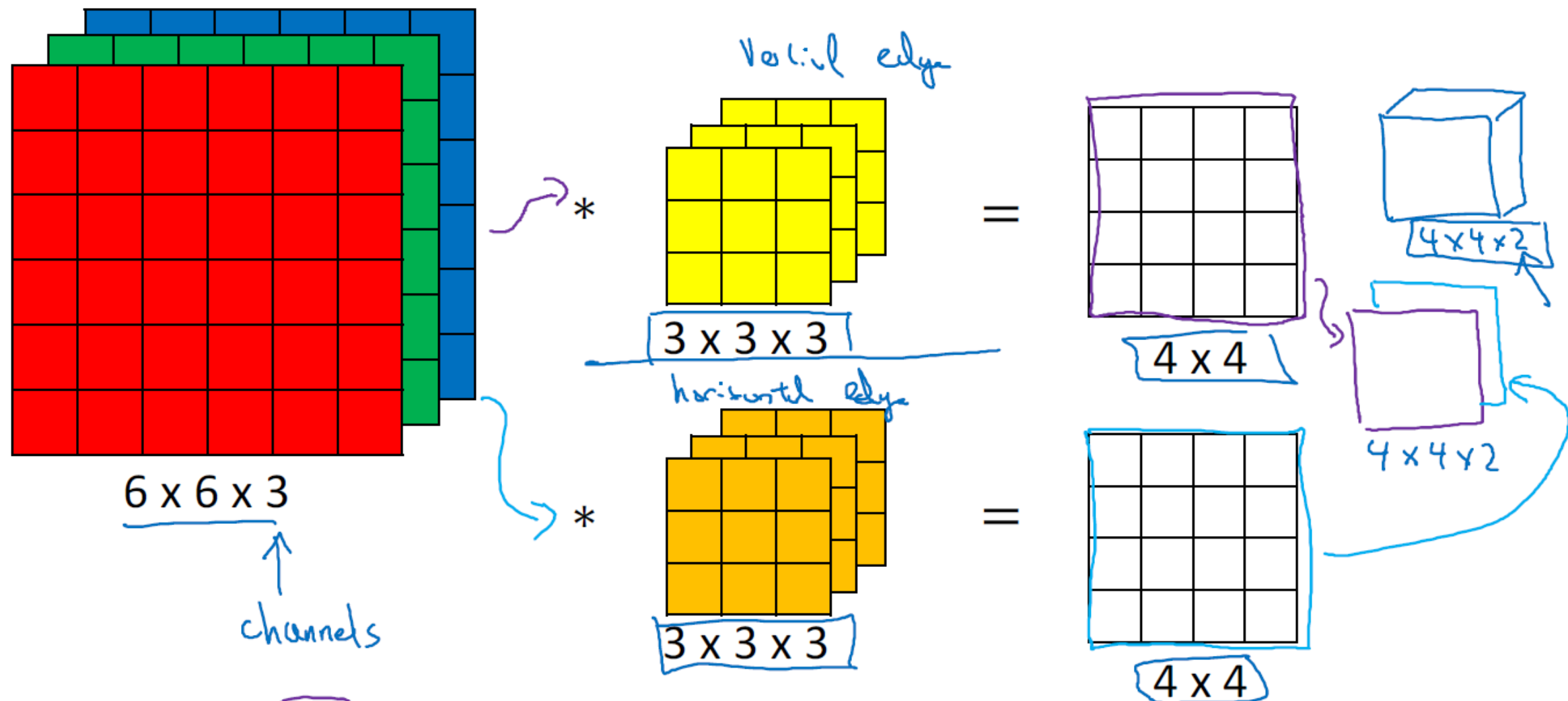
$$\text{activation} \left(\text{Convolution Output} + \text{Bias} \right) = \text{Output}$$



Example of a layer



Multiple filters



Summary: $n \times n \times n_c$ $\times$ $f \times f \times n_c$ $\rightarrow$ $\frac{n-f+1}{4} \times \frac{n-f+1}{4} \times \frac{n_c}{2}$ $\uparrow$ # filters

$6 \times 6 \times 3$ $3 \times 3 \times 3$

Number of parameters in one layer

If you have 10 filters that are $3 \times 3 \times 3$ in one layer of a neural network, how many parameters does that layer have?

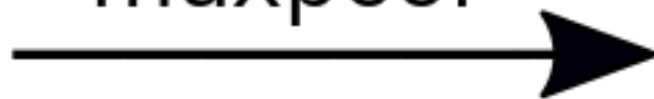
Pooling layer

- Pooling is a **downsampling operation** that summarizes a region of the feature map.
- It scans over each feature map (independently) and replaces a group of pixels with a **single representative value**.

Input

7	3	5	2
8	7	1	6
4	9	3	9
0	8	4	5

maxpool



Output

8	6
9	9

Max Pooling

4	3	1	5
1	3	4	8
4	5	4	3
6	5	9	4

$$\text{Max}([4, 3, 1, 3]) = 4$$

4	3	1	5
1	3	4	8
4	5	4	3
6	5	9	4



4	8
6	9

Average Pooling

4	3	1	5
1	3	4	8
4	5	4	3
6	5	9	4

$$\text{Avg}([4, 3, 1, 3]) = 2.75$$

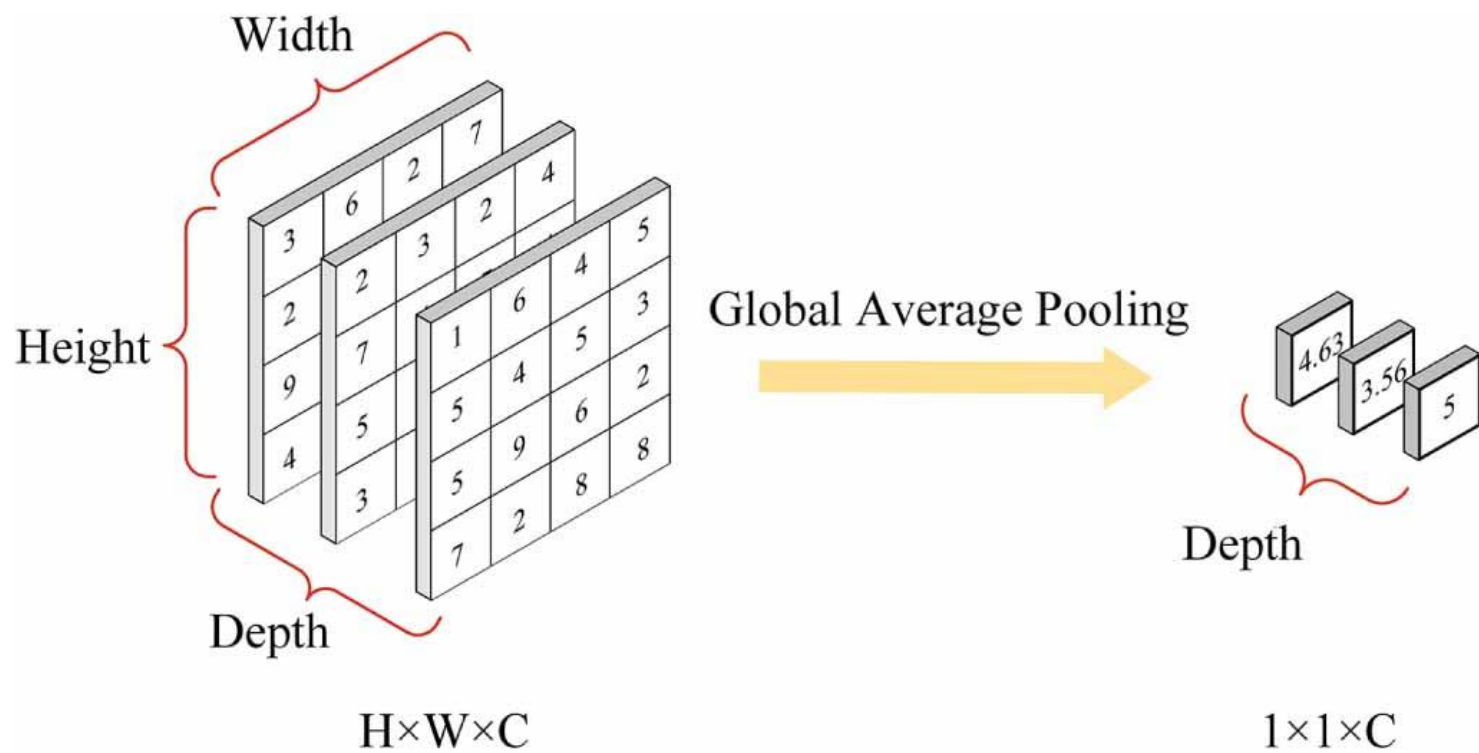
4	3	1	5
1	3	4	8
4	5	4	3
6	5	9	4

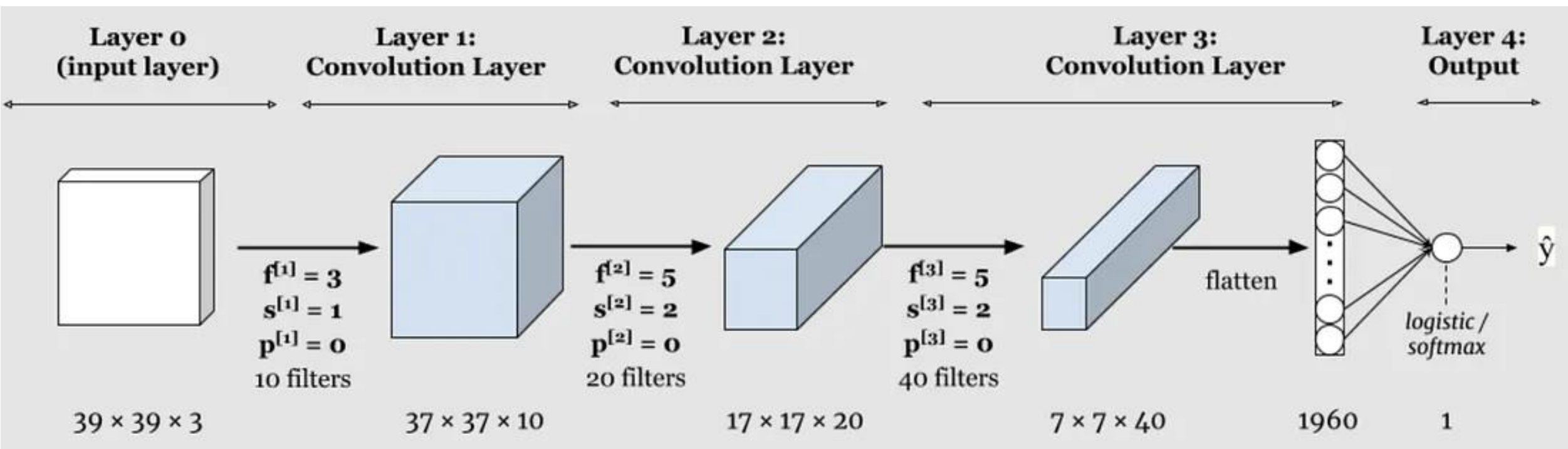


2.8	4.5
5.3	5.0

Type	Operation	Intuition
Max Pooling	Takes the maximum value in the window	Keeps strongest activation (most confident feature)
Average Pooling	Takes the average	Smooths feature map (less used today)
Global Average Pooling	Reduces each feature map to a single number	Used in modern CNNs before dense layers

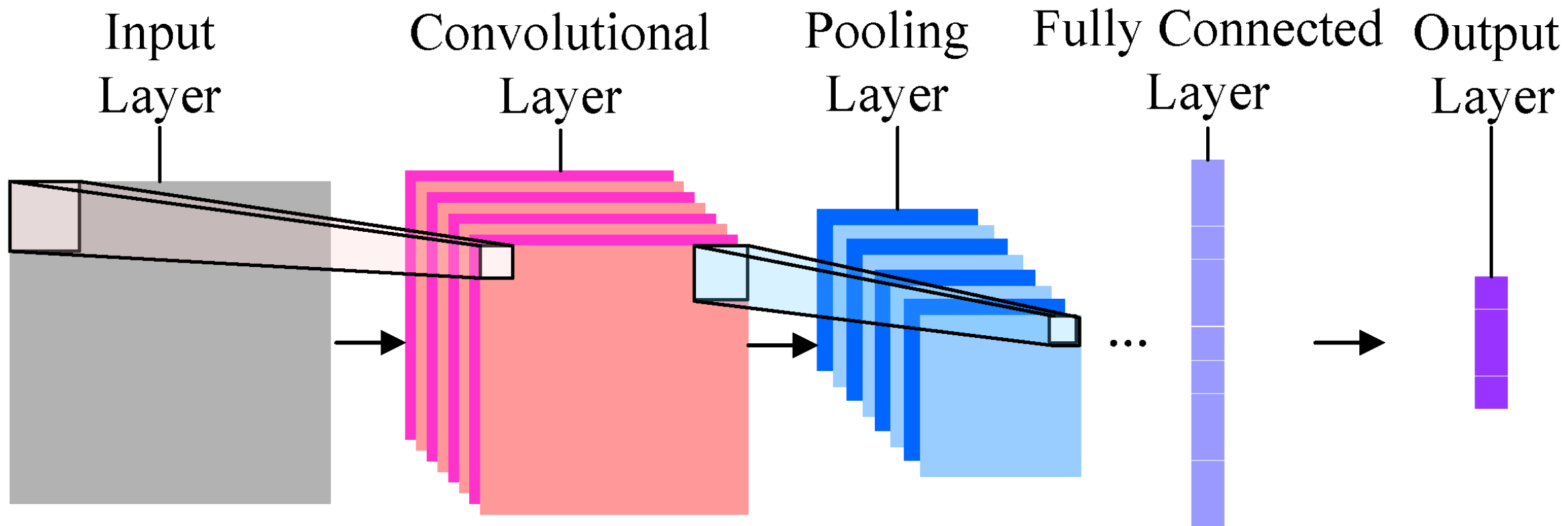
Layer	Input Shape	Output Shape	Trainable Parameters
Flatten + Dense(10)	$7 \times 7 \times 512$	10	$7 \times 7 \times 512 \times 10 = 250,880$
GlobalAvgPool + Dense(10)	$7 \times 7 \times 512$	$1 \times 1 \times 512 \rightarrow 10$	$512 \times 10 = 5,120$

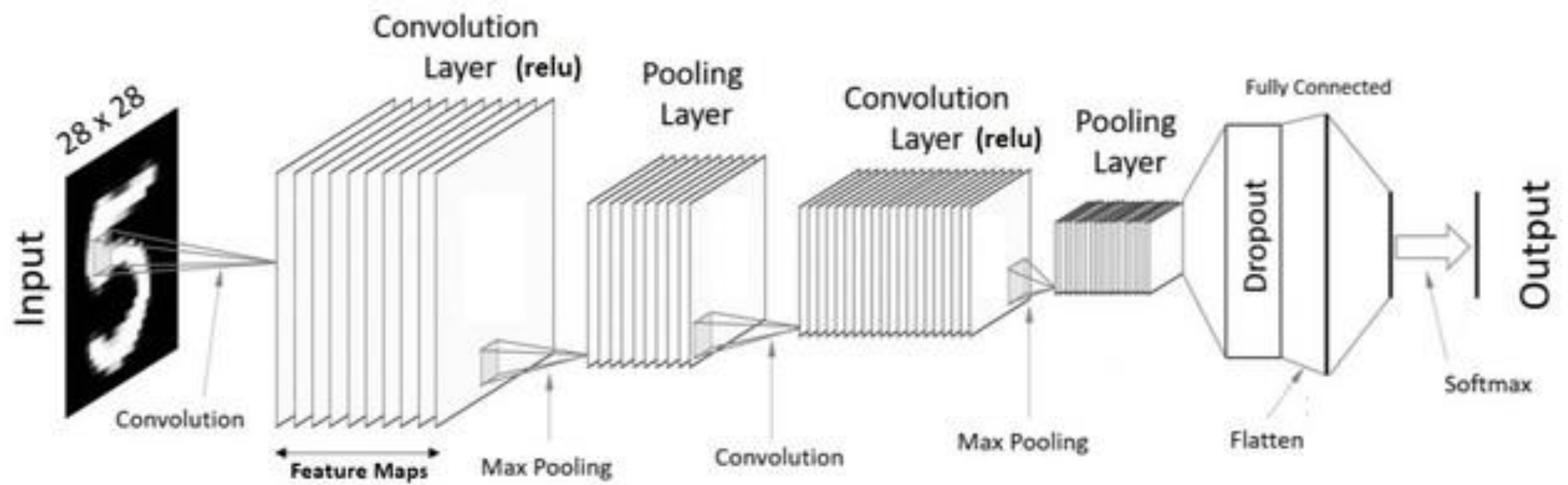




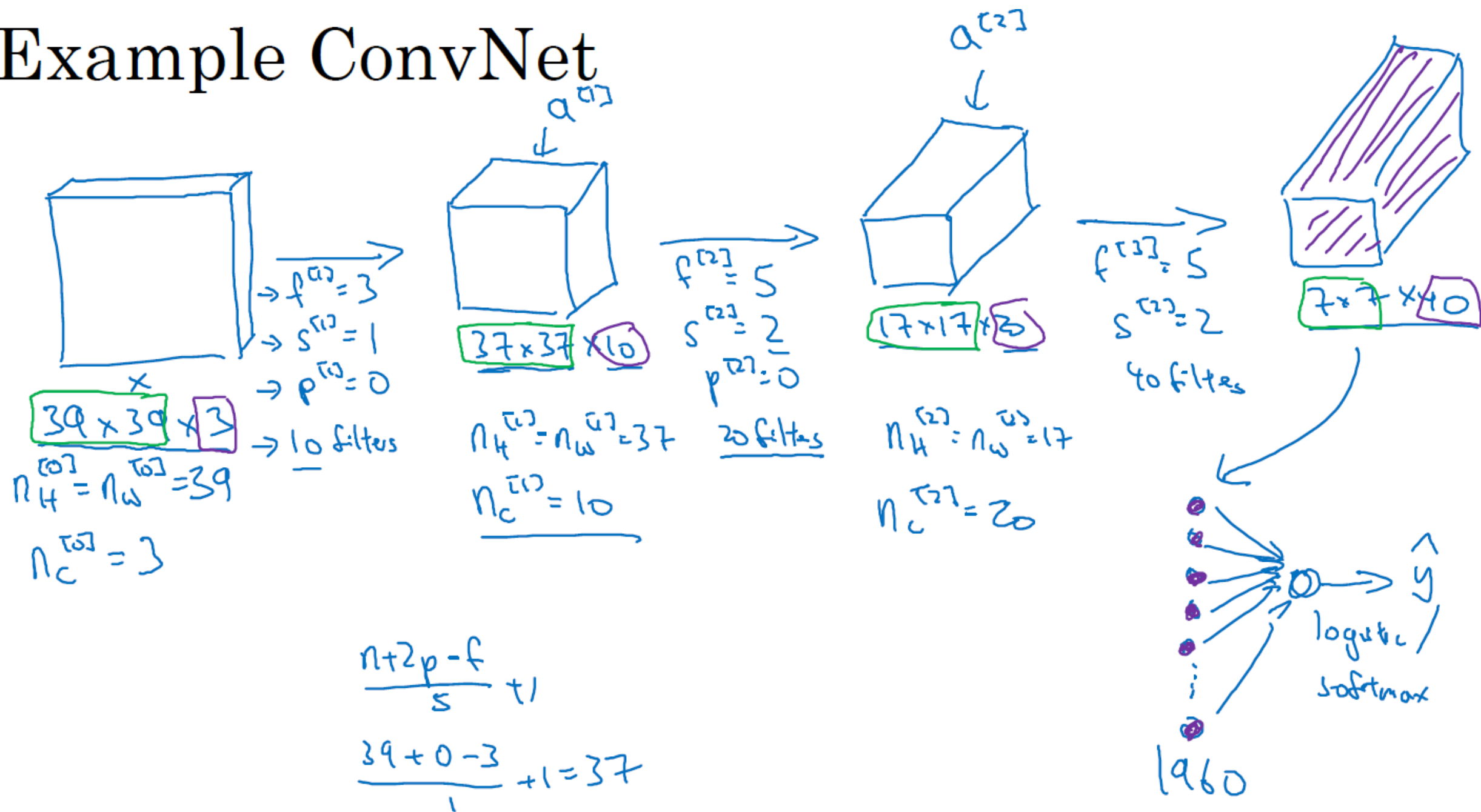
CNN

- Conv layer
- Pooling layer
- Fully connected (Dense) layer

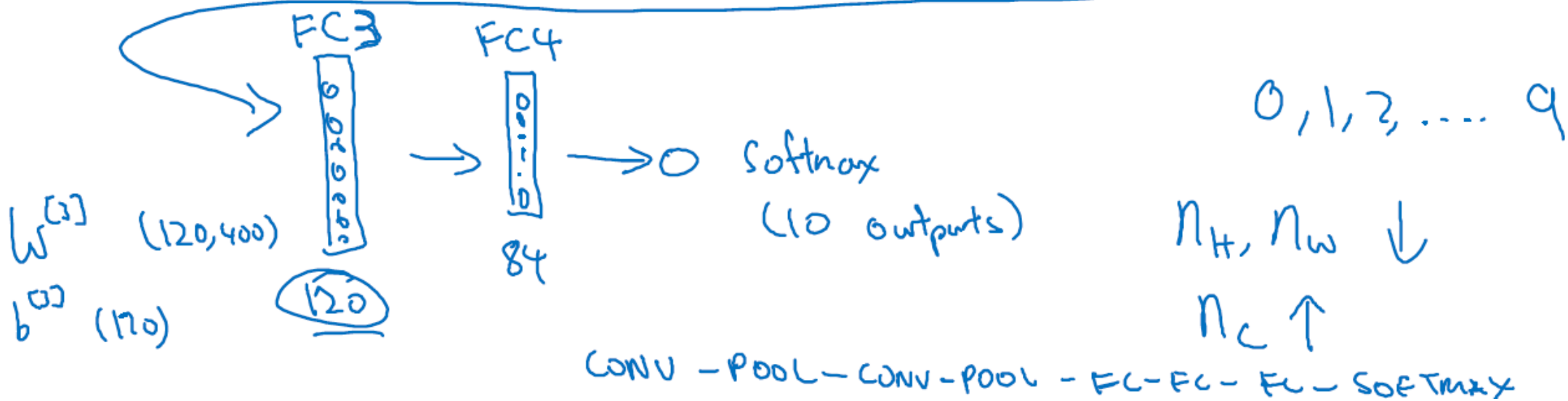
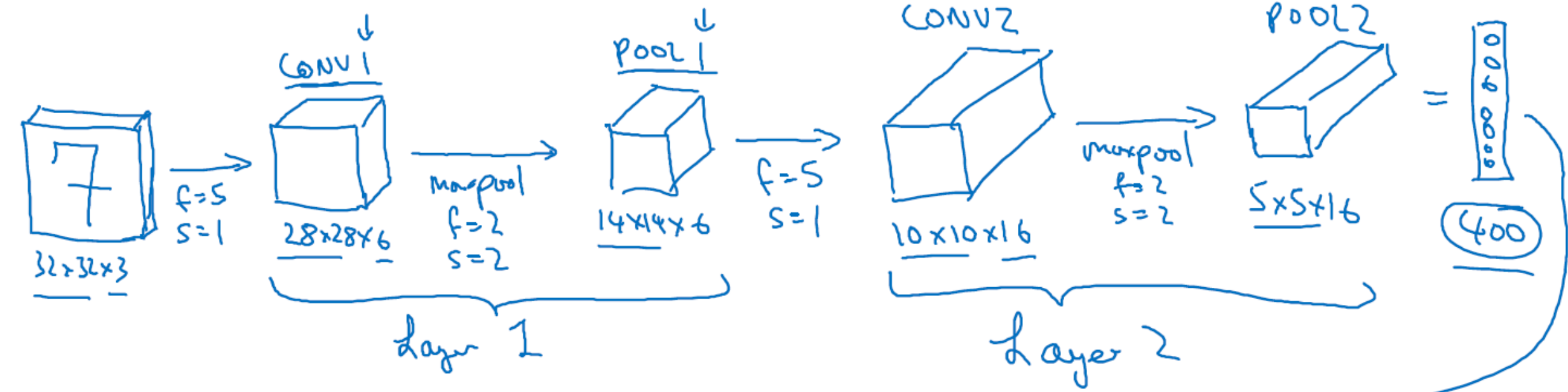


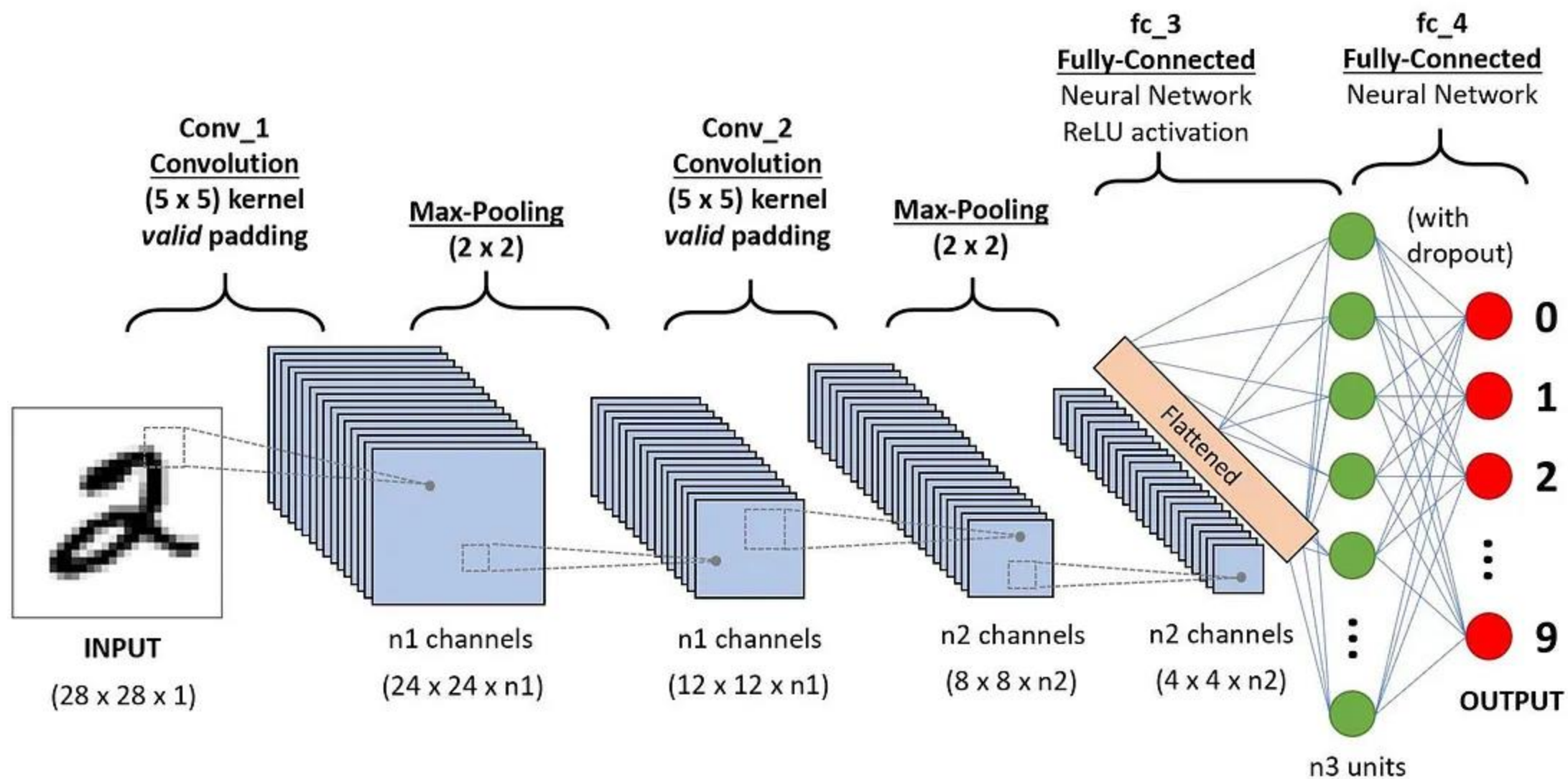


Example ConvNet



Neural network example (LeNet-5)





◆ Problem Statement

Consider a **Convolutional Neural Network (CNN)** that takes a **$32 \times 32 \times 3$ RGB image** as input.

The network architecture is as follows (no padding used anywhere):

Layer	Type	Filter Size / Units	Stride	Number of Filters
1	Convolution	5×5	1	6
2	Max Pooling	2×2	2	—
3	Convolution	5×5	1	16
4	Max Pooling	2×2	2	—
5	Fully Connected	—	—	120 neurons
6	Fully Connected	—	—	84 neurons
7	Output Layer	—	—	10 classes

- **Describe the CNN architecture**
- **Calculate the output size after each convolution and pooling layer.**
- **Compute the number of parameters in each layer.**

Layer	Input Size	Filter	Stride	Output Size
Conv1	$32 \times 32 \times 3$	5×5	1	$(32-5)/1 + 1 = 28 \times 28 \times 6$
Pool1	$28 \times 28 \times 6$	2×2	2	$(28-2)/2 + 1 = 14 \times 14 \times 6$
Conv2	$14 \times 14 \times 6$	5×5	1	$(14-5)/1 + 1 = 10 \times 10 \times 16$
Pool2	$10 \times 10 \times 16$	2×2	2	$(10-2)/2 + 1 = 5 \times 5 \times 16$
Flatten	$5 \times 5 \times 16 = 400$	—	—	400
FC1	400	—	—	120
FC2	120	—	—	84
Output	84	—	—	10

Code for CNN

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models

# 1) Load data
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

# 2) Preprocess
# Scale to [0,1], add channel dim => (N, 28, 28, 1)
x_train = (x_train.astype("float32") / 255.0)[..., None]
x_test  = (x_test.astype("float32") / 255.0)[..., None]

num_classes = 10
y_train_cat = keras.utils.to_categorical(y_train, num_classes)
y_test_cat  = keras.utils.to_categorical(y_test, num_classes)

print("Train:", x_train.shape, " Test:", x_test.shape)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 ————— 0s 0us/step

Train: (60000, 28, 28, 1) Test: (10000, 28, 28, 1)

Calculate #parameters?

```
model = models.Sequential([
    layers.Input(shape=(28, 28, 1)),

    layers.Conv2D(32, kernel_size=3, strides=1, activation="relu"),
    layers.MaxPooling2D(pool_size=2, strides=2), # default padding is valid

    layers.Conv2D(64, kernel_size=3, strides=1, activation="relu"),
    layers.MaxPooling2D(pool_size=2, strides=2),

    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dense(num_classes, activation="softmax")
])

model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)

Trainable params: 225,034 (879.04 KB)

Non-trainable params: 0 (0.00 B)

4) Compile

```
model.compile(optimizer=keras.optimizers.Adam(1e-3),  
              loss="categorical_crossentropy",  
              metrics=["accuracy"])
```

5) Train (keep epochs small for a quick run)

```
history = model.fit(  
    x_train, y_train_cat,  
    validation_split=0.1,  
    epochs=5,  
    batch_size=128,  
    verbose=1  
)
```

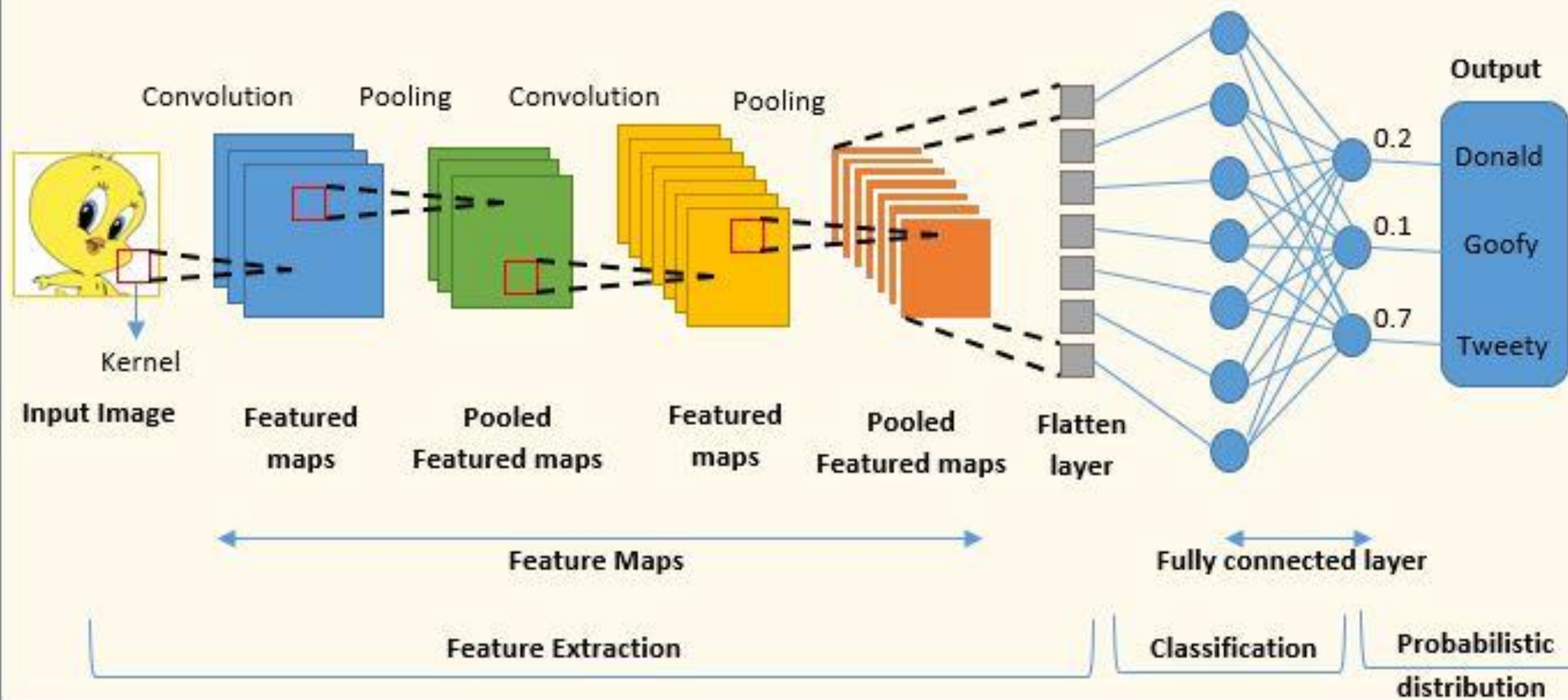
```
# 6) Evaluate
```

```
test_loss, test_acc = model.evaluate(x_test, y_test_cat, verbose=0)
```

```
print(f"Test accuracy: {test_acc:.4f}")
```

Test accuracy : 0.9916

A Typical Convolutional Neural Network (CNN)

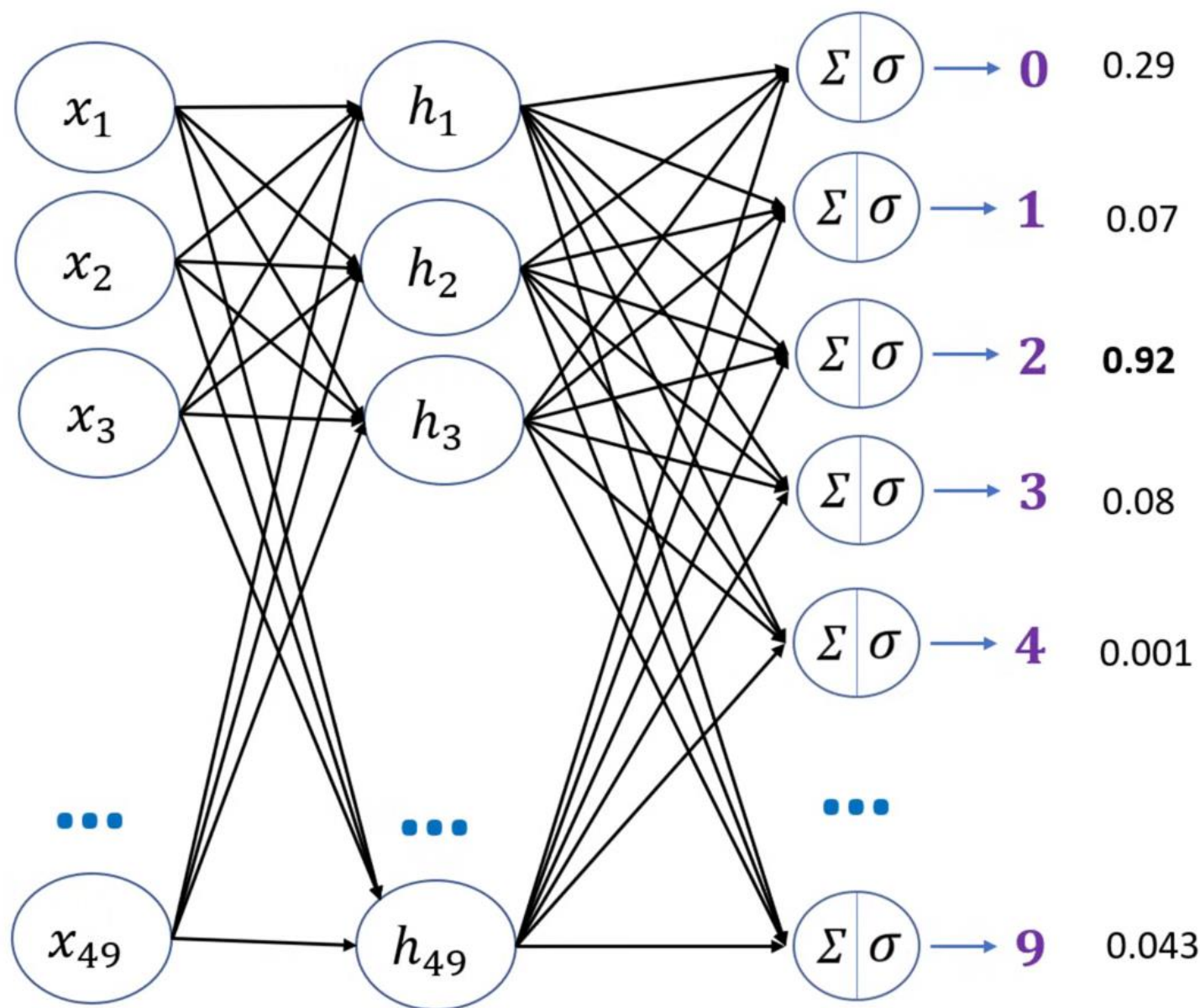
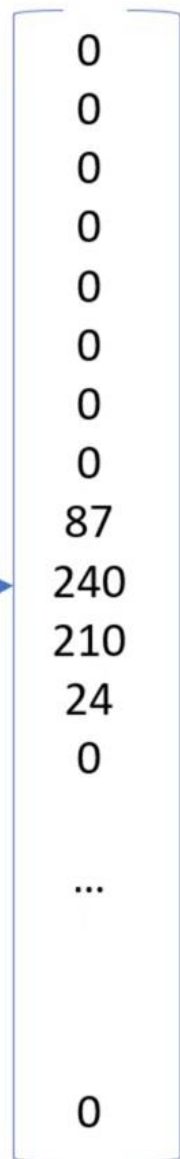


Intuitive explanation of CNN filters



0	0	0	0	0	0	0
0	87	240	210	24	0	0
0	13	0	101	195	0	0
0	35	167	99	210	0	0
0	145	230	240	201	189	140
0	0	102	67	17	13	0
0	0	0	0	0	0	0

7 by 7 grid

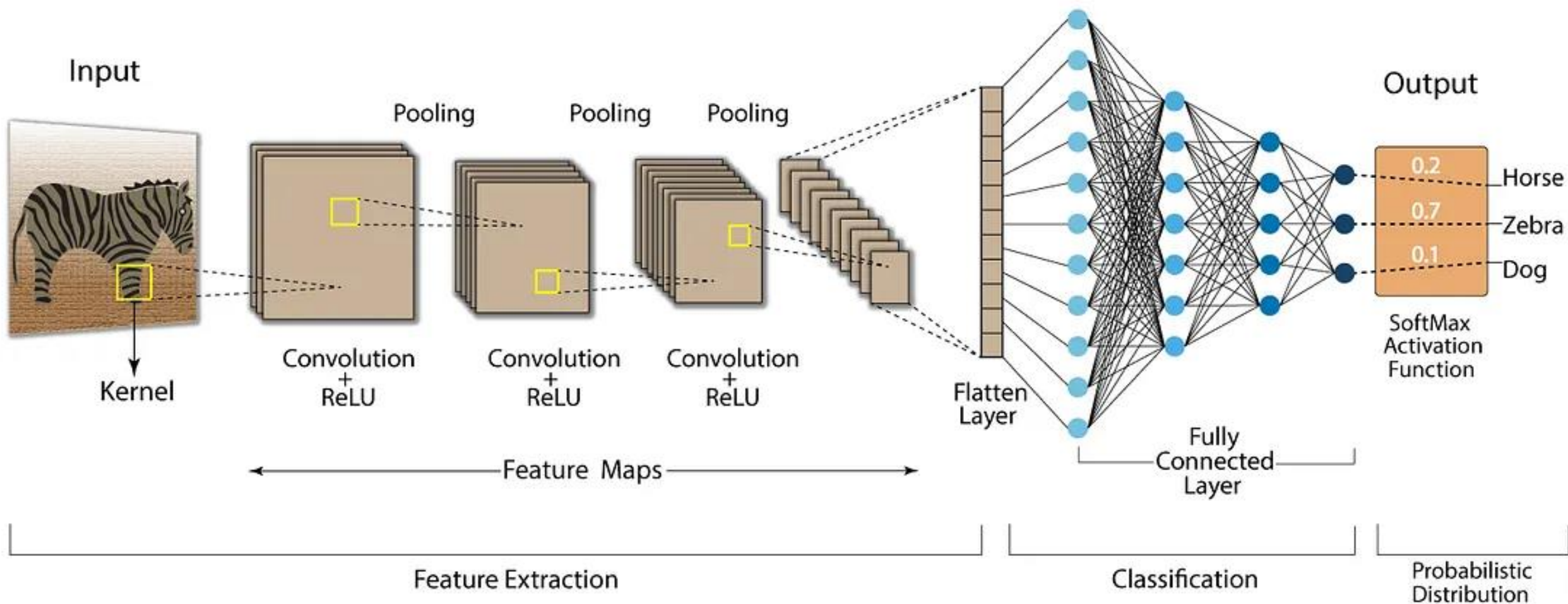


$$49 * 49 = 2401$$

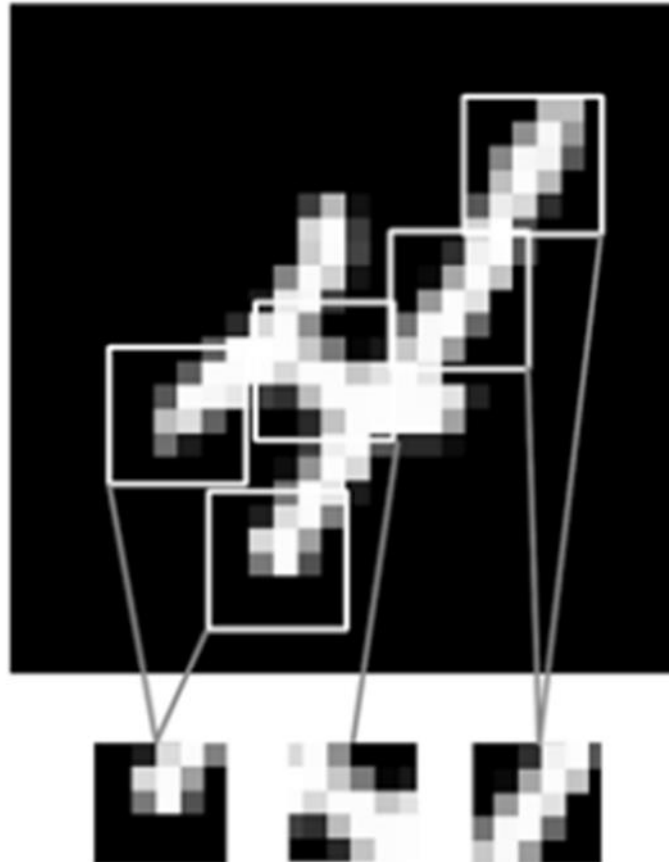
$$49 \cdot 10 = 490$$

What is the problem in Dense layers (Fully connected NN)?

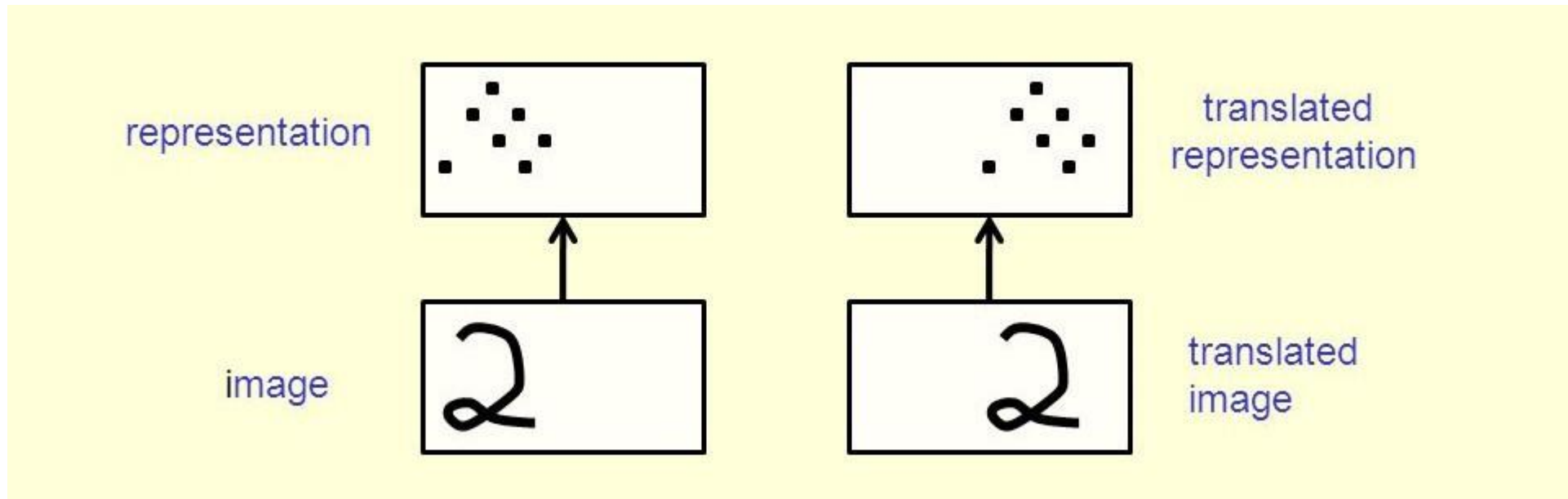
1. Too much computation
2. Treats local pixels same as pixels far apart
3. Sensitive to location of an object in an image



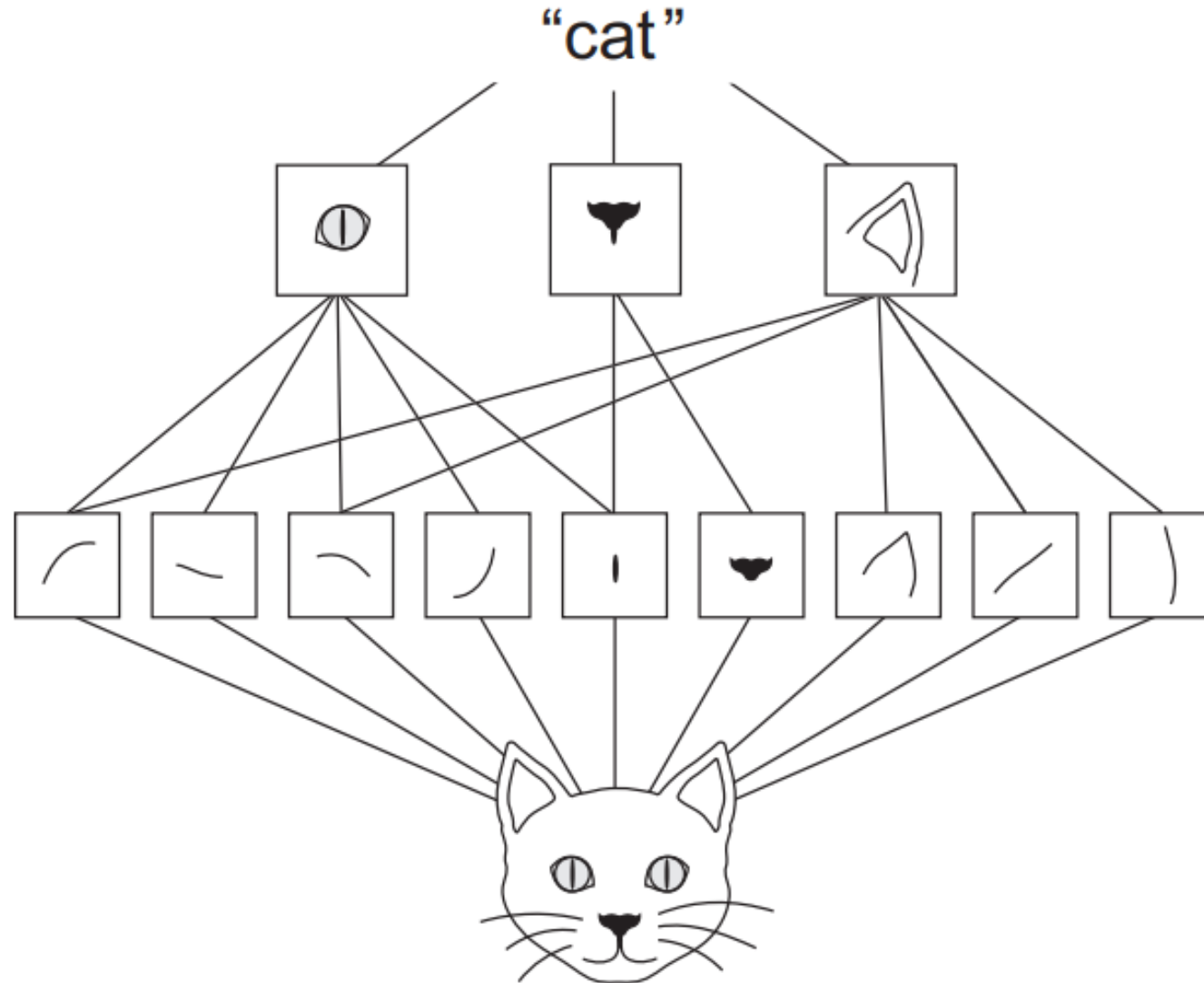
The fundamental difference between a densely connected layer and a convolution layer is this: **Dense layers learn global patterns in their input feature space (for example, for a MNIST digit, patterns involving all pixels), whereas convolution layers learn local patterns**

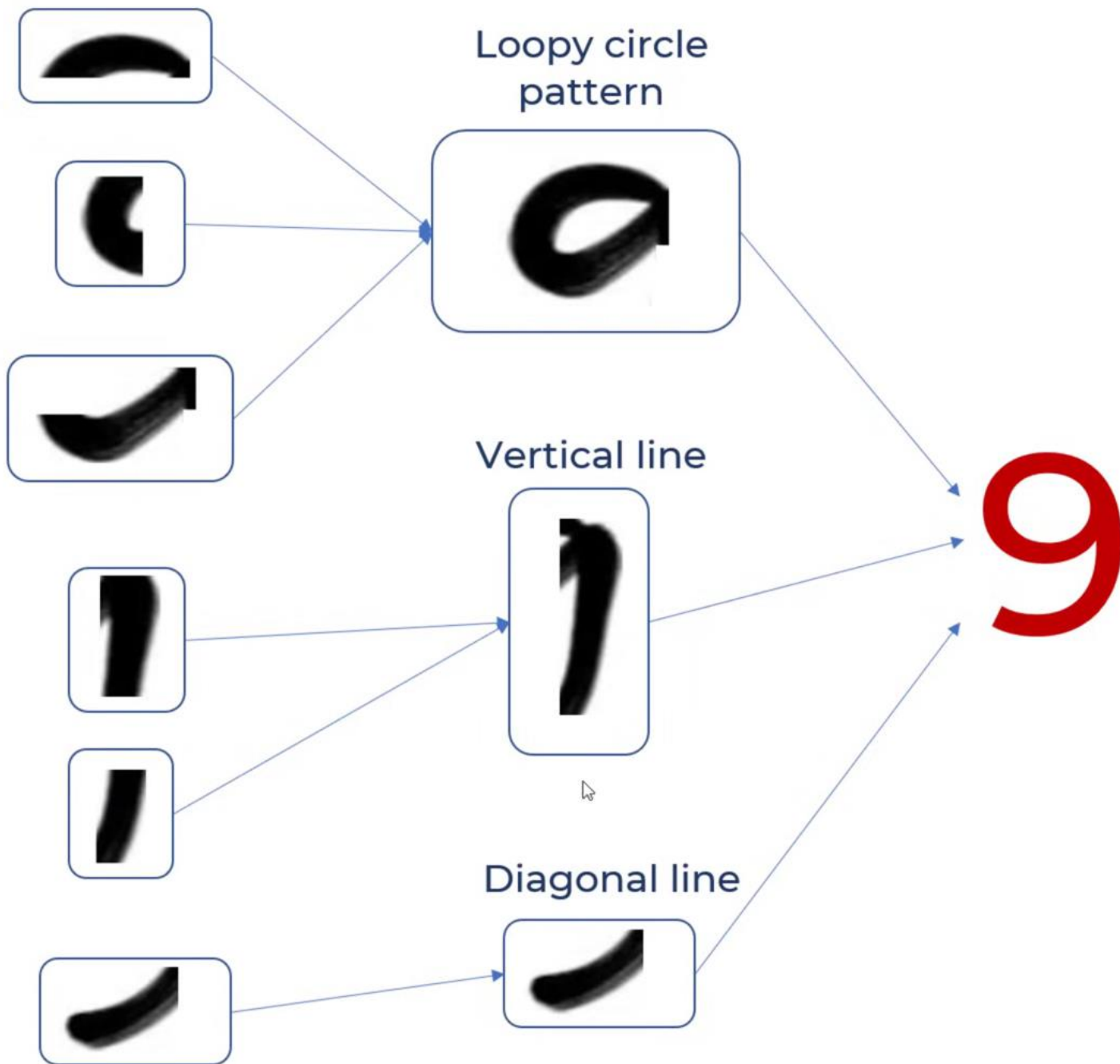


The patterns they learn are translation invariant



They can learn spatial hierarchies of patterns





Loopy pattern detector

$$\begin{matrix} 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & 1 \end{matrix} = \begin{matrix} & 1 & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \end{matrix}$$

Vertical line detector

$$\begin{matrix} -1 & 1 & -1 \\ -1 & 1 & -1 \\ -1 & 1 & -1 \end{matrix} = \begin{matrix} & & & & \\ & & & & \\ & & 1 & & \\ & & & & \\ & & & & \end{matrix}$$

Diagonal line detector

$$\begin{matrix} -1 & -1 & 1 \\ -1 & 1 & -1 \\ 1 & -1 & -1 \end{matrix} = \begin{matrix} & & & & \\ & & & & \\ & & & & \\ & & 1 & & \\ & & & & \end{matrix}$$

Loopy pattern detector

 *

1	1	1
1	-1	1
1	1	1

=

	1	

[illegible]

Loopy pattern detector



1	1	1
1	-1	1
1	1	1

	1			
	1			

Loopy pattern detector

The image shows a large handwritten digit "96" in black ink. To its right is a multiplication symbol "*". Further right is a 3x3 grid of green squares containing white numbers, representing a kernel or filter.

1	1	1
1	-1	1
1	1	1

=

The image shows a 5x5 grid of light gray squares. Two squares are highlighted in green: one at row 1, column 2 (top-left) and one at row 5, column 4 (bottom-right). Both green squares contain the number "1". A mouse cursor arrow points to the bottom-right green square.



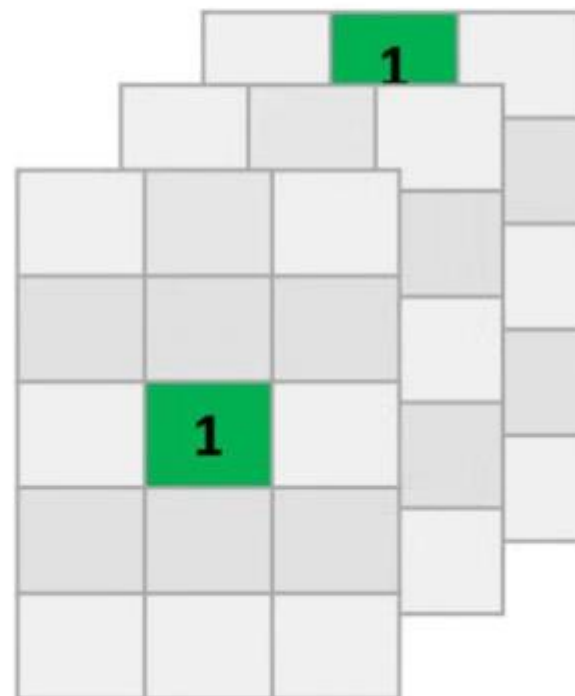
*

Filters



=

Feature Maps



-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

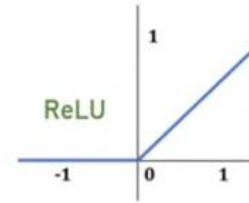
*

Loopy pattern
filter

1	1	1
1	-1	1
1	1	1



-0.11	1	-0.11
-0.55	0.11	-0.33
-0.33	0.33	-0.33
-0.22	-0.11	-0.22
-0.33	-0.33	-0.33



0	1	0
0	0.11	0
0	0.33	0
0	0	0
0	0	0

The koala is an iconic Australian animal.





Koala's **eye**? = Y



Koala's **nose**? = Y



Koala's **ears**? = Y



Koala's **hands**? = Y



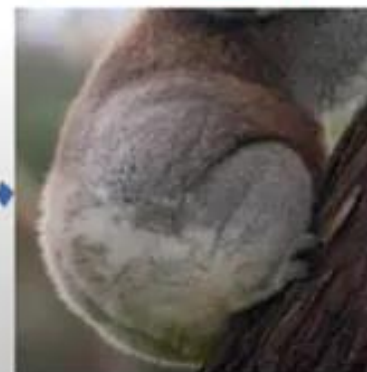
Koala's **legs**? = Y



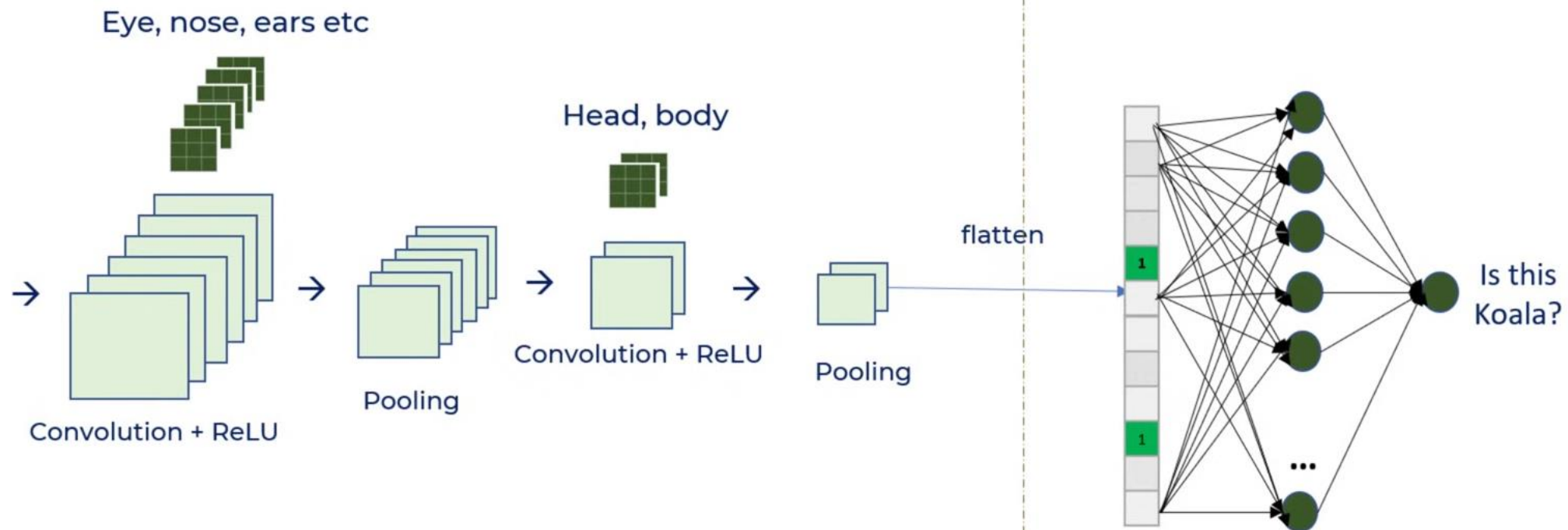
Koala's **head**? = Y



Koala's **body**? = Y



Is it **Koala**? = Y



Feature Extraction

Classification

Consider a scenario: Training a convnet from scratch on a small dataset

- A small image dataset → with a smaller number of samples per class
- What can happen if we apply representation learning (deep learning model)?
- Is it possible to create a high quality embedding space with limited data?

What actions can be taken to get a high-performing model?

Example: Dogs vs. Cats dataset

You can download the original dataset from www.kaggle.com/c/dogs-vs-cats/data

The pictures are medium-resolution color JPEGs



- 2,000 training images, 1,000 validation images, and 1,000 test images.
- Each split contains the same number of samples from each class:
- This is a balanced **binary-classification problem**

```
from keras import layers
from keras import models

model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu',
                        input_shape=(150, 150, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Flatten())
model.add(layers.Dense(512, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
```

```
>>> model.summary()
```

Layer (type)	Output Shape	Param #
=====		
conv2d_1 (Conv2D)	(None, 148, 148, 32)	896
maxpooling2d_1 (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_2 (Conv2D)	(None, 72, 72, 64)	18496
maxpooling2d_2 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_3 (Conv2D)	(None, 34, 34, 128)	73856
maxpooling2d_3 (MaxPooling2D)	(None, 17, 17, 128)	0
conv2d_4 (Conv2D)	(None, 15, 15, 128)	147584
maxpooling2d_4 (MaxPooling2D)	(None, 7, 7, 128)	0
flatten_1 (Flatten)	(None, 6272)	0
dense_1 (Dense)	(None, 512)	3211776
dense_2 (Dense)	(None, 1)	513
=====		

```
Total params: 3,453,121
```

```
Trainable params: 3,453,121
```

```
Non-trainable params: 0
```

```
from keras import optimizers

model.compile(loss='binary_crossentropy',
              optimizer=optimizers.RMSprop(lr=1e-4),
              metrics=['acc'])
```

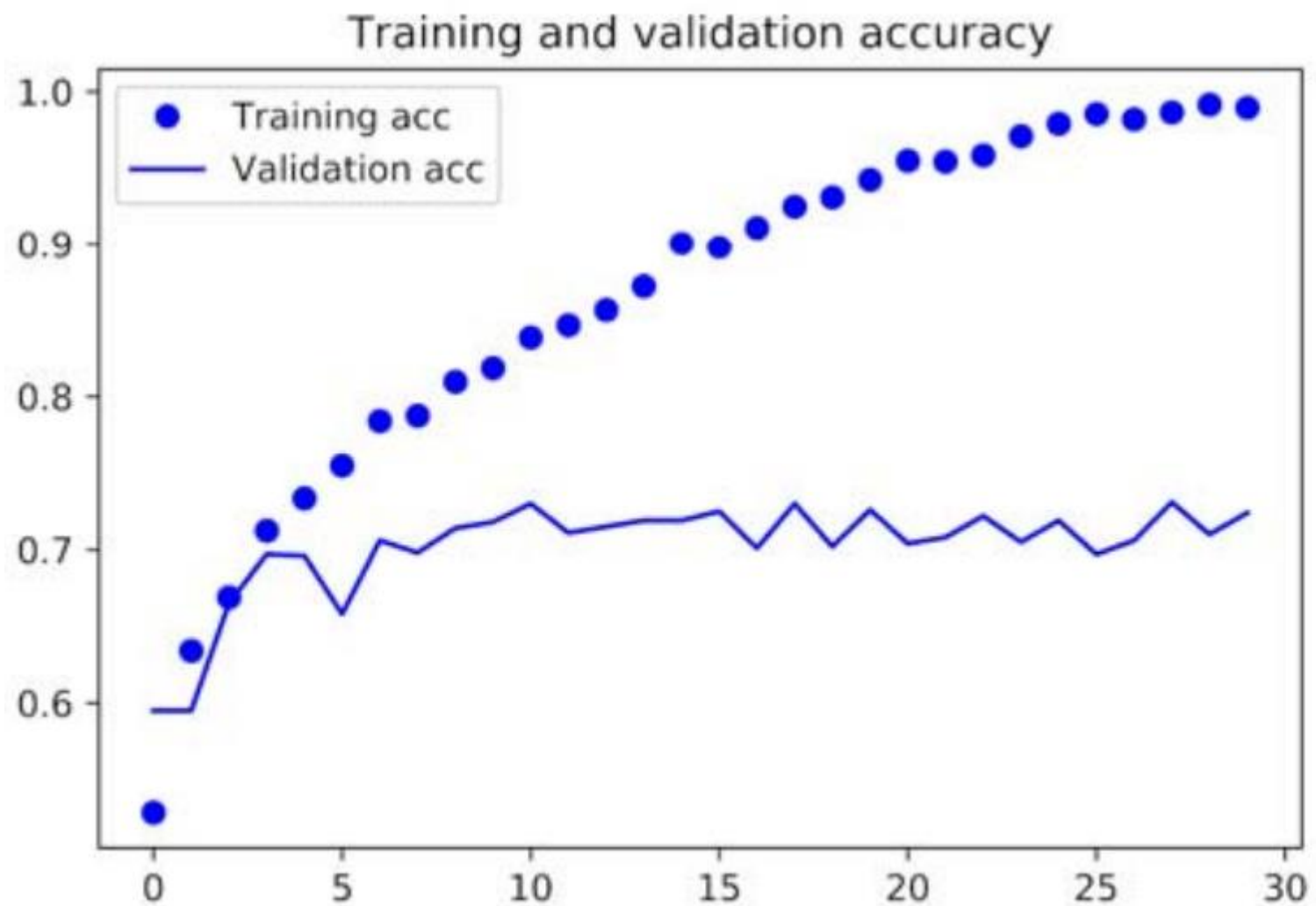


Figure 5.9 Training and validation accuracy

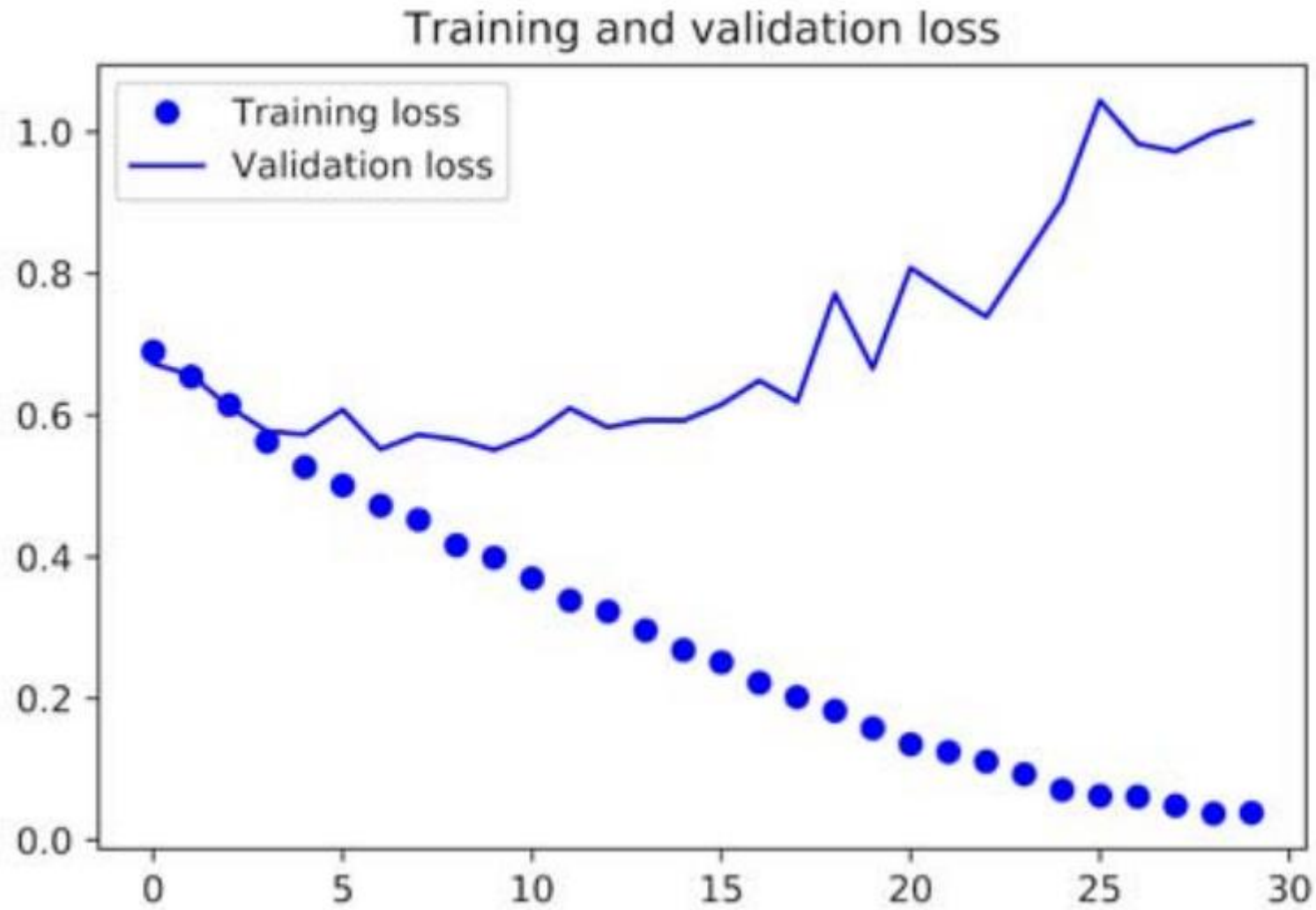


Figure 5.10 Training and validation loss

Observation

- These plots are characteristic of overfitting.
- The training accuracy increases linearly over time, until it reaches nearly 100%, whereas the validation accuracy stalls at 70–72%.
- The validation loss reaches its minimum after only five epochs and then increases, whereas the training loss keeps decreasing linearly until it reaches nearly 0.

Why???

Why???

....Because you have relatively few training samples (2,000), overfitting will be your number-one concern.

Approach 1: Adding weight regularization

- A common way to mitigate overfitting is to put constraints on the complexity of a network by forcing its weights to take only small values, which makes the distribution of weight values more regular. This is called weight regularization.
- It's done by adding to the loss function of the network a cost associated with having large weights.

We modify the loss as:

$$L = L_0 + \lambda R(W)$$

where

- $R(W)$ = regularization term,
- $\lambda > 0$ = regularization strength (controls penalty).

Suppose you use MSE loss:

$$L_0 = \frac{1}{2N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Then **with L2 regularization**:

$$L = \frac{1}{2N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \frac{\lambda}{2} \sum_j w_j^2$$

During gradient descent:

$$w_j \leftarrow w_j - \eta \left(\frac{\partial L_0}{\partial w_j} + \lambda w_j \right)$$

where η is the learning rate.

Approach 2: Apply Dropout

When you train a neural network, neurons (hidden units) can become **too dependent** on specific other neurons — like a few “overconfident teammates” always working together.

Dropout breaks this dependency by randomly turning off (dropping) some neurons during training.

Think of it as a *group project* where each day, random team members are absent — so everyone learns to work independently and contribute meaningfully.

How to apply dropout

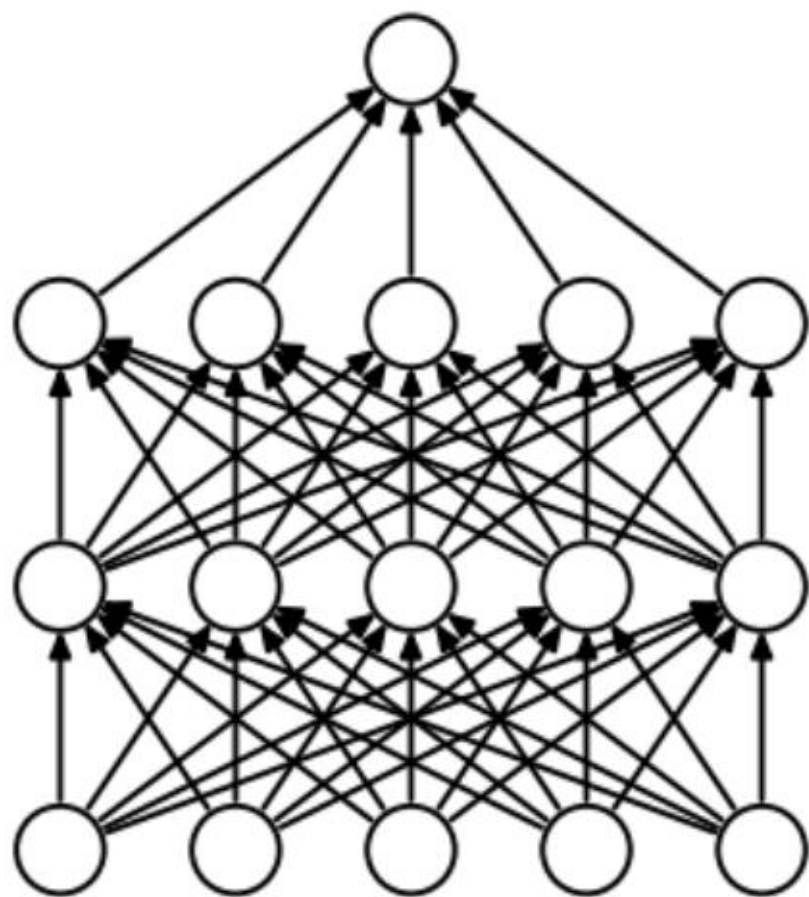
- During training, it randomly drops out (sets to zero) a certain fraction of the neuron outputs in a layer.
- Suppose a layer produces an output vector:
 $[0.2, 0.5, 1.3, 0.8, 1.1]$.
- After applying dropout, with some units dropped randomly, it might become:
 $[0, 0.5, 1.3, 0, 1.1]$.

Dropout Rate (p):

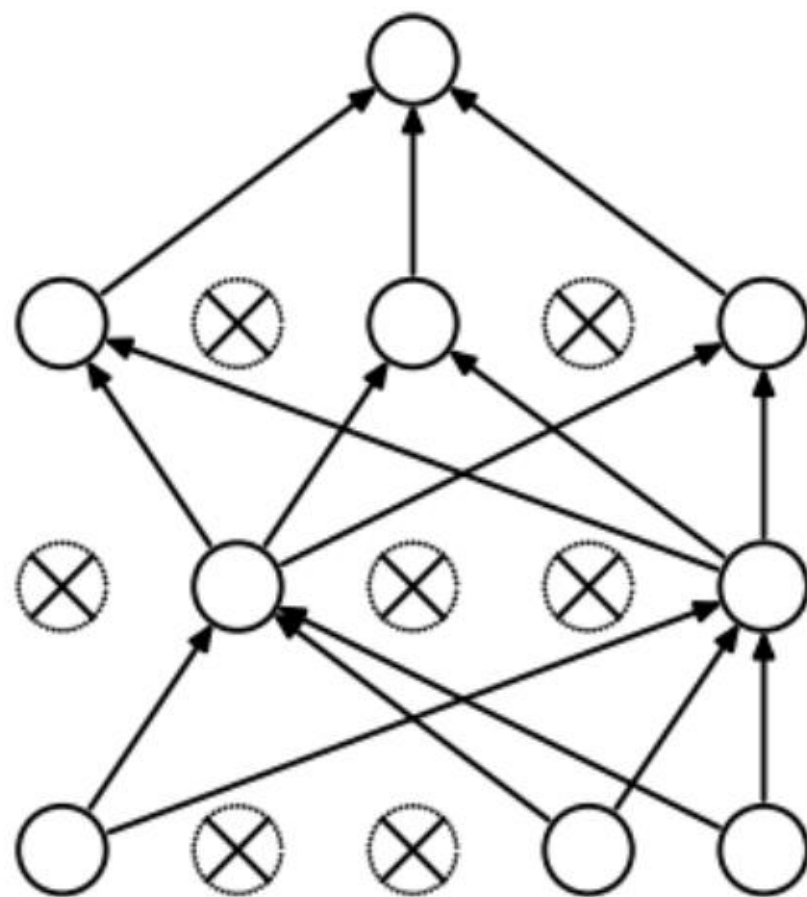
- Defines the **fraction of neurons** to be dropped (set to zero).
- Typical values: **0.2 to 0.5**.
- Example: $p = 0.5$ means half of the neurons are deactivated randomly in each training step.

Training Phase Behavior:

- Each iteration trains on a **different subset of neurons**.
- This forces neurons to **learn independently**, not relying on specific others.



(a) Standard Neural Net



(b) After applying dropout.

0.3	0.2	1.5	0.0
0.6	0.1	0.0	0.3
0.2	1.9	0.3	1.2
0.7	0.5	1.0	0.0

50%
dropout

0.0	0.2	1.5	0.0
0.6	0.1	0.0	0.3
0.0	1.9	0.3	0.0
0.7	0.0	0.0	0.0

Suppose you're preparing for an exam in a study group.

If you always rely on one friend to solve calculus, you might fail when that friend isn't there.

Dropout forces you to study as if your friend might not show up — so you learn to handle all types of problems on your own.

That's exactly what it does to neurons — they learn robust, independent features.

Dropout try to teach the network to “stand on its own feet” by breaking up neuron collaborations —resulting in a stronger, more general, and less overfitted model.

Regularization & Dropout

- Dropout makes the *model* robust to neuron loss.
- Try to achieve better generalization
- Basic problem arises from the data size limitation
- The capability of dropout is limited. It can restrict the model from overfitting to some extent.

Two popular methods

- **Data augmentation** → increase the dataset size by creating new data samples from the existing dataset.
- **Transfer learning** → Pre-trained model is used as a reference for target task

Data augmentation

- **Data augmentation** is a technique used to artificially increase the diversity and quantity of training data by applying various transformations—such as rotation, flipping, scaling, cropping, adding noise, or changing brightness—to the existing samples.
- Intuitively, it teaches the model that *the same object can look different* in the real world. For example, a cat might appear in different lighting, angles, or positions, but it's still a cat.
- By showing these varied versions to the model, we prevent it from memorizing specific examples and force it to learn the **core patterns** that define the object or signal.



Data augmentation makes the model focus on **what truly matters**—the invariant features—rather than surface details.

Figure 5.11 Generation of cat pictures via random data augmentation

Augmentation Type	Example Effect
Rotation	Turn the image by $+15^{\circ}$ or -20°
Flipping	Mirror the mushroom horizontally
Zoom / Crop	Zoom in or crop a random portion
Brightness / Contrast	Make the image darker or lighter
Noise / Blur	Add random grain or small blur
Translation	Shift the mushroom slightly up, down, left, or right

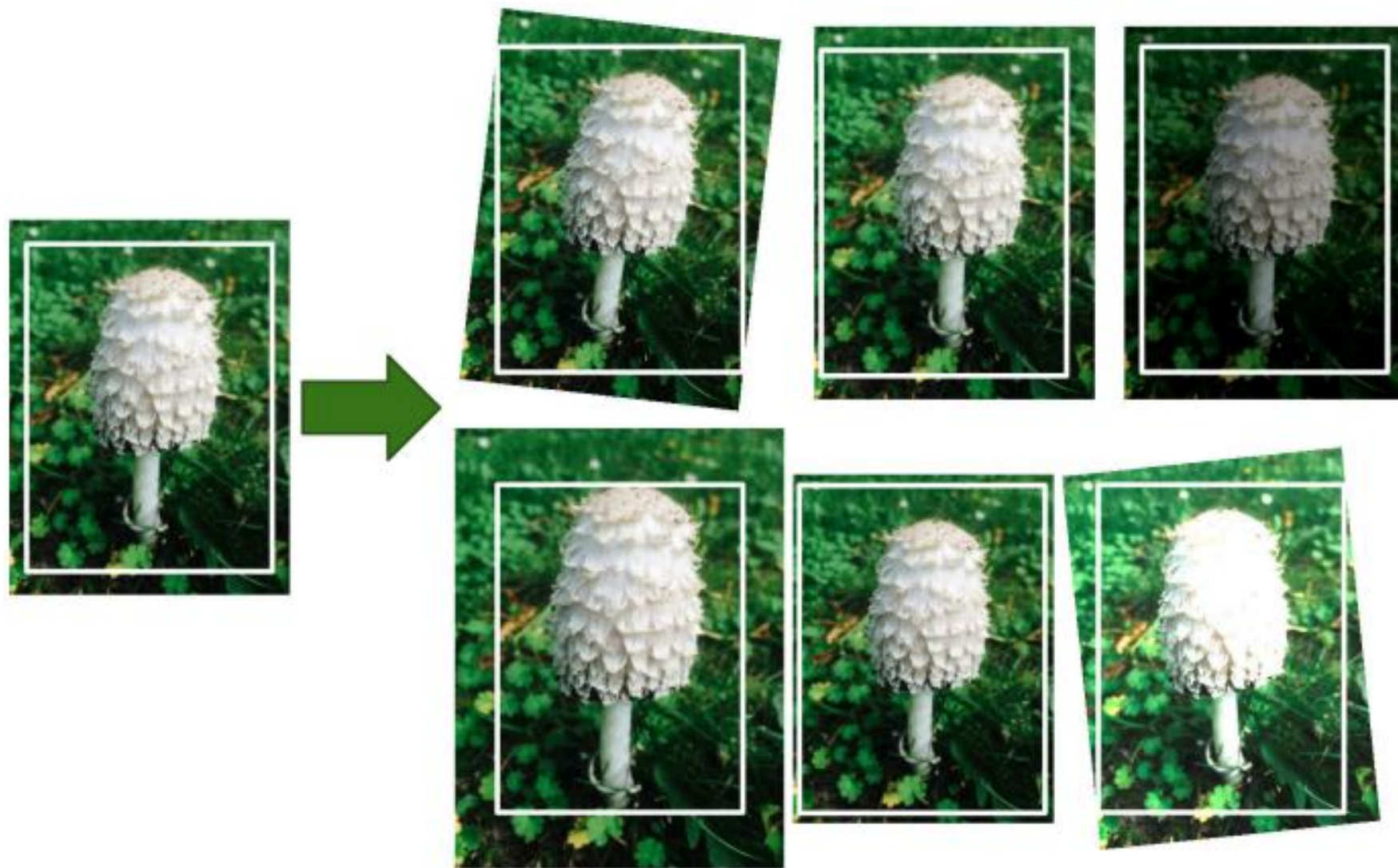
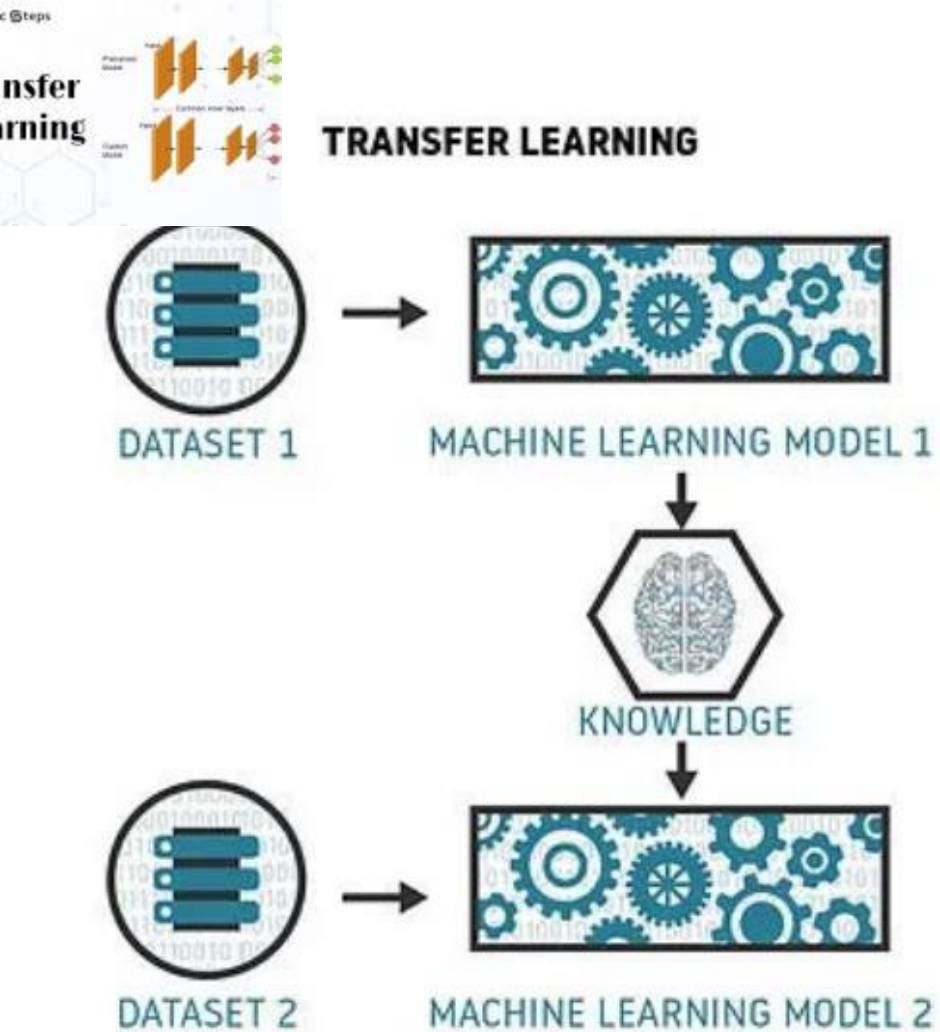
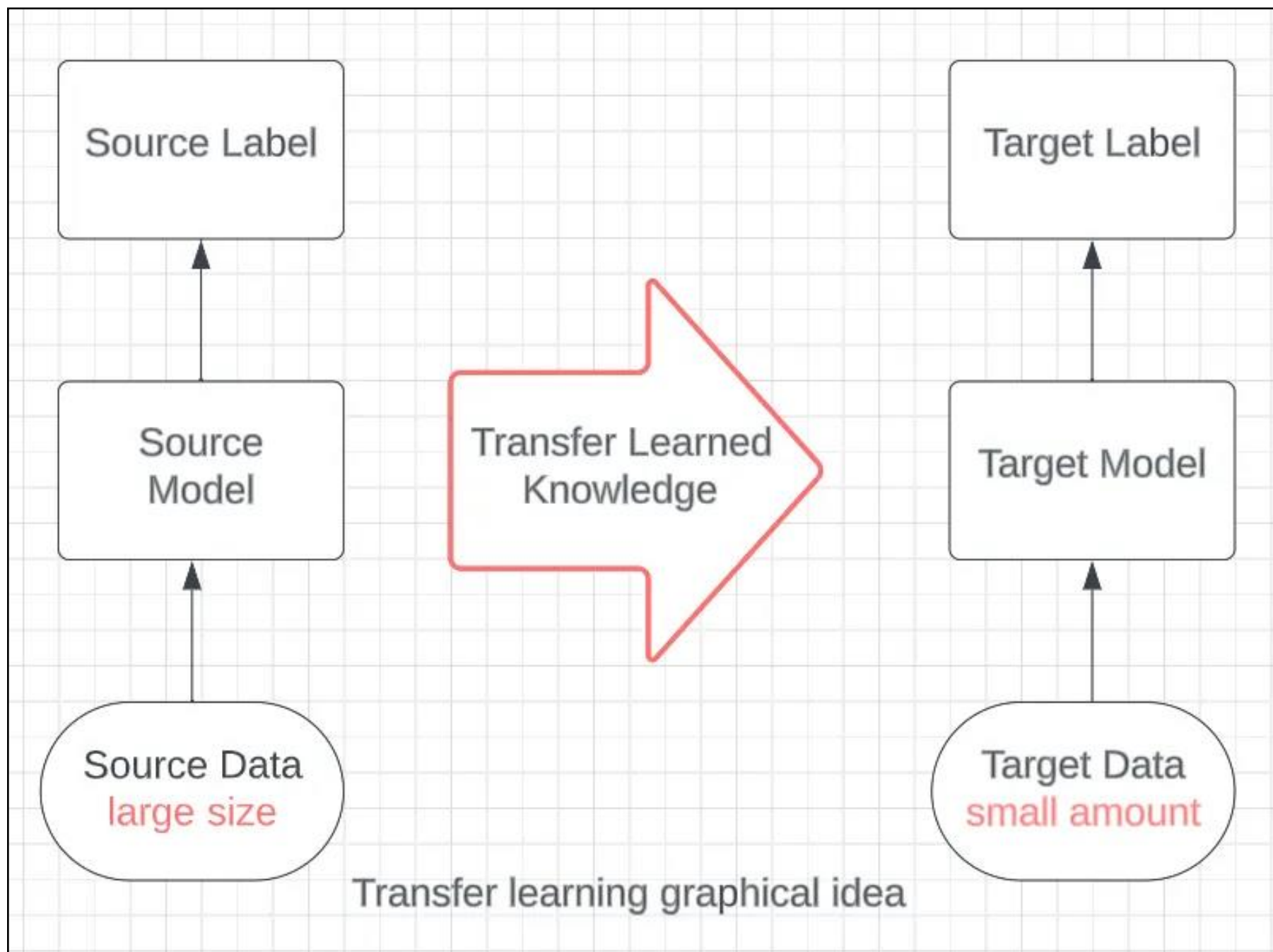


Figure 14-12. Generating new training instances from existing ones

Data augmentation **teaches the model that identity doesn't change under small transformations** — the mushroom is still a mushroom whether it's brighter, rotated, or zoomed.



Transfer learning is an approach to machine learning where a model trained on one task is used as the starting point for a model on a new task. This is done by transferring the knowledge that the first model has learned about the features of the data to the second model.



Pretrained network

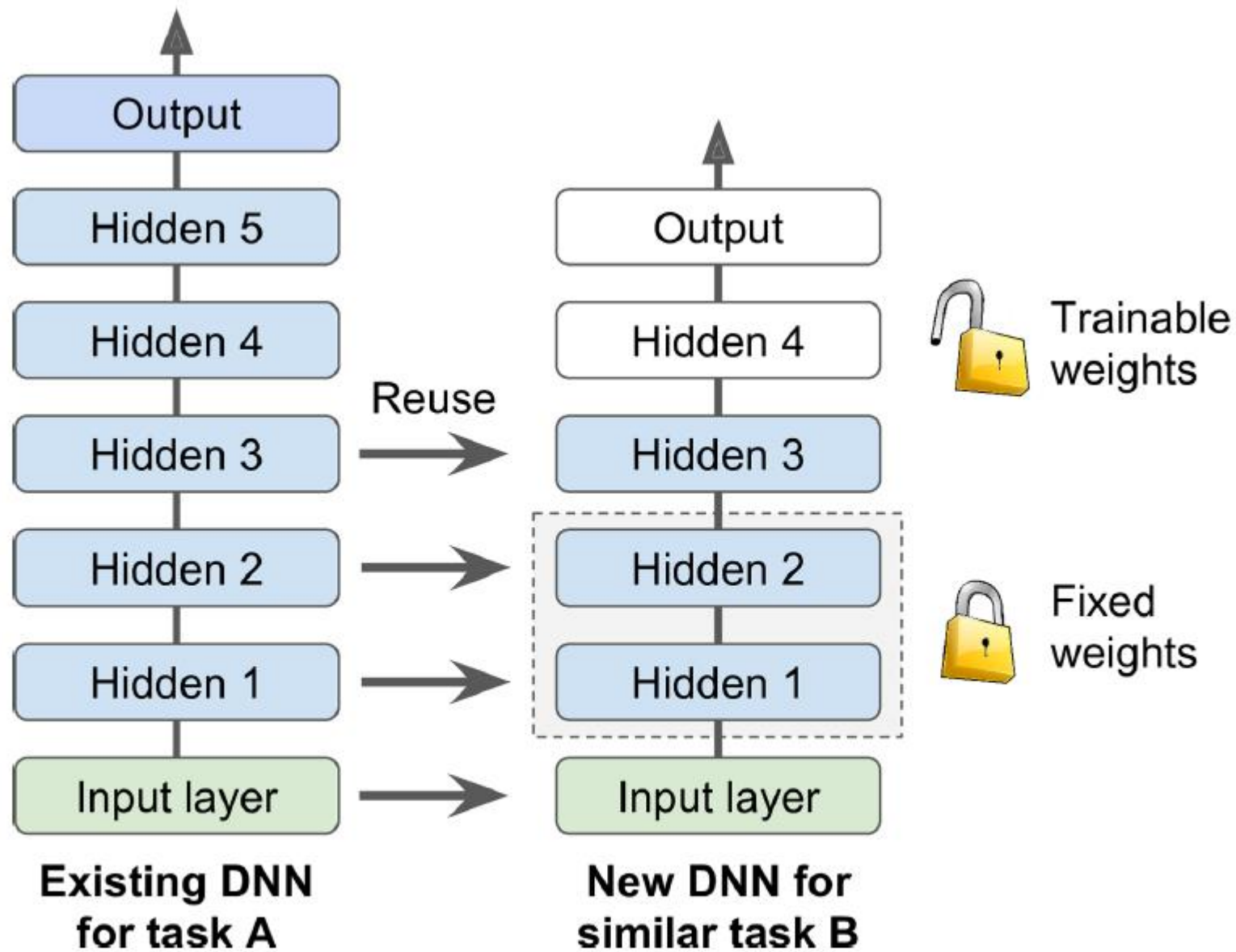
- A pretrained network is a saved network that was previously trained on a large dataset, typically on a large-scale image-classification task.
- If this original dataset is large enough and general enough, then the spatial hierarchy of features learned by the pretrained network can effectively act as a generic model of the visual world.
- Features extracted from its embedding space can be useful for many different computer vision problems, even though these new problems may involve completely different classes than those of the original task

Model Name	Trained On Dataset
VGG16	ImageNet
VGG19	ImageNet
ResNet50	ImageNet
ResNet101 / 152	ImageNet
InceptionV3	ImageNet
InceptionResNetV2	ImageNet
Xception	ImageNet
DenseNet121	ImageNet
DenseNet169 / 201	ImageNet
MobileNetV1	ImageNet
MobileNetV2	ImageNet

Keras Pretrained Models and Their Training Datasets

Simple analogy...

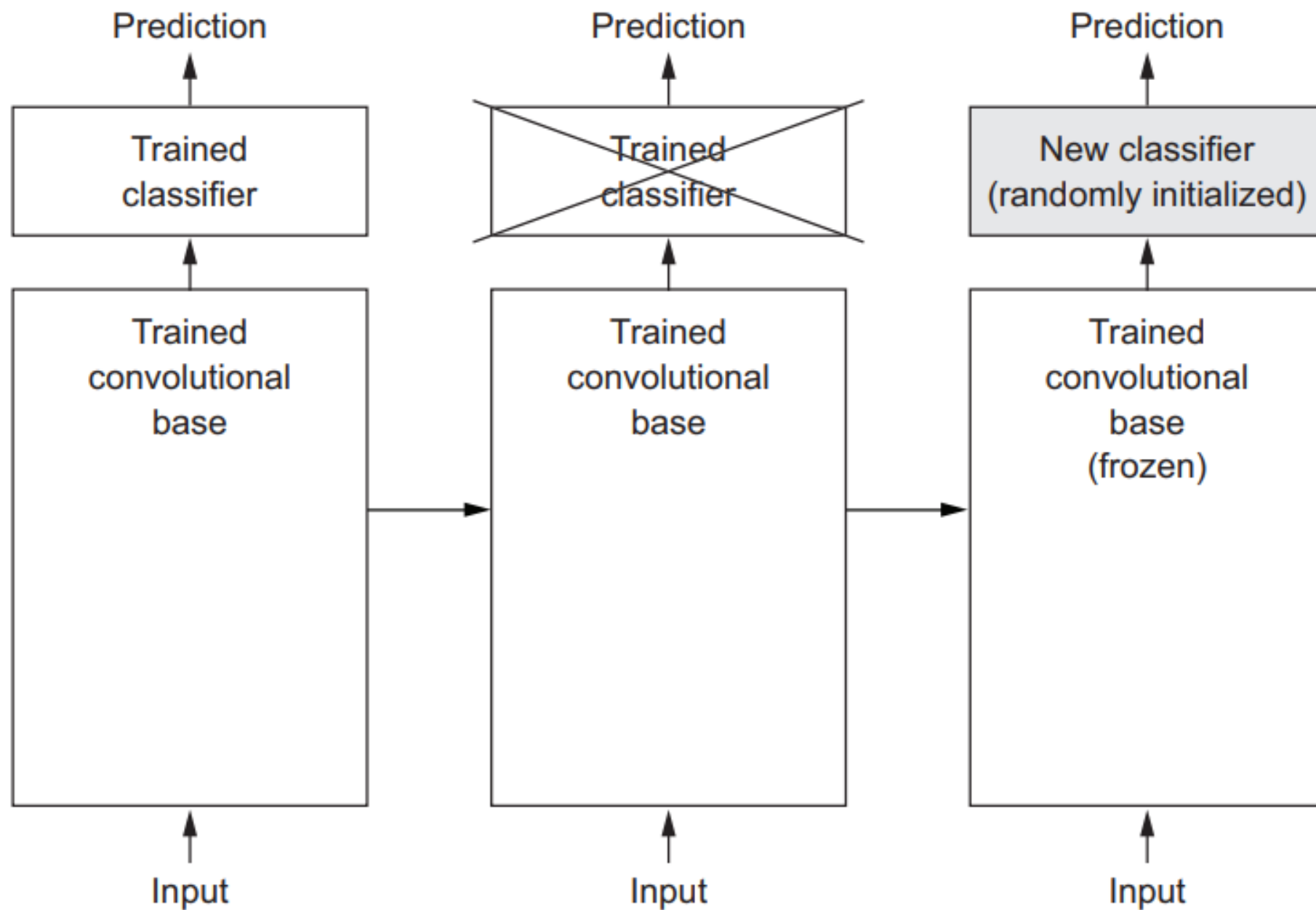
- Suppose you know how to add, sub, mul, div. You are very well trained in your high school.
- You are asked to learn $d(u/v)$, or more complex numerical computation....



Reusing pretrained layers

Why upper-level layers are removed??

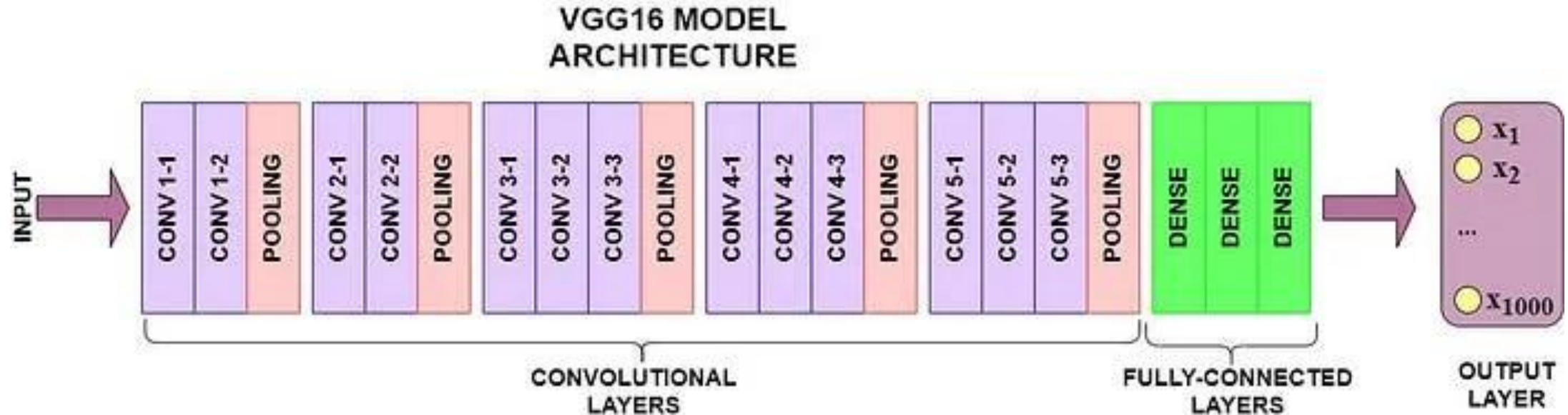
- The output layer of the original model should usually be replaced **since it is most likely not useful at all for the new task, and it may not even have the right number of outputs for the new task.**
- Similarly, the upper hidden layers of the original model are less likely to be as useful as the lower layers, **since the high-level features that are most useful for the new task may differ significantly from the ones that were most useful for the original task.**



VGGNet (Visual Geometry Group Network) is a deep convolutional neural network architecture developed by the Visual Geometry Group at Oxford University.

A large convnet trained on the ImageNet dataset (1.4 million labeled images and 1,000 different classes).

ImageNet contains many animal classes.



```
from keras.applications import VGG16  
  
conv_base = VGG16(weights='imagenet',  
                    include_top=False,  
                    input_shape=(150, 150, 3))
```

Total params: 14,714,688

Trainable params: 14,714,688

Non-trainable params: 0

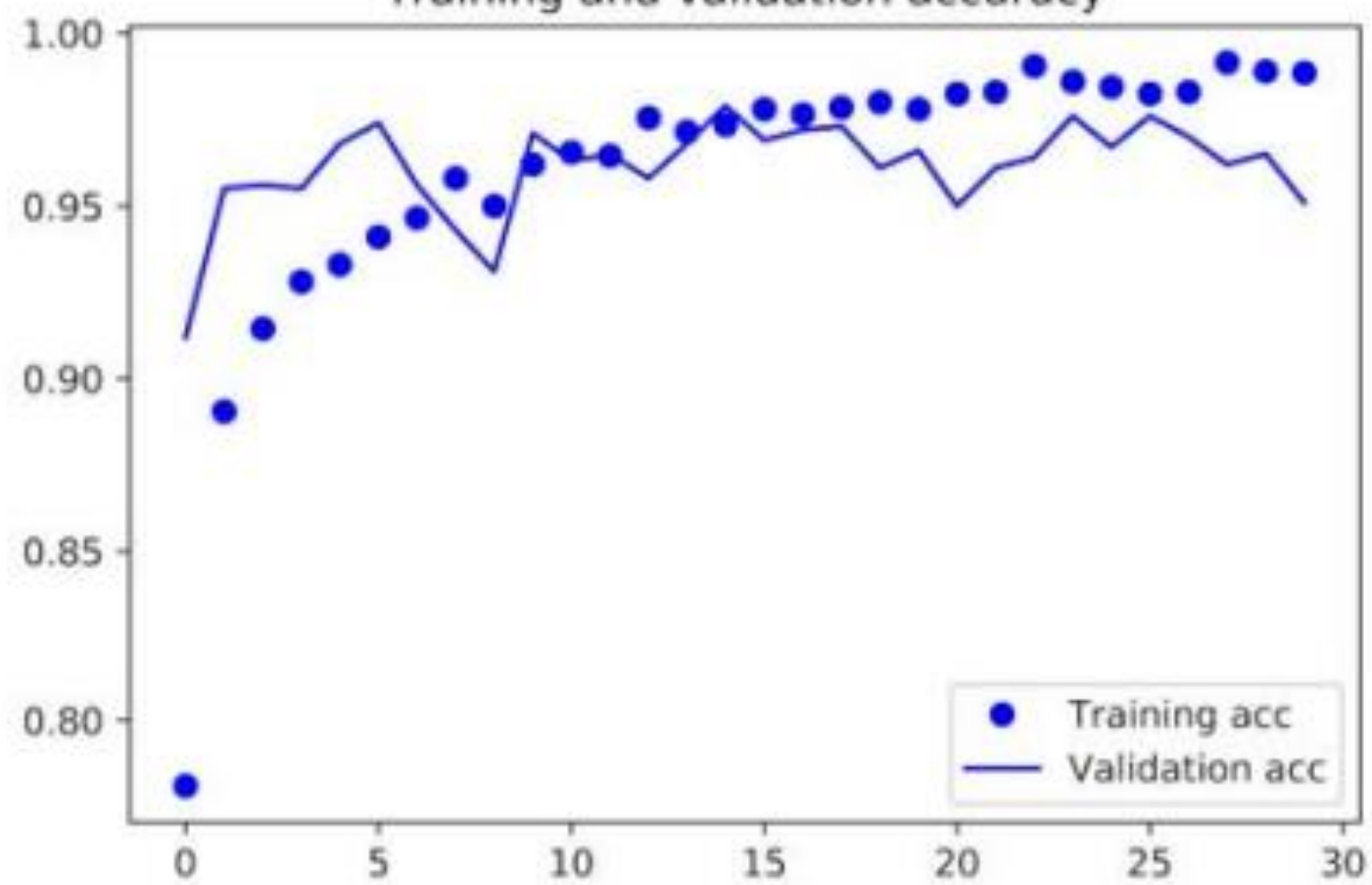
```
from keras import models
from keras import layers

model = models.Sequential()
model.add(conv_base)
model.add(layers.Flatten())
model.add(layers.Dense(256, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
```

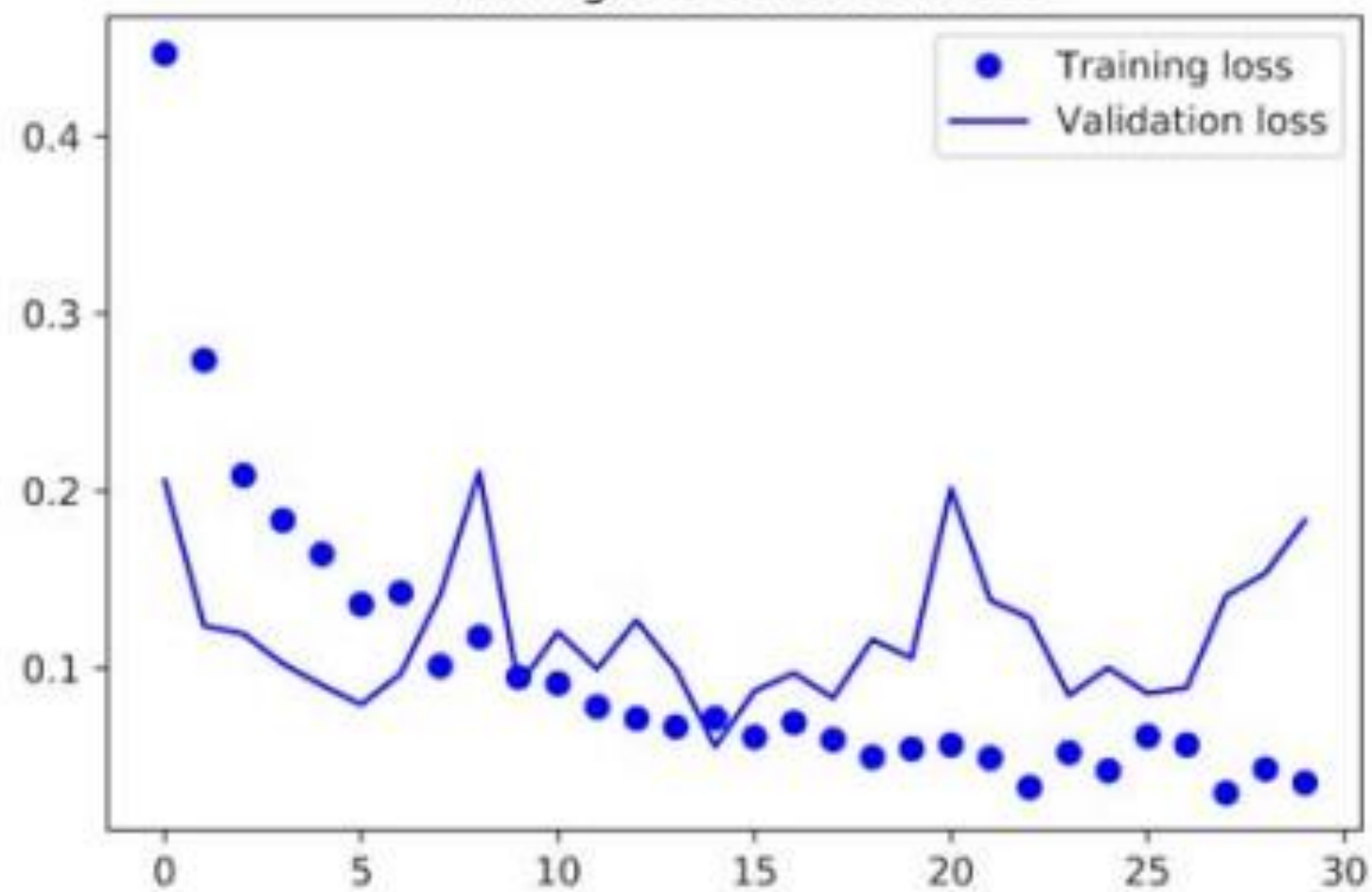
```
>>> model.summary()
```

Layer (type)	Output Shape	Param #
=====		
vgg16 (Model)	(None, 4, 4, 512)	14714688
flatten_1 (Flatten)	(None, 8192)	0
dense_1 (Dense)	(None, 256)	2097408
dense_2 (Dense)	(None, 1)	257
Total params: 16,812,353		
Trainable params: 16,812,353		
Non-trainable params: 0		

Training and validation accuracy



Training and validation loss



Concluding remarks...

- Convnets are the best type of machine-learning models for computer-vision tasks. It's possible to train one from scratch even on a very small dataset, with decent results.
- On a small dataset, overfitting will be the main issue. Data augmentation is a powerful way to fight overfitting when you're working with image data.
- It's easy to reuse an existing convnet on a new dataset via feature extraction. This is a valuable technique for working with small image datasets.
- As a complement to feature extraction, you can use fine-tuning, which adapts to a new problem some of the representations previously learned by an existing model. This pushes performance a bit further.